

**A mesh free method for optimal transport and a binary integer linear
programming solver**

by

Giulio Trigila

A dissertation submitted in partial fulfillment
of the requirements for the degree of
Doctor of Philosophy
Department of Mathematics
New York University
September 2013

Professor Esteban G. Tabak

Acknowledgements

“I’ve seen things you people wouldnt believe” (Blade Runner)

After all these years I spent at Courant, after so many courses, exams and so many people with different backgrounds I have met, these are the words that come into my mind. Unlike the replicant in the movie Blade Runner, though, I will not forget my experiences like tears in rain and will always remember the people that, directly or indirectly, helped me throughout the time as a Ph.D. student at Courant.

First and foremost, I couldnt have chosen a better advisor than Esteban Tabak. From the first day on, he supported me in a way that I think I will hardly experience again in my life. We met regularly every week, starting an exchange of ideas like I have never had before. I always felt comfortable talking to him. He was not only willing to explain me what I couldn’t understand, but most important, he has the rare virtue of *listening* to his students, encouraging them to pursue their own ideas, and patiently showing them what to change when they are wrong. I hope we will keep in touch, not only to continue our collaboration, but also to enjoy Estebans perspective and approach to Math problems and life in general.

Another mentor I could have not renounced to during my Ph.D. was Bob Kohn. He helped me in several critical moments by making the effort to look at the problem not from the perspective of a Full Professor at Courant, but by taking over my view as a Ph.D. student.

His empathy and sensitivity were also reflected in the way he explained Math to me. He always made the effort to show me the solution to a Math problem, from a very practical point of view, trying to emphasize intuition over Math rigor. This was certainly better suited for someone that, like me, was coming from a Physics background.

Other two faculties at Courant that I would like to thank are Henry McKean and Raghu Varadhan. Their classes in complex and real variables represented the first (of many) hard walls I hit during my studies at Courant. They were the first to teach me

something that may seem obvious, but nevertheless took me some time to learn: under layers and layers of bounds, of ϵ 's and δ 's, that you need to come up with when proving a theorem, there are often few ideas, which are most of the time less abstract than what students usually think. Oftentimes, these ideas can be explained with a picture and few steps of Math that, even though they don't constitute a proof of a given theorem, provide a demonstration of it.

Also, I would like to thank my two thesis readers, Robert Kohn and Jonathan Goodman. I bet they will have a hard time trying to understand my Engliano (English mixed with Italiano), and an even harder time making sense of the ideas discussed here. I apologize for both issues!

All the support I received at Courant, was coordinated by Tamar. She always tried to make my life easier, even before my arrival at Courant. For instance, when I was still in Rome, she called from the US to remind me of a document I still had to submit. Her help was not only valuable from the administrative point of view, she also became a close friend. She often organized lunch meetings for students to support their integration at Courant. And she was always open to discuss anything I could possibly think of, from cultural and personal to academic issues.

Naturally, I will never forget all the friends I have met at Courant. I have learned something from each of them. For instance, to mediate between a past-oriented culture, which will always be part of me, and a more practical view of life. In particular, I would like to thank:

Sandra, Klaus, Rebeca, Thomson, Levént (aka Levànt, spiritual son of Mustafa Kemal Atatürk, father of the Turkish revolution), Irena, Enrico, Maddalena, Luca, Marco, Dharshi, Chaitu, Mitch, Steve, Adam, Andres, Muños, Mediñas, Eduardo, Travis, Manas, Sir Oliver, Maria, Carl, Lyuba, Juliana, Paul, Saverio, Yuan Yuan, Xiao Xiao, Lin Lin, Fang Fang.

Chengcheng and Jens Christian Bo Jorgensen, my office mates for the last three

years, deserve special mention. There is so much to say about these two that it would take too long to explain it here. Probably, it is enough to mention that the only time I have missed a flight in my life was with Jens, and that Chengcheng promised to lead the next Chinese revolution.

I would also like to thank Susanne, my (very German) girlfriend. Besides that I received a lot of support from her, mainly through shocks of the kind you may expect when a goal-oriented German meets an idealistic Italian, I need to thank her for her infinite reservoir of patience.

Last but not least, I would like to thank my family and, in order to stress a stereotype that otherwise would occur anyway, the last word is to thank la mia Mamma.

Abstract

Optimal transport (OT) theory and binary integer linear programming (BILP) are two of the major topics in optimization theory. In this thesis we will analyze these two topics from the numerical prospective providing two algorithms to compute their solution.

These two branches of optimization shares many common features from the mathematical point of view. In particular OT and BILP can be viewed as two different instances of a linear programming problem.

While BILP, having as a solution an n -dimensional vector with entries that can assume only the value 0 or 1, is intrinsically discrete, OT admits also a continuous formulation. In this context solving an OT problem means to find the least costly mass preserving map, mapping a starting probability density function (PDF) into a target one. Because in this thesis we will focus on this continuous aspect of OT, we will treat OT and BILP separately.

Numerical methods for continuous OT are mainly based on partial differential equations type of techniques. If on the one hand this allows for an accurate convergence analysis, on the other hand these techniques are based on spatial grids that limit their use to spaces of dimension smaller than about three.

In this thesis we propose a mesh-free algorithm for OT using data sets sampled from the starting and the target PDF respectively. This extends the use of OT to study data sets defined in high dimensional spaces, for instance in biological applications.

Regarding BILP the algorithm proposed here is based on the so called branch and bound strategy. The main difference with other approaches using the same strategy is that in our case we avoid to solve a relaxed linear programming problem (usually performed with simplex or interior point method) at each node of the binary search tree. The algorithm is instead based on an event driven kind of dynamics used to optimize the Lagrangian through which we reformulate the problem. Numerical tests are performed

to compare our approach to the matlab solver `bintprog`.

Contents

Acknowledgements	iii
Abstract	vi
List of Figures	x
Introduction	1
1 Optimal Mass Transport Problem	5
1.1 Description of the problem	5
1.2 Dual formulation and quadratic cost assumption	7
1.3 The unconstrained problem	10
1.4 Existing approaches	14
2 Construction of the optimal map via maximization of the log-likelihood	16
2.1 Maximizing of the log-likelihood	17
2.2 Compact support and boundary conditions	29
2.3 Numerical experiments	32
3 A numerical scheme based on the Kantorovich optimal transport problem	51
3.1 Introductory considerations	52
3.2 A Primal-Dual Approach	55
3.3 Numerical results	69

4	An event driven approach to binary integer linear programming	82
4.1	Linear programming, inequality constraints	83
4.2	The algorithm	84
4.3	Algorithm refinement and proof of convergence	86
4.4	Extension: equality constraints	90
4.5	Binary integer linear programming, an event driven approach	91
5	Conclusion	100
A	Linear Programming	103
A.1	Theorem of the alternative	103
A.2	Duality	105
A.3	A particular case of LP: discrete optimal transport	109
	Bibliography	111

List of Figures

2.1	Extension of the map	26
2.2	Elementary map	28
2.3	Translation of a Gaussian, 1D	34
2.4	Translation 1D, convergence of the map	35
2.5	Translation 1D, convergence of the cost	35
2.6	Cubic map, 1D	36
2.7	Cubic map 1D, convergence of the map	37
2.8	Cubic map 1D, convergence of the cost	37
2.9	First example with compact support	39
2.10	First example with compact support. Convergence of the map and cost	40
2.11	Second example with compact support	40
2.12	Second example. Convergence of the map and the cost	41
2.13	Mass splitting	41
2.14	Mass splitting. Convergence of the map and the cost	42
2.15	Translation of a 2D Gaussian distribution	43
2.16	Translation of a 2D Gaussian distribution. Log-likelihood and cost	44
2.17	Translation of a 2D Gaussian distribution. Map convergence	44
2.18	Cubic potential in 2D	45
2.19	Cubic potential in 2D. Log likelihood and cost.	46
2.20	Cubic potential in 2D. Map convergence	47

2.21	2D compact support.	49
2.22	2D compact support. Log likelihood and cost	49
2.23	2D compact support. Map convergence	50
3.1	Pseudospectral method: Time evolution of the density	59
3.2	Divergence and curl of the displacement field	61
3.3	Projection step.	69
3.4	Non isotropic Gaussian into a non isotropic Gaussian.	73
3.5	Non isotropic Gaussian into a non isotropic Gaussian: Convergence of the map.	74
3.6	Two dimensional mass splitting.	76
3.7	Two dimensional mass splitting: Evolution of the map	77
3.8	Projection vs non projection	77
3.9	Compact support on uniform square	78
3.10	Multi-dimensional example: Densities	80
3.11	Multi-dimensional example: Convergence	81
3.12	Average time per iteration	81
4.1	Convergence LP algorithm	90
4.2	Binary tree	92
4.3	Binary integer linear programming	98

Introduction

This thesis is organized in two main parts regarding optimal transport and binary integer linear programming respectively. The two topics are considered from the numerical point of view and, even though they share some common features, they are treated separately. This distinction is reflected in the introduction, where we first discuss the motivation that leads us to formulate the algorithms relative to optimal transport, and then that one relative to the algorithm for the solution of a binary integer linear programming problem.

The general problem of finding a one-to-one correspondence between two probability density functions has many applications in a variety of fields of applied Mathematics and Physics.

The first example, is that one regarding density estimation. Let $\rho(x)$ and $\mu(x)$ ($x \in \mathbb{R}^n$) be respectively the *starting* and the *target* probability densities and $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$ the function mapping $\rho(x)$ into $\mu(x)$. In [28] it is shown how, through a normalization procedure, it is possible to determine T when the analytic form of $\mu(x)$ is known and only a sample of data drawn from $\rho(x)$ is available.

Density estimation is one of the fundamental challenges in statistics and, generally speaking, it is important when the information we are interested in, is not directly accessible by computing a finite set of the empirical moments from the data sampled from $\rho(x)$.

Let's suppose, for instance, that we want to estimate the probability that the results of a given medical test are associated with a given population of patients having a certain disease. In this case it is preferable to have an estimate of the complete distribution (rather than just few moments of it) characterizing the results of the patients affected by the disease. In [1] density estimation is used to classify tumors from gene expression data.

Mapping a probability density into a different one is also used in computer vision. An

image can be thought as a probability density if we establish a correspondence between the color scale and the different values of the probability density. The process of changing an image into another one in a seamless transition is called image morphing and is often used in computer vision to create, for instance, slow motion effects. Image morphing is often achieved by iterative algorithms that at each step gradually transform two given images one into the other.

If the map between the two densities is optimal in the sense of optimal transport theory (see Chapter 1) then there are much more applications. In this framework, among all the possible maps between the two densities, we look for the unique, mass preserving map minimizing a given convex functional.

In imaging analysis, optimal transport is often used for imaging registration where different sets of data are transformed into one common frame of reference. In medical application this technique can be used, for instance, to characterize the growth of a tumor [25]. In particular the problem is to quantify the change in shape of a tumor by analyzing two MRI images of it taken, for example, before and after a given therapy. Because of the uniqueness of the optimal map between the two images and its mass preserving nature, this map is well suited to characterize the displacement field of the tumor tissue.

Optimal transport theory finds additional applications in atmospheric sciences [6], astrophysics [12] and finance [8]. A complete list of topics relevant to optimal transport can be found in [30] and a brief introduction to the topic to support the material of this thesis, is given in Chapter 1.

In Chapters 2 and 3 we will formulate a class of iterative methods for mapping a probability density into another one. In doing so, we will have in mind two main goals:

- Since the kind of application we are interested in is mostly related to data mining and statistics, we want the algorithm to perform well when dealing with data defined in high dimensional spaces. This is often the case, for instance, in medical

or biophysical applications.

- Since finding the optimal map, rather than any map, between two densities offers a much wider range of applications, we will pay attention to design algorithms such that the map from the starting to the target density is optimal.

In particular in Chapter 2 we will present a first method for the computation of the optimal map. In implementing this method we assume the knowledge of the target distribution $\mu(x)$, its first derivative, and a data sample from the starting distribution $\rho(x)$. In Chapter 3 we will propose an alternative method that requires only the knowledge of data samples drawn from the starting and the target densities respectively. The performance of both algorithms is tested on artificial data sets.

The philosophy underlying both algorithms is to find the map from the starting density $\rho(x)$ to the target $\mu(y)$ by using discrete sampled data points without forgetting that they are drawn from continuous probability density distributions. This is somehow in contrast with the pure combinatorial approach used to find correspondence between two discrete data sets (of equal number) minimizing a given cost function, and the continuous approach in which $\rho(x)$ and $\mu(y)$ are known pointwise and the map from $\rho(x)$ to $\mu(y)$ is usually found introducing a grid in space to compute the solution of some kind of partial differential equation.

Chapter 4, is dedicated to a new, to the best of our knowledge, branch and bound kind of algorithm for solving binary integer linear programming (BILP) problems. BILP is ubiquitous in applications, ranging from assignment problems to network optimization and in general to all those linear programming problems that are subject to a “yes or no” kind of solution. In these problems we need to optimize a linear objective function subject to linear constraints, requiring that each entry x_i of the solution x satisfies $x_i \in \{0, 1\}$. The work described in Chapter 4 originated from a problem that was realized in collaboration with the Department of Education of New York City.

The goal of the project was to award contracts to different bus companies in order

to offer a Pre-Kindergarden bus service over the New York City area. This had to be done with the aim of minimizing the overall cost for the department of education. To accomplish this task the city area was divided in 43 aggregates. Each one big enough to offer a reasonable choice in terms of schools a child could apply to. A given number of buses was assigned to each area in order to ensure good service in terms of frequency of the buses, adequate number of users and time spent by each user in the bus on their route to school. The assignment process was subject to the following constraints:

1. Each aggregate has to be covered.
2. The total number of zones assigned to each bus company must be consistent with the maximum number of buses available to that company.
3. It is not possible for two bus companies to share the same aggregate.

From the mathematical point of view the problem above was reformulated in a BILP problem that was solved with an ordinary branch and bound approach where at each node of the binary search tree we solved a relaxed problem via the simplex method. While doing this, we speculated on whether it was possible to avoid solving a linear programming problem at each node of the tree and the result of this speculation is presented in the algorithm formulated in Chapter 4. In this chapter we provide a detailed description of the algorithm together with some numerical tests showing that, at least for a given range of sizes of the BILP problem, the algorithm is competitive when compared to the matlab solver `bintprog`.

Chapter 1

Optimal Mass Transport Problem

In this chapter we will provide some background material that will be used to develop the numerical methods described in Chapters 2 and 3. The main references for this chapter are [7], [11], [16] and [9].

1.1 Description of the problem

The optimal transport problem was formalized by Monge in 1781 and deals with transporting a given mass of a given material, let's say sand, from a location A to a location B such to minimize a cost to be defined and conserving the total mass of the material.

By setting the total mass of sand to one, we can model the initial and the final mass distribution of the material with two probability measures $d\rho(x)$ and $d\mu(y)$. In this thesis we will always assume that these two measures are absolutely continuous with respect to the Lebesgue measure, namely they have probability densities $\rho(x)$ and $\mu(y)$ with support respectively K_1 and K_2 both $\in \mathbb{R}^n$. In this context we can state the Monge transport problem as follows:

Given $\rho(x)$ and $\mu(y)$ two probability densities with $x, y \in \mathbb{R}^n$, find the map $y = y(x) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ that transports the density $\rho(x)$ to $\mu(y)$ minimizing the cost functional

$$M(y) = \int_{R^n} c(y-x)\rho(x)dx. \quad (1.1)$$

In order to transport $\rho(x)$ to $\mu(y)$ the map is required to satisfy the relation

$$\int_{y^{-1}(K_1)} \rho(x)dx = \int_{K_1} \mu(y)dy \quad (1.2)$$

that if y is one-to-one and smooth is equivalent to

$$\rho(x) = \mu(y(x))J^y(x) \quad (1.3)$$

where $J^y(x)$ is the Jacobian of the map, namely $J^y(x) = \det(\nabla y(x))$. A map that satisfies eq. (1.3) and at the same time minimizes $M(y)$ is called an *optimal map*.

The problem described so far requires that all the mass at point x is mapped to a single point $y(x)$. This may be an issue if for instance we consider the example in which the starting and the target measure are given by

$$d\rho(x) = \delta(x-x_0), \quad d\mu(y) = \frac{\delta(x-x_1) + \delta(x-x_2)}{2}. \quad (1.4)$$

Since the map $y(x)$ would require to map x_0 to two different points x_1 and x_2 , in this case, the Monge problem does not have any solution.

Kantorovich in the 1940's proposed a relaxed version of the Monge transport problem where the mass at a given point y may come from a combination of different points x in the support of the starting probability $\rho(x)$.

In this case the way in which ρ should be rearranged in order to obtain μ is described by a joint probability density $\pi(x, y)$ specifying how much of $\rho(x)dx$ goes into $\mu(y)dy$. The density $\pi(x, y)$ must be such that

$$\rho(x) = \int_{K_2} \pi(x, y)dy, \quad \mu(y) = \int_{K_1} \pi(x, y)dx \quad (1.5)$$

and the problem is now to minimize the functional

$$F(\pi) = \int c(y - x)\pi(x, y)dxdy \quad (1.6)$$

over all the $\pi(x, y)$ satisfying eq. (1.5). A minimizer of (1.6) subject to (1.5) is called *optimal plan*.

There are two considerations to be done:

- In the Kantorovich sense the class of possible solutions admits always at least one plan, given by $d\pi(x, y) = d\rho(x)d\mu(y)$, mapping the starting probability distribution to the target one.
- Because the graphs of one-to-one functions can be viewed always as a positive measure in the product space where the starting and the target measure are defined, it's clear that the class of possible maps in the Monge problem is contained in the class of possible maps of the Kantorovich problem.

In particular, in [22] is shown that if $\rho(x)$ and $\mu(y)$ have finite second moment and $c(y - x)$ is a strictly convex function of its argument $y - x$, then the Kantorovich problem and the original Monge problem have the same, unique solution in the sense that

$$\min M(y) = \min F(\pi) \quad (1.7)$$

and

$$\pi(S) = \rho(\{x \in K_1 : (x, y(x)) \in S\}). \quad (1.8)$$

1.2 Dual formulation and quadratic cost assumption

The problem described by the equations (1.5) and (1.6) can be seen as the continuous version of a category of linear programming problems called “assignment problem” (see Section A.3 of Appendix A). In the same way every linear programming problem is

characterized by its dual formulation, the Kantorovich problem admits the following equivalent form:

$$\text{Maximize} \quad L(u, v) = \int_{K_1} u(x)\rho(x)dx + \int_{K_2} v(y)\mu(y)dy \quad (1.9)$$

over all the possible continuous function u and v satisfying

$$u(x) + v(y) \leq c(y - x). \quad (1.10)$$

The problem described by equations (1.9) and (1.10) is dual with respect to the problem described by (1.5) and (1.6) in the sense that $\max L(u, v) = \min F[\pi]$.

In this and in the following sections we will suppose $c(y - x) = \frac{|y-x|^2}{2}$. In this case, after redefining $u(x) \rightarrow \frac{x^2}{2} - u(x)$ and $v(y) \rightarrow \frac{y^2}{2} - v(y)$, the dual problem becomes:

$$\text{Minimize} \quad L(u, v) = \int_{K_1} u(x)\rho(x)dx + \int_{K_2} v(y)\mu(y)dy \quad (1.11)$$

over all continuous u and v subject to

$$u(x) + v(y) \geq xy. \quad (1.12)$$

This substitution is useful for using techniques from convex analysis in order to prove (see [11] for a proof) the following proposition:

Proposition 1.1. *The functional $L(u, v)$ admits a unique couple of minimizing functions $(\bar{u}, \bar{v})$ which are convex conjugates*

$$\begin{cases} \bar{u}(x) = \max_{y \in K_2} (xy - v(y)) \equiv v^*(x) \\ \bar{v}(y) = \max_{x \in K_1} (xy - u(x)) \equiv u^*(y) \end{cases} \quad (1.13)$$

and in addition the optimal plan for the Monge problem is given by

$$y(x) = \nabla \bar{u}(x). \quad (1.14)$$

The function $u^*(y) \equiv \max_{x \in K_1} (xy - u(x))$ is the Legendre transform of u where the domain over which $xy - u(x)$ is maximized, is restricted to K_1 rather than all $\mathbb{R}^n$. Because $\bar{u}$ is obtained as the envelope of its supporting lines, $\bar{u}$ is a convex function of its argument.

Proposition (1.1), together with equation (1.7), is saying that if we find a convex function $\phi : \mathbb{R}^n \rightarrow \mathbb{R}$ satisfying

$$\rho(x) = \mu(\nabla \phi(x)) \det(D^2 \phi)(x), \quad (1.15)$$

then $\nabla \phi$ is the optimal map between $\rho(x)$ and $\mu(y)$. The relation (1.15) is called *Monge-Ampere equation*. In Chapter 2 we will use this observation to compute a numerical approximation to the optimal plan. In particular, we will find a procedure to map ρ into μ making sure that the computed map is the gradient of a convex potential.

To have an idea about the geometry of optimal transport and in particular why the potential ϕ has to be convex, it is worth to consider the discrete case of the Monge problem. Suppose we need to find a one-to-one correspondence (or assignment) between a set of N points $\{x_i\}$ and another set of N points $\{y_i\}$ so to minimize the overall quadratic cost

$$\sum_{i=1}^N |y_i - x_i|^2. \quad (1.16)$$

In this case finding the optimal map reduces to a combinatorial problem. Because we are considering only finitely many points there exists always a solution y such that

$$\sum_{i=1}^N |y(x_i) - x_i|^2 \leq \sum_{i=1}^N |y(x_{\pi_i}) - x_i|^2 \quad \forall \pi \quad (1.17)$$

where π is a generic permutation of the entries of the vector labeling the points x_i . Rewriting the norms as inner products and rearranging the terms, the eq. (1.17) becomes equivalent to

$$\sum_{i=1}^N \langle y(x_i) - y(x_{\pi_i}), x_j \rangle \geq 0 \quad \forall \pi. \quad (1.18)$$

The relation (1.18) is called *cyclical monotonicity*. If we consider the set of permutations swapping only two indexes then condition (1.18) reduces to

$$\langle y(x_i) - y(x_j), x_i - x_j \rangle \geq 0 \quad \forall i, j = 1 \dots N. \quad (1.19)$$

A map y satisfying (1.19) is called a *monotone* map.

It is possible to prove (see [11]) that even when dealing with continuous optimal transport, an optimal map has to be cyclically monotone and therefore monotone. It is also not hard to prove (for a proof see for instance [24]) that if a monotone map $y(x) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the gradient of some function $\phi(x) : \mathbb{R}^n \rightarrow \mathbb{R}$, then ϕ has to be a convex function.

1.3 The unconstrained problem

Reformulating the Kantorovich problem in terms of equations (1.11) and (1.12) is not only useful for proving proposition (1.1), but can also be used to find numerical approximations to the optimal map. In [9] the problem in (1.11) is related with the unconstrained minimization of the following functional

$$\tilde{L}(u) = \int_{K_1} u(x) \rho(x) dx + \int_{K_2} u^*(y) \mu(y) dy. \quad (1.20)$$

where it is easy to show that, by definition, the pair of functions $u(x)$ and $u^*(y)$ satisfies $u(x) + u^*(y) \geq xy$. In particular the solution of the minimization of L and $\tilde{L}$ are characterized by the following theorem:

Theorem 1.2. *The functional $\tilde{L}$ is convex, Lipschitz continuous and has a unique convex minimizer $\bar{u}$ whose gradient represents the solution to the Monge problem. Furthermore*

$$\frac{\delta \tilde{L}}{\delta u}(u) = \rho - \mu(\nabla u^{**}) \det(D^2 u^{**}). \quad (1.21)$$

Remark 1. It is possible to show that u^{**} is convex and so almost everywhere differentiable (see [24]) but it is not true in general that u^{**} has two derivatives, therefore the Hessian in eq. (1.21) may not be well defined. A way of overcoming this fact is to define $D^2 u^{**}$ in the Alexandrov sense (see for instance [14]).

Remark 2. If the variational derivative of $\tilde{L}$ is computed at a point u which is the conjugate of some function ($u = \phi^*$ for some ϕ) then $u^{**} = u$ and by setting (1.21) equal to zero we have that the minimizer of $\tilde{L}$ satisfies the Monge-Ampere equation.

The proof of theorem (1.2) is due to Gangbo [16] and is the same reported in [9]. Here we will only sketch the main aspects of it referring to [16] and [9] for more details. We will first show the convexity of $\tilde{L}$ and then prove equation (1.21).

Since the first integral defining $\tilde{L}$ is linear in u , in order to show the convexity of $\tilde{L}$ we just need to show that the operation $u \rightarrow u^*(y)$ is convex:

$$\begin{aligned} (tu_1 + (1-t)u_2)^*(y) &= \max_x (x \cdot y - tu_1(x) - (1-t)u_2(x)) \\ &\leq \max_x t(x \cdot y - u_1(x)) + \max_x (1-t)(x \cdot y - u_2(x)) = tu_1^*(y) + (1-t)u_2^*(y). \end{aligned} \quad (1.22)$$

We start to prove eq. (1.21) by computing the derivative of $\tilde{L}$ in the direction of a generic continuous function v :

$$\begin{aligned} \delta \tilde{L} &= \lim_{h \rightarrow 0^+} \frac{\tilde{L}(u + hv) - \tilde{L}(u)}{h} = \\ &= \int_{K_1} v(x) \rho(x) dx + \int_{K_2} \lim_{h \rightarrow 0} \frac{(u + hv)^*(y) - (u)^*(y)}{h} \mu(y) dy \end{aligned} \quad (1.23)$$

If we define

$$\begin{cases} x_0 = \arg \max_{x \in K_1} (x \cdot y - u(x)) \\ x_h = \arg \max_{x \in K_1} (x \cdot y - u(x) - hv(x)) \end{cases} \quad (1.24)$$

then we can rewrite the numerator of the integrand function in the second integral of (1.23) as

$$(u + hv)^*(y) - (u)^*(y) = x_h \cdot y - u(x_h) - hv(x_h) - (x_0 \cdot y - u(x_0)). \quad (1.25)$$

Using (1.25) together with the definitions given in (1.24), we can bound from above and below the term in (1.25) as follows:

$$\begin{aligned} x_0 \cdot y - u(x_0) - hv(x_0) - x_0 \cdot y + u(x_0) &\leq (u + hv)^*(y) - (u)^*(y) \\ &\leq x_t \cdot y - u(x_t) - hv(x_t) - x_t \cdot y + u(x_t) \end{aligned} \quad (1.26)$$

After rearranging the inequalities in (1.26) we obtain

$$0 \leq \frac{x_h \cdot y - u(x_h) - hv(x_h) - (x_0 \cdot y - u(x_0))}{h} + v(x_0) \leq v(x_0) - v(x_h). \quad (1.27)$$

We can now use (1.27) to assert that

$$\lim_{h \rightarrow 0} \frac{x_h \cdot y - u(x_h) - hv(x_h) - (x_0 \cdot y - u(x_0))}{h} = -v(x_0). \quad (1.28)$$

In order to do so we suppose that x_0 is the unique maximizer of $x \cdot y - u(x)$ (this is true if ∇u is a one-to-one map) and notice that, since u and v are continuous, we have that $\lim_{h \rightarrow 0} x_h = x_0$. This implies that $\lim_{h \rightarrow 0} v(x_h) - v(x_0) = 0$ and, by (1.27), we get (1.28).

From the considerations above we can rewrite the variation (1.23) as

$$\delta \tilde{L} = \int_{K_1} v(x) \rho(x) dx - \int_{K_2} v(\nabla u^*(y)) \mu(y) dy \quad (1.29)$$

where we used the fact that $x_0 = \nabla u^*(y)$ and that u^* is differentiable almost everywhere since it is a convex function.

By means of the change of variable $y = \nabla u^{**}(x)$ we can rewrite (1.29) as

$$\delta \tilde{L} = \int_{K_1} v(x) \rho(x) dx - \int_{(\nabla u^{**})^{-1}(K_2)} v(\nabla u^*(\nabla u^{**}(x))) \mu(\nabla u^{**}(x)) \det(D^2 u^{**})(x) dx \quad (1.30)$$

The change of variable in (1.30) is justified by the work of McCann [22]. Here we will observe that

1. Using the cancellation property of the Legendre transform we obtain that

$$\nabla u^*(\nabla u^{**}(x)) = x \quad (1.31)$$

2. By definition we have that

$$u^{**}(x) = \max_{y \in K_2} (x \cdot y - u^*(y)) \quad (1.32)$$

Supposing that the maximizer y_0 of $x \cdot y - u^*(y)$ is unique, and considering that the condition for an extremal is $\nabla u^*(y_0) = x$, each point of $x \in K_1$ is mapped to a point $y \in K_2$ by the map ∇u^* . Because ∇u^* is the inverse of ∇u^{**} the formal relation $(\nabla u^{**})^{-1}(K_2) = K_1$ holds.

The second point indicates that we can combine the two integrals in (1.30) to obtain

$$\delta \tilde{L} = \int_{K_1} [\rho(x) - (\nabla u^{**}(x)) \mu(\nabla u^{**}(x)) \det(D^2 u^{**})(x)] v(x) dx \quad \forall v \in \mathbb{C}(K_1) \quad (1.33)$$

that is what is stated by (1.21).

Finally, the assertion that the gradient of the unique minimizer $\bar{u}$ of $\tilde{L}$ represents the optimal map, is justified by the fact that the pair of functions $(\bar{u}, \bar{u}^*)$ are convex conjugates like the pair $(\bar{u}, \bar{v})$ minimizing L and defined in proposition (1.1).

1.4 Existing approaches

Existing algorithms to compute numerical solutions of the L^2 Kantorovich problem can be divided essentially in three different categories.

The first category includes algorithms that solve directly the Monge-Ampere equation via a suitable finite difference scheme that enforces the convexity of the solution (see [15] and [14]). If on one hand this approach allows for an accurate convergence analysis of the numerical solution towards the weak solution of the equation, on the other hand the computational cost grows exponentially with the number of dimensions of the domain in which the equation is defined.

A second category deals with algorithms that are based on iterative methods either used to minimize a given functional or to find the steady state solution of specific PDE's. Examples are [27] and [2]. In this category it is important to mention the work described in [3]. In this paper optimal transport is reformulated according to a fluid mechanics point of view in which the optimal plan is the minimizer of a given action subject to certain constraints. Such minimization is performed numerically using a saddle point method.

The approaches described above require a spatial grid to be implemented. This does not only affect the computational cost of the algorithm but represents a limit if, for instance, we have in mind a data mining kind of application. In this context the only information available about ρ and μ are data sampled by these two densities.

The third kind of approach, more similar to the spirit of what is discussed in this thesis, is followed by [19]. In this case the use of a mesh is avoided by approximating ρ

and μ with a Gaussian mixture. By using an opportune Lagrangian method the particles are mapped as so to reproduce the optimal plan.

Chapter 2

Construction of the optimal map via maximization of the log-likelihood

In this chapter, we will adapt the iterative method introduced in [28] to provide a numerical approximation of the optimal map between two probability densities.

The algorithm presented in this chapter belongs to the category of mesh-free algorithms. As already noted in the introduction of this thesis, this is a fundamental feature that allows to handle cases in which the probability densities are defined in spaces with dimension bigger than three.

The input data for the algorithm are a sample of points $\{x_i\}$ drawn from the initial probability density $\rho(x)$ and the analytical form of the target density $\mu(y)$. Analogously to what is done in [28], using the Kullback-Leibler (KL) divergence as objective function, the points $\{x_i\}$ are mapped such that their distribution in space corresponds to that of points sampled from the prescribed target density $\mu(y)$. What is discussed in this chapter is how to modify this approach so that the resulting map is optimal in the sense described in Chapter 1.

In [28] the idea of mapping a density $\rho(x)$ to another known density was used to perform density estimation on a given set of data. In particular, by computing any map from $\rho(x)$ to $\mu(y)$ and by the knowledge of $\mu(y)$, it is possible to estimate the density $\rho(x)$.

We will keep the same philosophy here showing numerical experiments in which we will estimate the density underlying a given set of data, together with the optimal map from $\rho(x)$ to $\mu(y)$.

2.1 Maximizing of the log-likelihood

Following the idea in [28] we would like to compute the optimal map $y(x)$ iteratively so that, as the iteration step k advances, the numerical approximation $y^k(x)$ of the optimal map $y(x)$ maps the starting density “closer and closer” to the target distribution.

In particular, given the numerical approximation y^k at the k -th iteration step, the approximation at the $k + 1$ iteration is found by minimizing the Kullback-Leibler divergence

$$KL[y^k] = \int \rho(x) \log \left(\frac{\rho(x)}{\rho^k(x)} \right) dx \quad (2.1)$$

computed in between ρ and the density $\rho^k(x)$ defined as

$$\rho^k(x) = \mu(y^k(x)) J^{y^k}(x). \quad (2.2)$$

In [29] it is shown that, in the time continuous case, by evolving the map according to the variational derivative of KL

$$\frac{dy_t(x)}{dt} = - \left. \frac{\delta KL[\phi \circ y_t]}{\delta \phi} \right|_{\phi=id} \quad (2.3)$$

the density $\rho_t(x) = J^{y_t}(x) \mu(y_t(x))$ (the time continuous analog of ρ^k) converges to ρ .

In particular we have that

$$\frac{d}{dt}KL[y_t] \leq 0 \quad (2.4)$$

with equality only if $\rho_t = \rho$.

This suggests that the procedure above will, if the family of maps from which y^k is chosen is rich enough, converge to a probability density as close to ρ as desired or, equivalently, that $KL[y^k]$ can be made as small as desired by taking k large enough and the allowed family of maps rich enough.

From the implementation point of view minimizing the KL in (2.1) is equivalent to maximizing the functional

$$L[y^k] = \int \rho(x) \log(\rho^k(x)) dx. \quad (2.5)$$

In the following we will describe in more details how to update $y^k(x)$ at each iteration step. The goal is to obtain a map that:

1. Maps the probability density function $\rho(x)$ into $\mu(x)$.
2. Is the gradient of some convex function we will often refer to with the name of potential function.

Because of proposition (1.1), if (1) and (2) are achieved (exactly) then $y^\infty(x)$ is indeed the optimal plan transporting $\rho(x)$ to $\mu(x)$.

The idea is to approximate the optimal plan $y(x)$ with a sum of maps each one a gradient of some elementary potential function $F(x; \beta)$ depending on a parameter β :

$$\begin{cases} y(x^0) \approx y^k(x^0) = x^k = x^0 + \sum_{i=1}^k \nabla_{x^0} F(x^0; \beta^i) = x^{k-1} + \nabla_{x^0} F(x^0; \beta^k) \\ y^0(x^0) = x^0 \end{cases} \quad (2.6)$$

The parameter β^k is chosen at each time step so to maximize $L[y^k]$. Because $L[y^k]$

is defined as the average of $\log(\rho(x)^k)$ with respect to the probability density $\rho(x)$, it is natural to represent $L[y^k]$ in terms of its Monte Carlo estimate, yielding to the log-likelihood

$$L[y^k] \approx LL^k = \frac{1}{m} \sum_{i=1}^m \log[\mu(x_i^k) J^k(x_i)] \quad (2.7)$$

where we used the short notation J^k for J^{y^k} . In this way, we can reach our purpose of not using the pointwise knowledge of ρ by maximizing LL^k rather than $L[y^k]$.

It is important to notice that the error between LL^k and $L[y^k]$ in principle is proportional to $1/\sqrt{m}$ (where m is the number of sampled points). Even if in practice the approximation becomes less and less accurate as the dimension of the space increases¹, using Monte Carlo estimates is one of the key ingredients for this algorithm to perform well in a space with a dimension bigger than about three.

Once we introduce (2.7), the goal becomes to update the position of the sampled points in order to obtain $LL^{k-1} \leq LL^k$. This can be done choosing the parameter β^k in (2.6) to be proportional to the gradient of LL^k

$$\beta^k \propto \left. \frac{\partial LL^k}{\partial \beta^k} \right|_{\beta^k=0}. \quad (2.8)$$

Because of the simplicity of each elementary map, rather than just following the gradient of LL , it's convenient to implement the Newton method by choosing β at each time step such to maximize the quadratic approximation of the quantity defined in (2.7)²:

$$LL \approx LL_0 + G\beta + \frac{1}{2}\beta^T H \beta \quad (2.9)$$

where

$$LL_0 = LL|_{\beta=0}, \quad G_i = \left. \frac{\partial LL}{\partial \beta_i} \right|_{\beta=0}, \quad H_{ij} = \left. \frac{\partial^2 LL}{\partial \beta_i \partial \beta_j} \right|_{\beta=0}. \quad (2.10)$$

¹As a simple metric here, the Monte Carlo error scales with the standard deviation of $L[y^k]$

²In this expression we are considering β as a vector of parameters

Maximizing (2.9) with respect to β yields

$$\beta = -H^{-1}G. \quad (2.11)$$

The value of β is always limited from above by a given step tolerance. It may happen in fact that the quantity LL is locally very flat causing the Hessian H to be a singular matrix and resulting in a diverging value of β . In order to avoid this, at each iteration step we impose a threshold, called *learning rate*, on the modulus of β .

2.1.1 The choice of the elementary potential

In order to have the algorithm performing well in high dimensional spaces it is convenient to choose the elementary potential F to depend from x^0 in a radial symmetric way. Our choice is to have $F(x^0; \beta) = \beta F(r)$ with $r = |x^0 - \tilde{x}|$ where $\tilde{x}$ is a point chosen randomly at each iteration step among the sampled initial data $\{x_i^0\}$.

With this choice eq. (2.6) takes the form

$$x^k = x^{k-1} + \beta^k \nabla_{x^0} F(r^k) = x^{k-1} + \beta^k f(r^k)(x^0 - \tilde{x}^k) = x^0 + \sum_{i=1}^k \beta^i f(r^i)(x^0 - \tilde{x}^i) \quad (2.12)$$

where $f(r) = \frac{1}{r} \frac{dF(r)}{dr}$. The position of the point x^{k-1} at iteration $k-1$ is updated by moving it by the amount $\beta^k f(r^k)(x^0 - \tilde{x}^k)$, that is a radial compression or expansion (depending on the sign of β^k) centered at $\tilde{x}^k$. We can rewrite (2.12) from the point of view of the potential $\phi^k(x^0)$ by integrating eq. (2.12)

$$\phi^k(x^0) = \frac{(x^0)^2}{2} + \sum_{i=1}^k \beta^i F(|x^0 - \tilde{x}^i|). \quad (2.13)$$

where $\nabla \phi^k(x^0) = x^k$.

Given the equation (2.13) it is crucial to make the following observation:

We are computing the coefficients β^i by maximizing the quadratic approximation of the

log-likelihood. While this is, in principle, sufficient for computing a map transporting ρ to μ , we cannot guaranteed that the potential ϕ^k remains a convex function for each iteration step k . This is due to the fact that, because β is not always postive, we are adding convex and concave functions obtaining as a result a function that may not be convex.

Since the sum of convex functions is still a convex function, in principle we could enforce the convexity of ϕ^k by making sure that each term of the sum in (2.13) is a convex function obtaining therefore a convex function as a result.

If on the one hand we have the freedom of choosing $F(|x^0 - \tilde{x}|)$ to be convex in the variable x^0 , on the other hand we cannot choose a priori the sign of the coefficients β^i computed maximizing the log-likelihood. It seems in fact to restrictive to constraint β to non negative values. It may be possible in fact that the density ρ^k ends up in a configuration from which, by following the gradient of KL , it is very hard to go out by means of a gradient of a convex function.

In the next subsection (2.1.1.1), we will describe how to deal with this problem using a rejection strategy. In the remaining part of this section we will instead focus on the properties that $F(|x^0 - \tilde{x}|)$ should have in order to make sense of the maximization of the quadratic approximation described in eq. (2.9).

Since we want to maximize the quadratic approximation of LL we need to be sure that the hessian H defined in (2.10) remains negative definite at each iteration step. In order to determine the conditions under which this is true, we start by taking the gradient with respect to x^0 on both sides of (2.12):

$$\begin{aligned}\nabla y^k(x^0) &= \nabla y^{k-1}(x^0) + \beta^k \left[I f(r^k) + \frac{1}{r^k} \frac{df(r^k)}{dr^k} (x^0 - \tilde{x}^k) \otimes (x^0 - \tilde{x}^k) \right] \\ &\equiv \nabla y^{k-1}(x^0) + \beta^k P^k(x^0)\end{aligned}\tag{2.14}$$

where $\nabla y^0(x) = I$ with I the identity matrix. In the following we will indicate the

Jacobian matrix $\nabla y^k(x_i)$ with the symbol $\mathbf{J}_i^k$, while its determinant with the symbol J_i^k . In addition we will use the short notation P_i^k and f_i^k to indicate $P^k(x_i^0)$ and $f(r_i^k)$ respectively.

Substituting eq. (2.14) in eq. (2.7) and taking first and second derivative with respect to β we obtain

$$\begin{aligned} G^k &= \sum_{i=1}^m \left[\frac{\partial \log(J_i^k)}{\partial \beta^k} + \frac{\partial \log(\mu(x_i^k))}{\partial \beta^k} \right]_{\beta^k=0} = \\ &= \sum_{i=1}^m \text{tr} \left((\mathbf{J}_i^{k-1})^{-1} P_i^k \right) - f_i^k \langle x_i^k, (x_i^0 - \tilde{x}^k) \rangle \end{aligned} \quad (2.15)$$

and

$$H^k = \sum_{i=1}^m -\text{tr} \left((\mathbf{J}_i^{k-1})^{-1} P_i^k (\mathbf{J}_i^{k-1})^{-1} P_i^k \right) - (f_i^k r_i^k)^2. \quad (2.16)$$

Let us suppose that up to the iteration $k-1$ our map is the gradient of a convex function, namely that $\mathbf{J}_i^{k-1}$ is a positive definite matrix for each i . If P^k is a positive definite matrix, the product $(\mathbf{J}_i^{k-1})^{-1} P_i^k (\mathbf{J}_i^{k-1})^{-1} P_i^k$ can be viewed as the product of two positive definite matrices $(\mathbf{J}_i^{k-1})^{-1} P_i^k (\mathbf{J}_i^{k-1})^{-1}$ and P_i^k respectively. Using the fact that a positive definite matrix can be decomposed according to the Cholesky factorization, we can assert that the trace of the product of two positive definite matrices is positive and therefore that

$$\text{tr} \left((\mathbf{J}_i^{k-1})^{-1} P_i^k (\mathbf{J}_i^{k-1})^{-1} P_i^k \right) > 0. \quad (2.17)$$

Summarizing, a sufficient condition for H to be a negative definite matrix is to have the matrix P^k positive definite.

What does it mean having P^k positive definite in terms of the elementary potential $F(r)$ appearing in eq. (2.6)?

Using the definition of P given in eq.(2.14) it is easy to verify that P is a positive definite matrix if

$$f(r) = \frac{1}{r} \frac{dF}{dr} > 0 \quad (2.18)$$

$$f(r) + r \frac{df(r)}{dr} = \frac{d^2 F(r)}{dr^2} > 0 \quad (2.19)$$

Therefore, if F is a convex function of r with positive derivative in r then P is a positive definite matrix.

2.1.1.1 Convexity condition

In this section we will deal with the issue on how to enforce the convexity of the RHS of the equation (2.13). The idea is to use the fact that positive values of β_i lead always to a convex approximation of the potential while negative values not necessarily. This suggests the following procedure:

- Compute β from the maximization of LL
- If $\beta > 0$ move the points according to (2.12)
- If $\beta < 0$ for each data point x_j^0 check that:
 1. $\phi(x_i^0) - \phi(x_j^0) > \langle \nabla \phi(x_j^0), (x_i^0 - x_j^0) \rangle$ for all x_i^0 in the sampled data set.
 2. $\det(D^2(\phi(x_j^0))) > 0$
- If the relations 1 and 2 are true then accept the move, otherwise reject and recompute β choosing a different center $\tilde{x}$ among the sampled points $\{x_i^0\}$.

Point (1) ensures that for each data point the tangent plane to ϕ at the point $(x_j^0, \phi(x_j^0))$ should lie below all the other data points $(x_i^0, \phi(x_i^0))$ with $i \neq j$. This is sufficient to say that the value of the potential computed at the data points can at least interpolate a convex function. Point (2) is a statement about the uniqueness of the map $y(x) = \nabla \phi(x)$ evaluated at the data points.

The convexity check is fairly easy to implement. The gradient of the potential $\nabla \phi(x_j^0)$ is given by the map described in the equation (2.12) and, given x_j^0 , we don't need to perform the check over all the other data points but only over the nearest neighbors of

x_j^0 . To define nearest neighbors of a point we perform a Delaunay triangulation on the set of data points $\{x^0\}$ at the beginning of the iterative procedure.

Even though the triangulation is done once, at the beginning of the procedure, its computational cost may be one of the bottleneck in using the approach described so far. It is important to mention, though, that there are robust methods for performing Delaunay triangulation in 6 dimensions (see for instance [4]).

Besides its computational cost, the major issue in using this procedure is related with the fact that a fixed number of data points, become increasingly “sparse” as the dimensionality increases. As a consequence of this fact checking whether the values of the potential, computed at the data points, interpolate a convex function may not make anymore sense. For this reason different strategies are needed to reject values of β leading to non-convex potentials. Finding an efficient strategy in this context may not be an easy task and the strategy based on the triangulation was proposed because it is easy to implement. Rather than focusing on this topic we preferred to explore a different approach.

In the next chapter, in fact, we will present a different algorithm where instead of approximating the optimal map with a sum of gradient of functions, we will map ρ into μ by composing elementary maps. In this new setting we will not have any more the problem of rejecting certain values of β .

We close this section by considering the termination criterion of the algorithm so far described, and under which conditions this converges to the optimal map. There are two conditions under which the algorithm terminates.

The first condition is simply given by $\beta \approx 0$, and corresponds to the quantity defined in (2.15) to be zero. In the ideal case this happens when $\rho^k \approx \mu$ but it may also be that the family of elementary maps we are using to maximize the log-likelihood is not rich enough. In the latter case, we may reach a point in which the log-likelihood is stationary along all the directions of the infinite dimensional space we are exploring when using a

given family of maps.

The second circumstance under which the algorithm terminates is related instead with the number of rejections. In this case β may also be different from zero, but the sampled points from ρ have been mapped to a configuration from which they can be moved without violating the convexity constraint and without maximizing the log-likelihood at the same time. From proposition of Chapter 1 we know that, given two probability densities, there is always a map, given by the gradient of a convex function, between them. As in the previous case, though, we don't know if such map can be approximated with the family of elementary functions we are using.

Both the termination conditions are therefore relative to the family of elementary maps we are using at each iteration step. From the point of view of whether the family of functions over which we are maximizing the log-likelihood is big enough to guarantee convergence to the optimal map, no rigorous analysis has been done in this thesis. The criterion we follow to choose this family is based on the reasonable fact that maps acting locally should always be able to adjust the sampled points in order to adapt them to configurations that are closer and closer (in the KL sense) to that one of the target distribution μ . The same consideration holds in the case the value of β gets rejected because violating the convexity condition (see the end of section (2.1.1.2)).

2.1.1.2 Proposed map and the problem of over-resolving

An example of a radial map satisfying equations (2.18) and (2.19) is given by choosing $f(r) = \frac{\text{erf}(r/\alpha)}{r}$ in equation (2.12). The corresponding potential $F(x)$ associated with $f(r)$ through eq. (2.18) is

$$F(x) = r \text{erf} \left(\frac{r}{\alpha} \right) + \frac{\alpha}{\sqrt{\pi}} e^{-\left(\frac{r}{\alpha}\right)^2} \quad (2.20)$$

where $r = |x|$ and α is the length scale of the map. The locality of the map derives from the fact that $f(r) \xrightarrow{r \rightarrow \infty} 0$, and by introducing the parameter α we set the length scale

on which $f(r)$ becomes negligible.

The length scale α depends on the coordinates of the center $\tilde{x}$ of the map and has the crucial role of avoiding to over-resolve $\rho(x)$.

In particular, the approximation of $L[y^k]$ with the log-likelihood LL^k can be viewed as if we were considering the density $\rho(x)$ a sum of delta functions, each one centered at the sample data points $\{x_i^0\}$. Since at each step we are minimizing the KL divergence between ρ^k and the density $\rho(x)$, approximating $\rho(x)$ with a sum of delta functions would lead to a very peaked ρ^k around the sampled points drawn from $\rho(x)$. This is true only if we allow the function $\nabla F(x)$ at each step to be arbitrarily localized.

This problem is common in density estimation and it usually is solve by regularizing the log-likelihood function when maximizing it. This can be done, for instance, by introducing a penalty term while, in our case, the regularization is obtained by means of the length scale α . There are several possible strategies for choosing the value of α . The idea that should be common to these strategies is some how represented in figure (2.1).

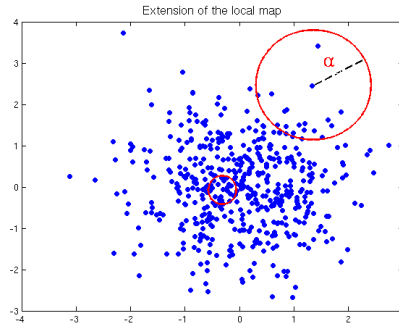


Figure 2.1: Illustration of the extension of the map. The circles in red represent the regions where the elementary map is affecting the sampled points. The value of α is the radius of such regions.

We would like that the same number of points np be affected by the elementary map regardless the location around which the elementary map is centered. Zones with high density of sampled points should be characterized by small values of α , while on the

tail of the density α should be bigger. In this way we avoid ρ^k to over-fit the data $\{x_i\}$ sampled from ρ .

A brute force approach for the choice of the value of α could be therefore to choose the center $\tilde{x}$ and increase in a progressive way α until the circle of radius α and center $\tilde{x}$ contains the desired number of sampled points np .

The ideal situation, however, corresponds to the case in which we are able to compute the pushforward $\tilde{\rho}^k$ of the starting density ρ at the center of the map $\tilde{x}$. In particular $\tilde{\rho}^k$ is defined as

$$\tilde{\rho}^k(y^k) = \frac{\rho(x)}{J^k(x)} \quad (2.21)$$

and corresponds to the distribution of the points $\{y^k(x_i)\}$ given the fact that the points $\{x_i\}$ are distributed according to $\rho(x)$.

Given $\tilde{\rho}^k$ a good definition of α is given by

$$\alpha = \left(\frac{np}{m \cdot \mathcal{V}_n \cdot \tilde{\rho}^k(\tilde{x})} \right)^{\frac{1}{n}} \quad (2.22)$$

where np is the number of typical points we want to be affected by the map, m is the number of points sampled from $\rho(x)$, $\mathcal{V}_n$ is the volume of the n dimensional sphere and n is the dimension of the space the points x are defined in.

If on the one hand in imaging processing it is fair to assume the knowledge of ρ on a mesh in space, and therefore interpolate such values in order to obtain $\tilde{\rho}^k$ and so the value of α , in data mining kind of applications, this is not the case. One way of solving this issue is presented in [20].

Even though the role of α is relevant for the algorithm to work properly, devising a strategy to determine its value could represent a separate task per se. In the following we will instead focus on how to find the optimal map once we have a reasonable way of estimating the value of α given the value of np .

In the following numerical tests the general strategy adopted to choose the value of np is to start with a relatively big value of np (typically 80% of the number of data points) and decrease this value as the simulation advances. In particular we choose to decrease the value of np by a unit every time that the convexity condition is violated according to the criterion:

$$np = \max(np - rej, th) \quad (2.23)$$

where rej is the number of moves rejected because violating the convexity condition and th is a lower bound (typically chosen to be 5% of the number of data points) below which we would have over-resolution.

This criterion is guided by the idea that, as the number of iteration steps increases, it would become more and more unlikely to improve the log-likelihood without violating the convexity condition by using maps that are not local enough.

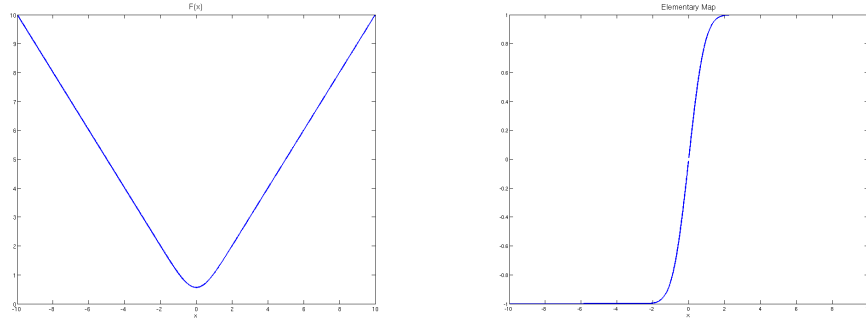


Figure 2.2: $F(x) = r\text{erf}(r) + \frac{e^{-r^2}}{\sqrt{\pi}}$.

2.2 Compact support and boundary conditions

As mentioned in the first section, because we need to evaluate $\frac{\partial \mu(y_\beta(x))}{\partial \beta}$ (eq. (2.8)), we are assuming that the target density μ is smooth. This excludes an important category of optimal transport problem in which μ has compact support and is differentiable on the interior of it but not on its boundary. Let us suppose that all the points $\{x_i\}$ are contained in the domain of the target density at the beginning of the iteration procedure. Because of the non differentiability of μ on the boundary $\partial\Omega$ of its support, it may happen that one or more of the data points are mapped outside Ω leading to values of LL of minus infinity.

In this section, we will describe two different ways for handling this problem. The first one, popular in the imaging processing community, is based on mollifying the target density while the second one, relies on which we will focus, on a particular choice of the elementary map $\nabla_{x^0} F$ appearing in the equation (2.6).

2.2.1 Mollification

If the target density μ has compact support Ω and is smooth up to and including the boundary $\partial\Omega$ then we would not have problems in dealing with the term $\log(\mu(x))$ on regions where μ is zero. Suppose that at the beginning of the iteration procedure all the points $\{x^0\}$ are inside Ω . Because of the smoothness of μ , no points will be mapped out of Ω since that would imply a progressive decreasing of the value of the log-likelihood. In this setting the transport boundary condition, that we remind requires only that the boundary of the starting density is mapped to the boundary of the target density without specifying how this should be done (see [13] for more details), are naturally fulfilled.

For this reason, one possible way of handling cases in which μ is non-smooth, is to implement the algorithm by initially using a smooth target density μ_ϵ close to the target density μ .

The parameter ϵ represents how well μ_ϵ approximates μ , in particular we define

$\mu_0 = \mu$, and the idea is to take the limit for ϵ going to zero as ρ is mapped to μ_ϵ . In this way we will have that ρ is mapped to μ_ϵ while μ_ϵ itself is mapped to μ resulting in a overall map from ρ to μ .

In implementing this procedure the convergence, regulated by ϵ , of μ_ϵ towards μ should be slow enough to allow the points $\{x^k\}$ to remain always inside Ω .

2.2.2 Elementary map for compact support target distributions

A second possible solution to the problem exposed in the introduction of this section is to use elementary maps that do not allow the data points to be mapped out of a certain domain.

For simplicity let's assume that the starting and the target probability densities are one dimensional and both defined on the same $[0, 1]$ support of the real line. Since the approximation to the optimal plan is given by equation (2.12) it is sufficient to choose as elementary map ∇F a function that vanishes at $x = 0$ and $x = 1$ (namely on the boundary of the support). This would guarantee that $y^k(0) = 0$ and $y^k(1) = 1$ at each iteration step and since we are rejecting non increasing map in the interval $[0, 1]$ (because otherwise they would not be a gradient of a convex potential) then we would have

$$y^k(0) \leq y^k(x^0) \leq y^k(1) \quad \forall x \in [0, 1] \quad (2.24)$$

In practice no sampled data point will be exactly on the boundary of the domain (namely at $x = 0$ and $x = 1$). This leads to the possibility of mapping points outside the domain since, given x_{min} and x_{max} ³, respectively the smallest and the biggest data points sampled from $\rho(x)$, equation (2.24) would read

$$y^k(x_{min}) \leq y^k(x) \leq y^k(x_{max}) \quad \forall x \in [0, 1] \quad (2.25)$$

³With $0 < x_{min}$ and $x_{max} < 1$

where now $y^k(x_{min})$ and $y^k(x_{max})$ are not guaranteed to be equal to zero and one respectively (since the points that are guaranteed to be fixed point of the map are only those one on the boundary of the support).

This can be fixed by introducing *ghost points* on the boundary of the domain that are not moved by the map. These points are not considered in any part of the algorithm except when we check that the convexity of the potential is preserved.

The preceding discussion extends to multidimensional spaces. By using the mean value theorem for multivariable functions (see for instance [31]) it can be shown that the convexity of the potential together with the fact that the gradient of the potential vanishes on the boundary of some convex region $\mathcal{B}$, are sufficient conditions to ensures that points inside $\mathcal{B}$ are mapped to points inside $\mathcal{B}$.

Even though this second way of handling non smooth μ was found to converge much faster than using mollification, the essential drawback is that, by choosing a vanishing map on the boundary of Ω , we are imposing a Neumann boundary condition on the potential appearing in the Monge Ampere equation instead of using the more general transport boundary condition.

2.3 Numerical experiments

In the following, we will consider optimal transport in one and two dimensions. One dimensional optimal transport is qualitative different from optimal transport in two or more dimensions. This is due to the fact that in one dimensions optimal transport is obtained just by “sliding” the mass. Yet, testing the algorithm on one dimensional examples, can give us some information about the richness of the family of maps we are using. This is especially true when the solution of the problem may developed some kind of singularity like in the mass splitting example showed in this section.

The proposed numerical experiments are examples of constructed solutions: starting from a set of points $\{y_i\}$ (with $y_i \in \mathbb{R}$ or $\mathbb{R}^2$) sampled from the target density $\mu(y)$, we will map them to a set of points $\{x_i\}$ using the inverse of a map that is the gradient of a convex function.

In this way, given the set of points $\{x_i\}$, we can use them as if they were sampled from the starting density $\rho(x)$ and implement the algorithm previously described to compute the numerical approximation $y^k(x)$ of the optimal map $y(x)$ from $\rho(x)$ to $\mu(y)$.

Because we know the exact target density μ , we can actually use the map $y^k(x)$ to estimate the starting distribution ρ by the change of variable formula $\rho_{est}(x) = J^k(x)\mu(y^k(x))$. This is done by evaluating μ and the Jacobian of the map $J^k(x)$ on a regular grid of Lagrangian markers x_g that move passively while computing $y^k(x)$.

In addition to the estimated density, we will give two other measures of the quality of the numerical solution. We will compute the value of the log-likelihood and the value of the cost on the sampled points defined as

$$\frac{1}{m} \sum_{i=1}^m \log(\rho^k(x_i)) \quad \frac{1}{m} \sum_{i=1}^m |y^k(x_i) - x_i|^2 \quad (2.26)$$

respectively.

Those values should converge as $k \rightarrow \infty$ and $m \rightarrow \infty$ to the values of the entropy

and the exact cost

$$\int \log(\rho(x))\rho(x)dx \quad \int |y(x) - x|^2 \rho(x)dx \quad (2.27)$$

computed using the values of the exact starting density and the exact map, both evaluated on the grid defined by the Lagrangian. Their values are displayed in red in the plots together with the quantities defined in (2.26).

2.3.1 1D case

2.3.1.1 Translation of a Gaussian

In this section we will consider a translation from a Gaussian $\mathcal{N}(2, 1)$ to a normal distribution $\mathcal{N}(0, 1)$. It is easy to verify that the potential $\phi(x) = \frac{x^2}{2} - 2x$, generating the map $y(x) = x - 2$, satisfies the Monge-Ampere eq. (1.15). The figures in (2.3) refer to simulations obtained using 1000 sampled points. The typical number of points affected by the map at each iteration was chosen to be $np = 800$ while the learning rate was set at the value $\epsilon = 0.3$.

In the figures (2.4) we compare the numerical approximation with the exact optimal plan. We choose to compare them on the interval $[-3, 7]$. At the extreme of the interval the value of the starting density $\rho(x)$ is about $1.5 \cdot 10^{-6}$ (5 sigmas from the mean) and there are no sampled points there. Therefore, we are expecting a low convergence of the numerical map at the extreme of the interval.

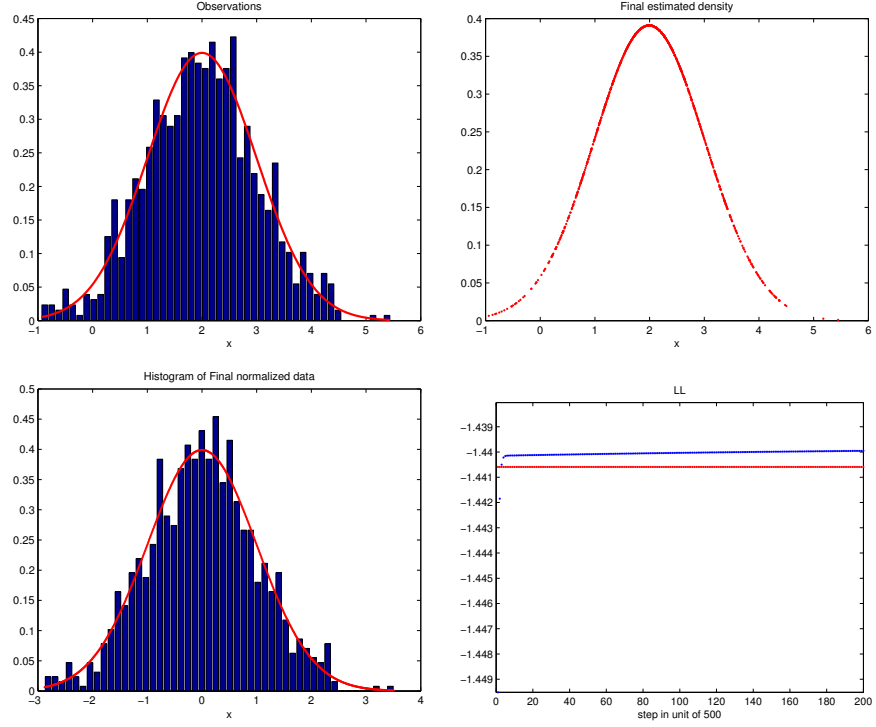


Figure 2.3: Results relative to 1000 data points. Top left: Initial data with the underlying theoretical probability density $\mathcal{N}(2, 1)$. Top right: Histogram of the transported data with the underlying theoretical target distribution $\mathcal{N}(0, 1)$. Bottom left: estimated density. Bottom right: the red line represents the entropy computed on the grid points while in blue is represented the log-likelihood defined in (2.26).

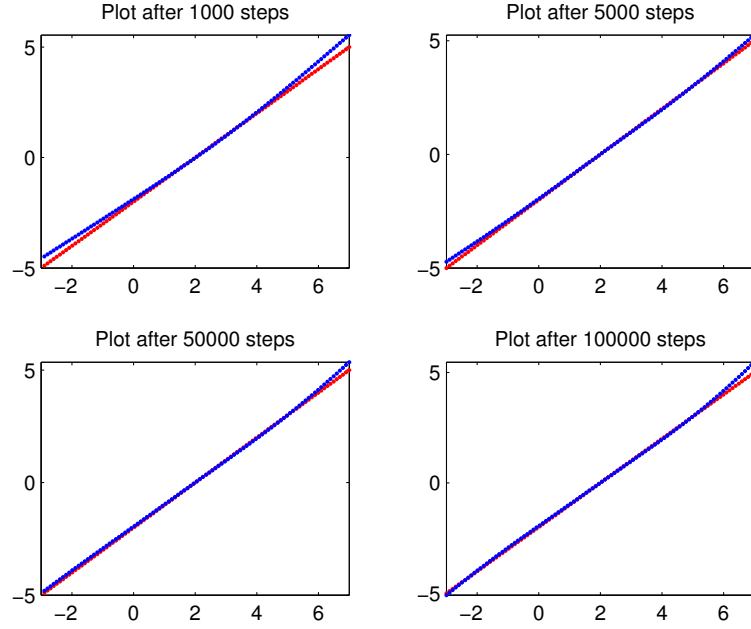


Figure 2.4: Convergence of y^k (in blue) to the exact map $y = x - 2$ (in red) when using 1000 data points.

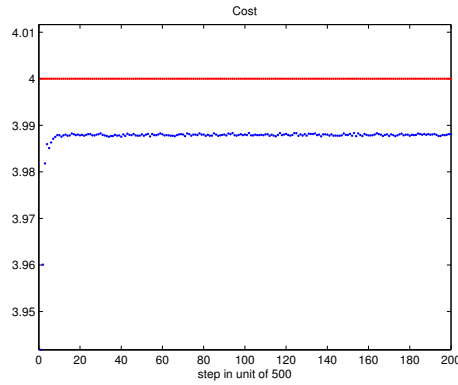


Figure 2.5: The blue curve represents the value of the functional defined in equation (1.1) estimated using the numerical approximation to the optimal plan evaluated at the sampled data points. The cost grows as the data points moves towards a configuration compatible with the target probability density. The red curve represents the value of the cost estimated using the grid points and the exact optimal plan.

2.3.1.2 Quartic Potential with $\mathcal{N}(0, 1)$ as target distribution

In this example we will transport the density

$$\rho(x) = \frac{3x^2}{\sqrt{2\pi}} e^{-\frac{x^6}{2}} \quad (2.28)$$

to a normal distribution. It is again easy to verify that the convex potential satisfying eq. (1.15) is given by $\phi = \frac{x^4}{4}$ and so the optimal map is $y = x^3$. Below we report the results relative to a simulation performed using 10^5 iterations with 10^3 data points, $\mathbf{np} = 600$ and a learning rate of 0.3.

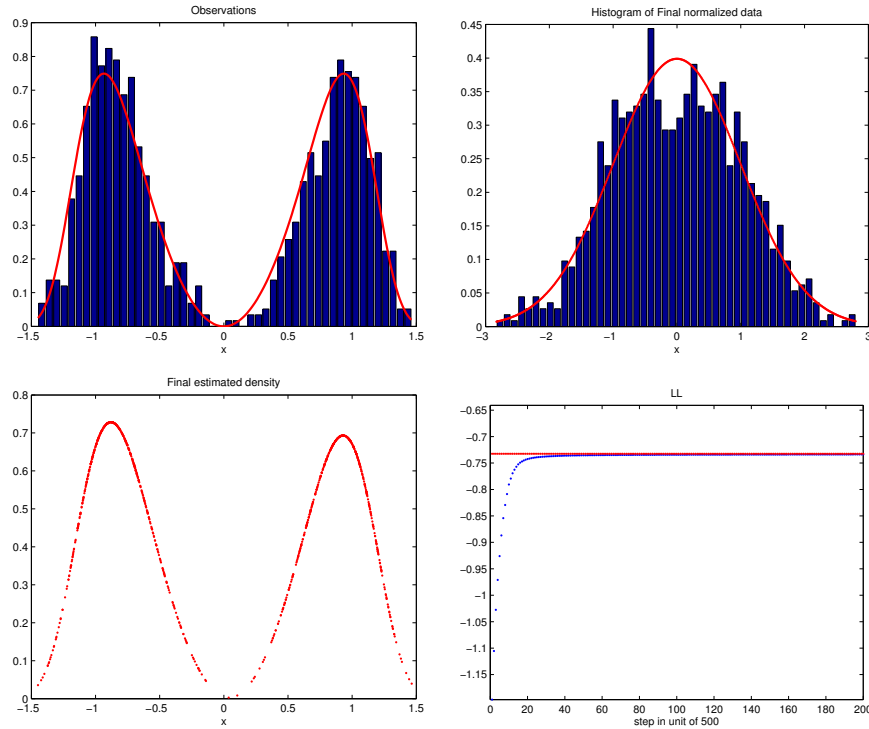


Figure 2.6: Results relative to 1000 data points. Compare with figure (2.3) Top left: Initial data with the underlying theoretical probability density defined in eq. (2.28). Top right: Histogram of the transported data with the underlying theoretical target density $\mathcal{N}(0, 1)$. Bottom left: Estimated density. Bottom right: The red line represents the entropy computed on the grid points while in blue is represented the log-likelihood defined in (2.26).

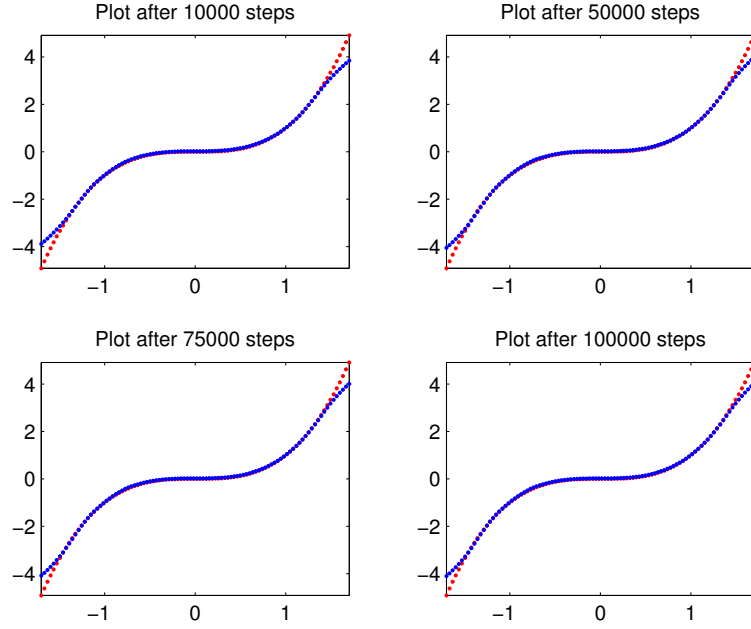


Figure 2.7: Convergence of y^k to the exact map $y = x^3$. The range on the x axis is $[-1.7, 1.7]$, and at the extreme of this interval the value of the starting density (eq. 2.28) is about $2 \cdot 10^{-5}$. As can be seen the map converged after about 50000 steps. The convergence is slower on the tails of $\rho(x)$.

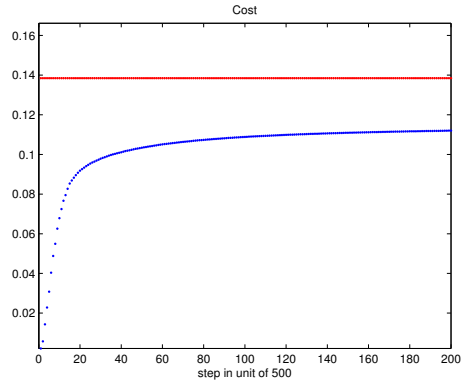


Figure 2.8: As in figure (2.5)

2.3.2 Compact support and mass splitting

In this section, are presented numerical results obtained with the procedure described in the subsection (2.2.2) relative to optimal transport between densities with compact support.

In the first example, we will test our algorithm on a simple problem where the optimal map $y(x) = x^3$ maps $\rho(x) = 3x^2 \mathbb{1}_{[0,1]}$ to $\mu(y) = \mathcal{U}_{[0,1]}(x)$ (see figure (2.9)).

The second example deals with mass splitting and is more challenging from the numerical point of view. In this case, the starting and the target distributions are respectively $\rho(x) = \mathcal{U}_{[0,1]}(x)$ and $\mu(x) = \frac{\cos(2\pi x) + a}{a} \mathbb{1}_{[0,1]}$ where the parameter a controls the distance of the local minimum of $\mu(x)$ located at $x = 0.5$ (see figure 2.11) from the abscissa axis. As a approaches zero, the target distribution splits in two parts by developing a zero at $x = 0.5$. This is reflected in the fact that the derivative of the map (given by $y(x) = f^{-1}(x)$, where $f(x) = \frac{1}{a} \left[\frac{\sin(2\pi x)}{\pi} + ax \right]$) diverges at the point $x = 0.5$ (see figure (2.14)).

In particular, we display two simulations obtained with different values of the parameter a . In the first simulation, where the target density it's not so different from the uniform distribution, we set the value of a to 2. Figure (2.13) is instead obtained by setting $a = 1.01$ and the target density is characterized by the fact that its minimum is very close to zero.

In both figures (2.11) and (2.13) it is important to note that the estimated density ρ_{est} is over-fitting the real density ρ . The value of np was chosen to be respectively $np = 200$ and $np = 50$ and results in a non smooth estimated density (top right panel). Nevertheless the optimal plan is well approximated by the numerical solution (figures (2.12) and (2.14)).

This is due to the fact that the relation between the optimal map, the target and the starting density is given by $\rho(x) = y'(x)\mu(y(x))$. The degree of smoothness of the derivative of the numerical solution is comparable to that one of the starting density

assuming a smooth enough target distribution. This results in a smooth optimal map even when we are over resolving making the algorithm particularly robust when choosing small values of np .

Both the examples are performed using the following elementary map

$$\nabla_{x^0} F(x^0) = x^0(x^0 - 1)e^{-\frac{(x^0 - \tilde{x})^2}{2\alpha^2}} \quad (2.29)$$

where $\tilde{x}$ is a random point chosen among the initial data points x^0 and α is the parameter defined by equation (2.22). The factor $x^0(x^0 - 1)$ ensures that the map vanishes at the boundary of the domain $[0, 1]$. In this way points that are initially in $[0, 1]$ remain inside it as described in section (2.2.2).

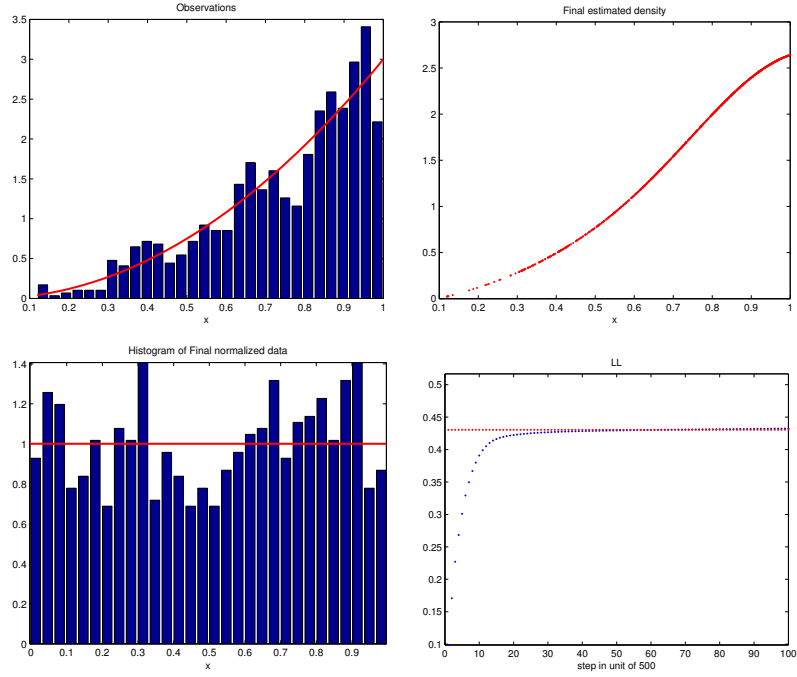


Figure 2.9: Example 1. Results relative to 1000 data points. The starting density, $\rho(x) = 3x^2$ (top left), is mapped into $\mathcal{U}_{[0,1]}(x)$ (bottom left).

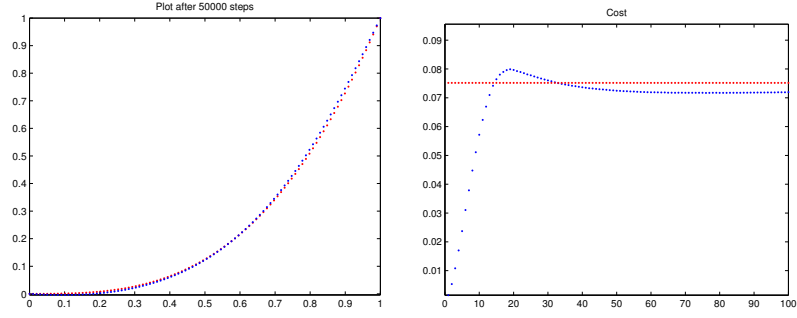


Figure 2.10: Example 1. Left: Convergence of y^k (in blue) to the exact map $y = x^3$ (in red) relative to the optimal transport shown in figure (2.9). Right: Convergence of the cost (blue) to the cost of the optimal plan (red).

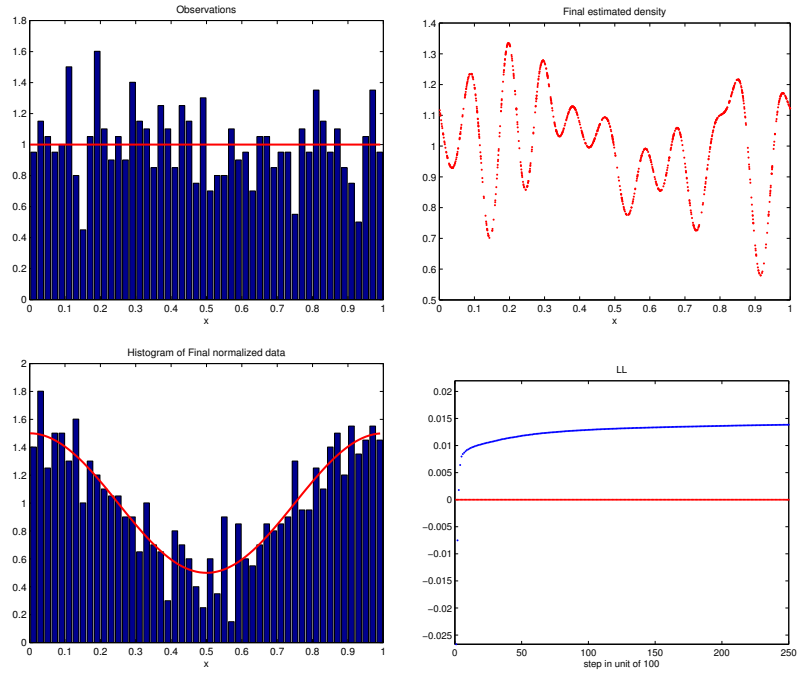


Figure 2.11: Example 2. Results relative to 1000 data points. The starting, $\rho(x) = \mathcal{U}_{[0,1]}(x)$ (top left), is mapped into $\mu(x) = \frac{\cos(2\pi x)+2}{2}$ (bottom left).

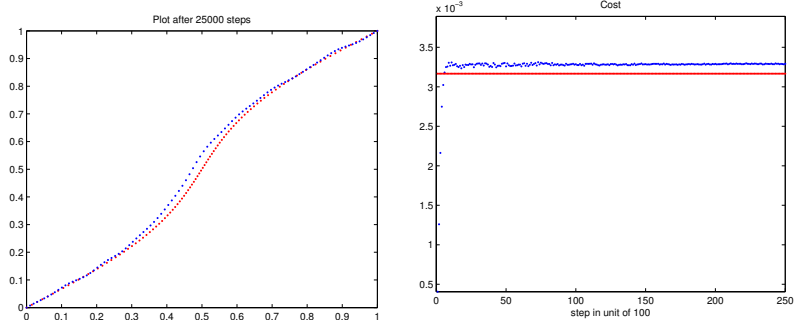


Figure 2.12: Example 2. Left: Convergence of y^k (in blue) to the exact map (in red) relative to the optimal transport shown in figure (2.11). Right: Convergence of the cost (blue) to the cost of the optimal plan (red).

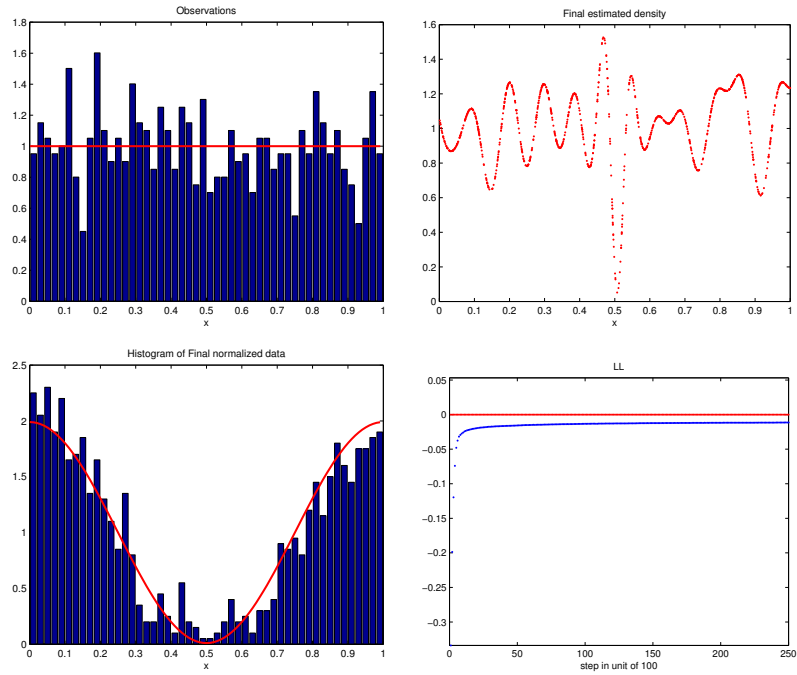


Figure 2.13: Example 2. Results relative to 1000 data points. The starting, $\rho(x) = \mathcal{U}_{[0,1]}(x)$ (top left), is mapped into $\mu(x) = \frac{\cos(2\pi x) + 0.01}{0.01}$ (bottom left).

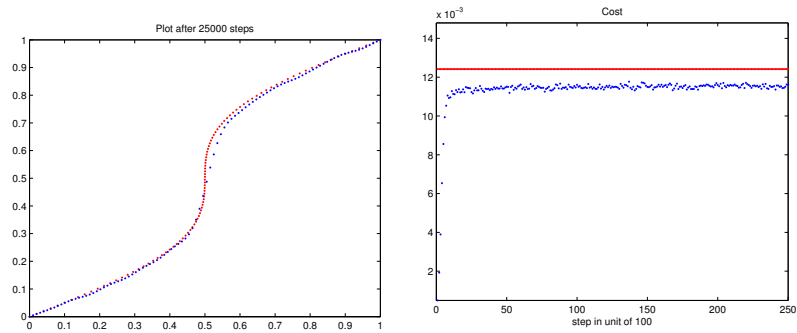


Figure 2.14: Example 2. Left: Convergence of y^k (in blue) to the exact map (in red) relative to the optimal transport shown in figure (2.13). Right: Convergence of the cost (blue) to the cost of the optimal plan (red).

2.3.3 Two dimensional case

In this section we will present results relative to numerical solutions of optimal map between two dimensional probability distributions. In particular, we will implement the two dimensional analogue of the translation and the quartic potential that we saw in the one dimensional case.

Translation

A two dimensional Gaussian centered at $(2, 2)$ with covariance matrix equal the identity is transported to a two dimensional normal Gaussian. It is easy to verify that the potential satisfying eq.(1.15) is given by $\phi(x, y) = \frac{x^2}{2} + \frac{y^2}{2} - 2x - 2y$ and so the optimal plan is given by

$$\nabla\phi(x, y) = \begin{pmatrix} x - 2 \\ y - 2 \end{pmatrix}. \quad (2.30)$$

The results presented in the following figures are relative to simulations performed with 1000 sample points, a tolerance for the time step of 0.3, and a typical number of points np affected by the map equal to 800.

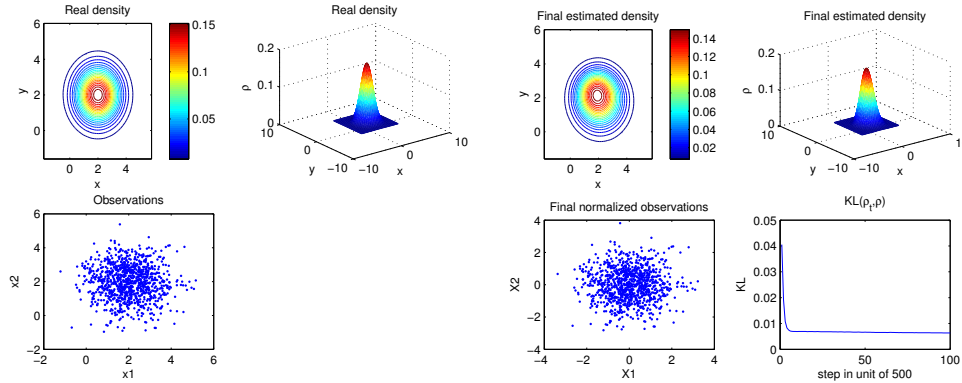


Figure 2.15: Translation of a 2D Gaussian distribution. Result relative to 1000 sample points.

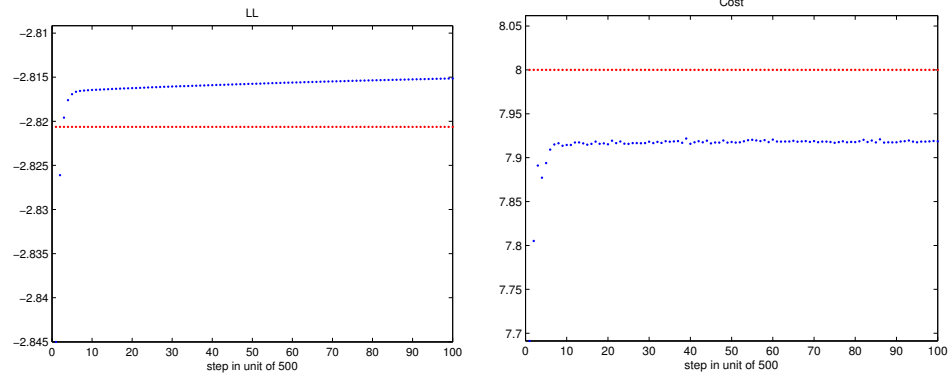


Figure 2.16: Log-likelihood computed from the evolving points (in blue) and using the exact density ρ (in red) as described at the beginning of section (2.3). The fact that the blue curve lies above the red curve, is due to a too small value of np .

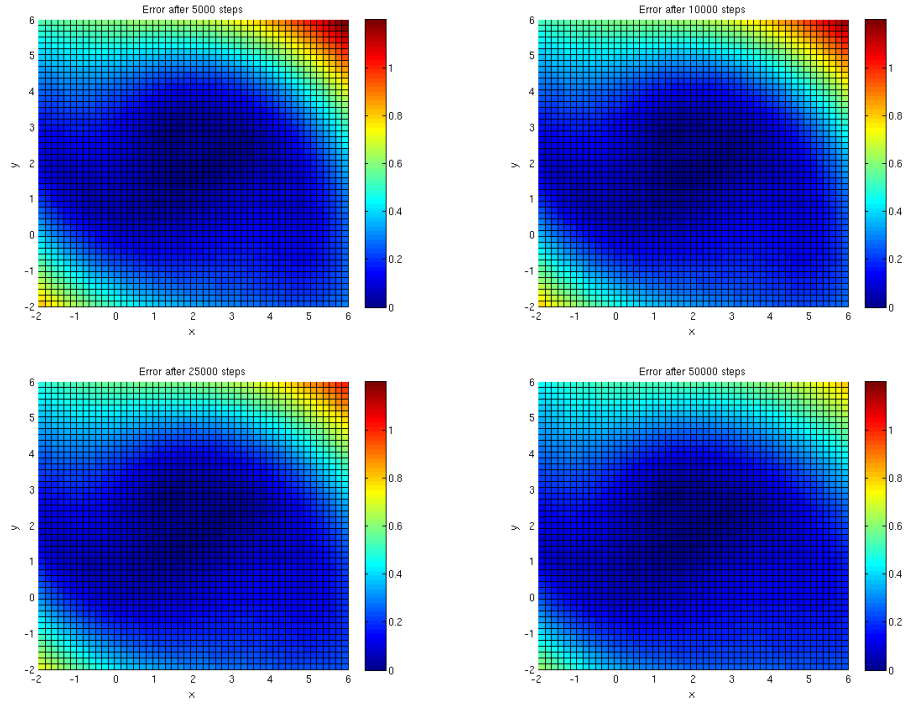


Figure 2.17: L^2 difference between the numerical solution and the exact one $\nabla\phi(x, y) = (x - 2, y - 2)$ at different values of the iteration step.

Quartic potential

We want to transport the density

$$\rho(x) = \frac{9(xy)^2}{2\pi} e^{-\frac{(x^6+y^6)}{2}} \quad (2.31)$$

to a two dimensional normal distribution. The solution to the equation (1.15) is given by $\phi(x, y) = \frac{x^4+y^4}{4}$. In the following we are using 1000 data points with $np = 800$.

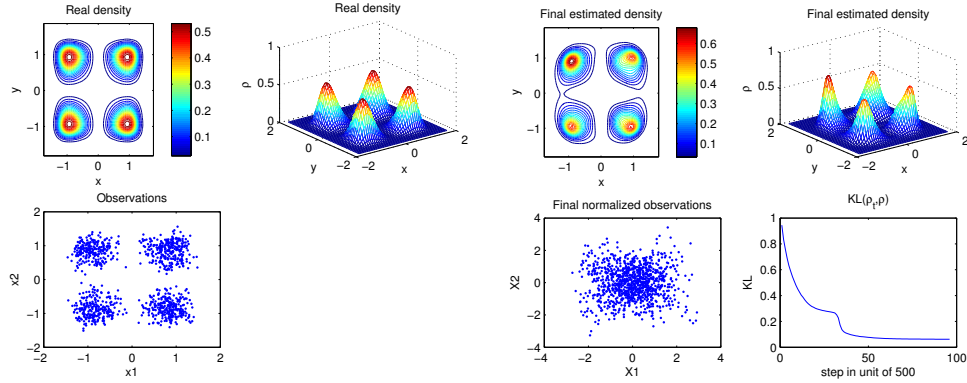


Figure 2.18: Result relative to 1000 sample points and 50000 iteration step. Notice how the rate of change of the KL distance (or analogously of the Log Likelihood of figure 2.19) varies along the simulation. This is due to the fact that the value of np becomes smaller as the number of rejection increases. Smaller values of np allow bigger local adjustment towards the target distribution.

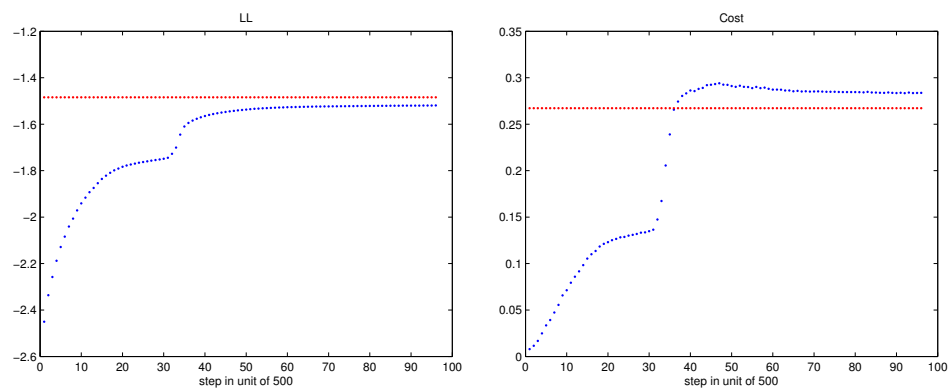


Figure 2.19: Log-likelihood computed from the evolving points (in blue) and Log-likelihood estimated using the exact density ρ (in red) as described at the beginning of section (2.3).

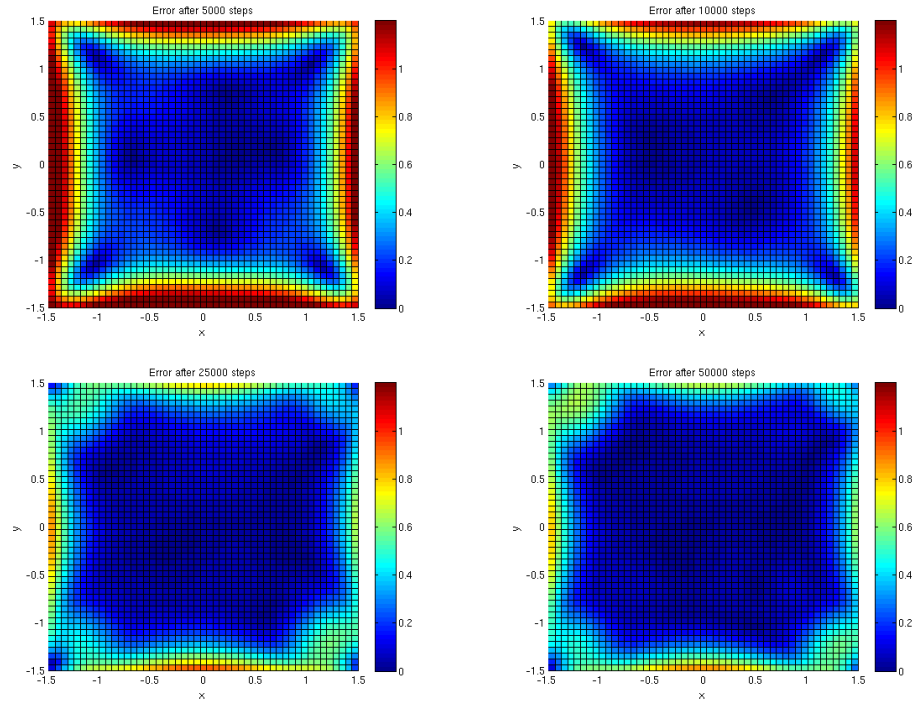


Figure 2.20: L^2 difference between the numerical solution and the exact one $\nabla\phi(x, y) = (x^3, y^3)$ at different values of the iteration step.

Compact support

In this section, we will present an example of a two dimensional optimal plan between distributions both supported on the unit disk. What was discussed for the one dimensional case holds also here. Since we are considering a two dimensional example, in which the density $\rho(x) = 2r^2$ supported on the unit disk is mapped into the uniform density on the unit disk, we want to impose that the numerical approximation of the optimal plan maps the unit circle into the unit circle. The elementary map used at each time step is the gradient of the following function:

$$F(x^0) = (1 - r^2)^{\frac{3}{2}} e^{-\frac{\tilde{r}^2}{2\alpha^2}}. \quad (2.32)$$

where $r = |x^0|$ and $\tilde{r} = |x^0 - \tilde{x}|$. The elementary potential in (2.32) corresponds to the elementary map

$$\nabla F(x^0) = (1 - r^2)^{\frac{1}{2}} e^{-\frac{\tilde{r}^2}{2\alpha^2}} \left[-3x^0 - \frac{2}{\alpha^2} (1 - r^2)(x^0 - \tilde{x}) \right] \quad (2.33)$$

whith Jacobian, being the Hessian of a scalar function, given by the symmetric matrix

$$\begin{aligned} P(x^0) = 3(1 - r^2)^{-\frac{1}{2}} e^{-\frac{\tilde{r}^2}{2\alpha^2}} A + 3(1 - r^2)^{\frac{1}{2}} e^{-\frac{\tilde{r}^2}{2\alpha^2}} \left[\frac{2}{\alpha^2} B - I \right] \\ + \frac{2}{\alpha^2} (1 - r^2)^{\frac{3}{2}} e^{-\frac{\tilde{r}^2}{2\alpha^2}} \left[\frac{2}{\alpha^2} C - I \right]. \end{aligned} \quad (2.34)$$

where I is the identity matrix, $A = x^0 \otimes x^0$, $B = x^0 \otimes (x^0 - \tilde{x}) + (x^0 - \tilde{x}) \otimes x^0$ and $C = (x^0 - \tilde{x}) \otimes (x^0 - \tilde{x})$.

The factor $(1 - |x^0|^2)^{\frac{3}{2}}$ grantees that both $F(x^0)$ and its gradient vanish on the unit circle.

The optimal plan between these two densities is given by

$$y(x) = r \begin{pmatrix} x \\ y \end{pmatrix}. \quad (2.35)$$

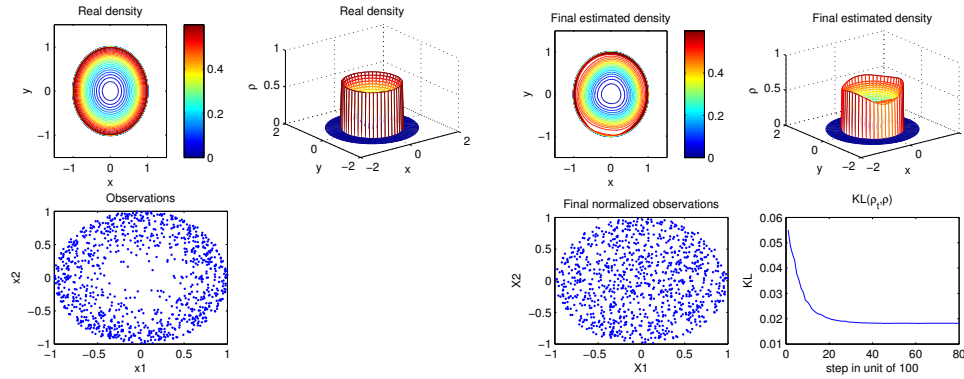


Figure 2.21: Result relative to 1000 sample points and 8000 iteration steps

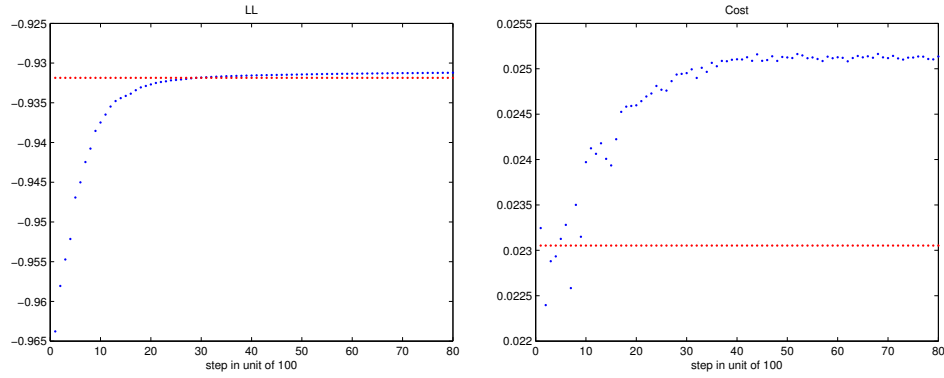


Figure 2.22: Log-likelihood computed from the evolving points (in blue) and estimated using the exact density ρ (in red) as described at the beginning of section (2.3).

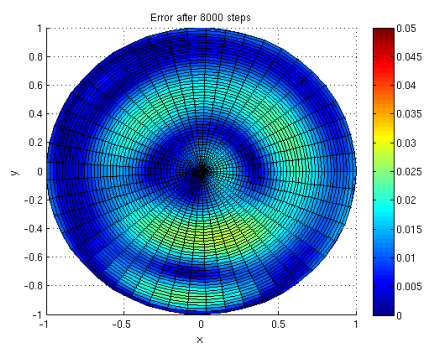


Figure 2.23: L^2 difference between the numerical solution and the exact one $\nabla\phi(x, y) = (x^3, y^3)$.

Chapter 3

A numerical scheme based on the Kantorovich optimal transport problem

The aim of this chapter is to modify the approach followed in Chapter 2 in order to handle cases in which the only information available about the target density μ is a data set drawn from it.

This is not possible by directly applying what described in Chapter 2 since we were maximizing the log-likelihood defined as an average with respect to the starting density ρ of some function of the target density μ . In particular, in order to take the derivative with respect to the parameter β (see equation (2.8)), we needed to have the analytic form of μ and know that this probability density was differentiable.

In principle it is possible to overcome this problem by approximating the target density by a mixture of elementary, differentiable distributions with known analytic form. For instance, given a data set drawn from μ , we could perform a kernel density estimation of μ in order to express it as a normalized, weighted sum of Gaussian distributions and then use the approach described in Chapter 2 to find the optimal map from ρ to the

aforementioned sum.

The pre-processing of the data sampled from μ would require extra work to perform in addition to the optimal map from ρ to μ we want to compute. In this chapter, we will present an alternative approach that does not require this pre-processing and that is based on the use of the functional (1.20). Inspired by [9], we will present an iterative algorithm that, compared to that one in [9], has the main advantage of being mesh-free.

In the first section of this chapter we will briefly discuss why and how we intend to integrate the approach in [9] with the algorithm described in Chapter 2. We will then present in more detail the algorithm and perform some numerical simulations on artificial data sets.

3.1 Introductory considerations

For the sake of the exposition we remind the definition of the functional in (1.20)

$$\tilde{L}(u) = \int_{K_1} u(x)\rho(x)dx + \int_{K_2} u^*(y)\mu(y)dy. \quad (3.1)$$

The theorem (1.2), of Chapter 1, assures that if we find the minimizer $\bar{u}$ of (1.20) over the class of continuous functions, then the gradient $\nabla\bar{u}$ of such minimizer gives the optimal map from ρ to μ .

As explained in [9], a way of computing $\bar{u}$ is by gradient descent of $\tilde{L}$:

$$\begin{cases} \frac{\partial u(x, t)}{\partial t} = -\frac{\delta\tilde{L}}{\delta u} \\ u(x, 0) = \frac{x^2}{2} \end{cases} \quad (3.2)$$

There are two main problems related to this approach:

1. The expression for the functional derivative $\frac{\delta\tilde{L}}{\delta u}$ given in (1.21) holds only if we are evaluating the derivative at a function u such that ∇u is a one-to-one map.

Moreover such expression is in general not easy to evaluate. In [9] this was done by introducing the following approximation

$$\frac{\delta \tilde{L}}{\delta u} \approx \rho - \mu(\nabla u) \det(D^2 u) \quad (3.3)$$

that amounts to approximate $u^{**}(x)$ with $u(x)$ and therefore in considering that we are computing the variation of $\tilde{L}$ always at a u that is convex¹. A priori there is no reason why, while u is evolving according to (3.2), u should keep the convexity property.

2. In our setting, the only information available are data sets drawn from $\rho(x)$ and $\mu(y)$ respectively. Therefore, even if we accept to approximate $u^{**}(x)$ with $u(x)$, we would not be able to use the “pointwise” expression in (3.3).

In the following sections we will propose a method overcoming the issues mentioned above. We will first describe the implication of using directly the approach exposed in Chapter 2 replacing the log-likelihood with $\tilde{L}$. We will then underline the difficulties connected with this approach to motivate a new primal-dual algorithm.

The feature that makes the functional in (1.20) appealing under our point of view is that $\tilde{L}$ is given by the sum of two averages with respect to the starting and the target density respectively. It appears natural therefore to approximate $\tilde{L}$ via its Monte Carlo estimate as follows

$$\tilde{L}(u) \approx LL(u) \equiv \frac{1}{N} \sum_{i=1}^N u(x_i) + \frac{1}{M} \sum_{j=1}^M u^*(y_j) \quad (3.4)$$

where $\{x_i\}$ and $\{y_j\}$ are the sets of points sampled from ρ and μ respectively.

Analogously to what is done in Chapter 2, in order to minimize LL , we could propose a parametric family of continuous functions u_β and minimize the approximation in (3.4)

¹Even if u is not convex it is possible to show that u^* and u^{**} are convex functions. Therefore, the equality $u^{**} = u$ implies the convexity of u .

over all the possible values of β .

By doing this, though, we would introduce an additional complication due to the fact that computing the derivative of $LL(u_\beta)$ with respect to β cannot be done exactly when the functional form of u is too complicated. In order to illustrate this fact let us indicate with $u^n(x)$ the approximation of $\bar{u}$ at time step n . In analogy to what was proposed in Chapter 2 we define the following dynamics for u^n :

$$\begin{cases} u^n(x) = u^{n-1}(x) + \beta^n F(x) \\ u^0(x) = \frac{x^2}{2} \end{cases} \quad (3.5)$$

where F is an elementary function with radial symmetry centered at the point $\tilde{x}$.

The scalar β^n is chosen such to minimize

$$LL^n(\beta) = \frac{1}{N} \sum_{i=1}^N [u^n(x_i) + \beta F(x_i)] + \frac{1}{M} \sum_{j=1}^M (u^n + \beta F)^*(y_j) \quad (3.6)$$

and a way of performing such minimization is by choosing

$$\beta^n \propto \left. \frac{\partial LL^n(\beta)}{\partial \beta} \right|_{\beta=0}. \quad (3.7)$$

The second term of (3.4) contains the convex conjugate of $u + \beta F$

$$(u + \beta F)^*(y_j) = \max_{x \in K_1} [xy_j - u^n(x) - \beta F(x)] \quad (3.8)$$

and its derivative with respect to β evaluated at $\beta = 0$ is given by

$$\left. \frac{\partial (u + \beta F)^*(y_j)}{\partial \beta} \right|_{\beta=0} = -F(\bar{x}_j(0)). \quad (3.9)$$

where

$$\bar{x}_j(\beta) = \arg \max_{x \in K_1} [xy_j - u^n(x) - \beta F(x)] \quad (3.10)$$

The value of $\bar{x}_j(0)$ cannot be computed analytically and one possibility is to compute its approximate value according to

$$\bar{x}_j(0) = \arg \max_{x \in K_1} [xy_j - u^n(x)] \approx \arg \max_i [x_i y_j - u^n(x_i)] \quad (3.11)$$

where we are taking the $\arg \max$ over the set of points $\{x_i\}$ sampled from $\rho(x)$. The closer the points $\{x_i\}$ are to $\bar{x}_j(0)$ the better is the quality of the approximation in (3.11).

The low accuracy of the approximation in (3.11) may be only one of the problems related with this approach. Another issue concerns the uniqueness of the map that needs to be enforced while updating the potential u . This was somehow done in Chapter 2 by enforcing strict convexity of the map.

3.2 A Primal-Dual Approach

In this section, we will propose a modified version of the algorithm described at the end of section (3.1). In particular we will devise a method that avoids the approximation in (3.11) and approximating the function u^{**} with u as described in (3.3).

If on the one hand such method is very efficient and simple to implement in order to compute a map transporting ρ into μ , on the other hand the computed map is in general not optimal in the sense described in Chapter 1.

We will deal with this problem in section (3.2.3) as it appears from the literature that the problem of finding an efficient way of mapping two probability densities by just using data sets drawn from them, it is an interesting problem per se. It allows us, for instance, to compare observations from different data sets and is often used in statistics to use tools that in general are applicable only to normal distributions. Moreover from the point of view of optimal transport, there are methods that are based on two steps procedures in which first the starting density is mapped to the target, and then the

obtained map is refined in order to be the gradient of a convex function (see for instance [18]).

Rather than minimizing $\tilde{L}$ directly, we find the map from ρ to μ by means of the following gradient flow:

$$\begin{cases} \dot{z} = \nabla_z \left[\frac{\delta \tilde{L}_t}{\delta u} \right]_{u=\frac{z^2}{2}} \\ z(t=0) = x \end{cases} \quad (3.12)$$

where

$$\tilde{L}_t = \int u(x) \rho_t(x) dx + \int u^*(y) \mu(y) dy \quad (3.13)$$

and ρ_t is the probability density underlying the points $z_i(t)$ that are obtained by evolving the sampled points $\{x_i\}$ with the flow in (3.12)².

In order to discuss the action of (3.12), let us for simplicity consider the time discrete case of (3.12) with time step length h . Starting at $t = 0$, with $z(t = 0) = x$, we first compute the variation of $\tilde{L}_0$ evaluated at $u(z) = \frac{z^2}{2}$ (we remind that $u = 0.5z^2$ is the potential corresponding to the identity map). Such variation defines a potential function $\frac{\delta \tilde{L}_0}{\delta u}$ that we use to move the state variable z according to

$$z(h) = z(0) + h \nabla_z \left[\frac{\delta \tilde{L}_0}{\delta u}(z) \right]_{u=\frac{z^2}{2}} \quad (3.14)$$

The distribution of the variable $z(h)$ defines a new probability density $\rho_{t=h}$ which in turn defines a new functional

$$\tilde{L}_h = \int u \rho_h + \int u^* \mu. \quad (3.15)$$

At each time step k , we evolve the variable z according to the gradient of the variation of $\tilde{L}_{kh}$ defined using the density underlying the z at that step. The algorithm has a primal-dual nature in the sense that to map the points $\{x_i\}$, related to the primal minimization problem, we follow at each step the flow of the functional defined in (1.20) relative to

² $\rho_{t=0}$ corresponds therefore to the starting density ρ

the dual problem.

In the next paragraph, we will show why and in which sense ρ_t converges to μ under the flow defined in (3.12). Substituting the formula in (1.21) to compute the variation of $\tilde{L}_t$ inside (3.12) we obtain

$$\begin{cases} \dot{z} = -\nabla_z(\rho_t(z) - \mu(z)) \\ z(0) = x \end{cases} \quad (3.16)$$

where $\rho_t(z)$ is the probability density function of the variable z . By considering that ρ_t satisfies the continuity equation

$$\frac{\partial \rho_t(z)}{\partial t} + \nabla_z \cdot [\rho_t(z) \dot{z}] = 0 \quad (3.17)$$

we obtain the following PDE for the density ρ_t :

$$\frac{\partial \rho_t(z)}{\partial t} - \nabla_z \cdot [\rho_t(z) \nabla_z(\rho_t(z) - \mu(z))] = 0. \quad (3.18)$$

with initial condition $\rho_{t=0}(x) = \rho(x)$.

A simple calculation shows that $\rho_t \xrightarrow{t \rightarrow \infty} \mu$ in the L^2 norm. In particular, once we define

$$\|\rho_t - \mu\|^2 = \int (\rho_t(x) - \mu(x))^2 dx, \quad (3.19)$$

we have that

$$\begin{aligned} \frac{d}{dt} \frac{\|\rho_t - \mu\|^2}{2} &= \int \frac{d}{dt} \frac{(\rho_t - \mu)^2}{2} dz = - \int (\rho_t - \mu) \nabla \cdot (\rho_t \dot{z}) dz = \\ &= - \int \nabla(\rho_t - \mu) \cdot (\rho_t \dot{z}) dz = - \int \rho_t |\nabla(\rho_t - \mu)|^2 dz \leq 0 \end{aligned} \quad (3.20)$$

where in the second equality we used (3.17), in the third equality we don't have the boundary term in the integration by parts since the tails of the densities tend to zero at

infinity, and in the last equality we used (3.16). Notice that $|\nabla(\rho_t - \mu)| = 0$ only when $\rho_t \propto \mu$ and, because ρ_t and μ are probability densities, this cannot happen unless ρ_t is equal to μ almost everywhere.

Equation (3.20) is saying that, if at each iteration step we map ρ_t according to the gradient flow of $\tilde{L}_t$, then we will get a sequence of densities converging (in the L^2 sense) to μ . This shows that the algorithm determines a map mapping ρ into μ . As shown in the next section such map is in general not curl-free, namely it is not in general the gradient of a convex potential.

A numerical experiment using a grid in space

In this section, we will perform a numerical experiment to further characterize the nature of the map computed using the flow in (3.12). In particular we will solve equation (3.18) on a periodic domain using the pseudo-spectral method implemented with a second order Adams-Bashforth time stepping scheme. The initial density (plotted in the upper left panel of figure (3.1)) is given by

$$\rho(x_1, x_2) = \frac{\cos(0.5x_1)^2 + \sin(0.5x_2)^2}{4\pi^2}, \quad (3.21)$$

while the target density (lower right panel of (3.1)) $\mu(x_1, x_2)$ is just obtained from ρ by a translation of π along the x_2 coordinate. Even though the starting and the target densities are obtained one from the other by a simple translation, it is not clear a priori what is the optimal map from ρ to μ . This is due to the periodic boundary conditions that have the effect of splitting the mass at some point of the domain favoring a rearrangement that is “cheaper” than a rigid translation moving all the mass by the same quantity.

In figure (3.1) it is shown the time evolution of the density ρ satisfying the equation (3.18). As can be seen ρ splits at $x_2 = \frac{\pi}{2}$ and $x_2 = \frac{3\pi}{2}$ in order to rearrange according to the target density μ . There are some considerations that need to be made on this numerical experiment:

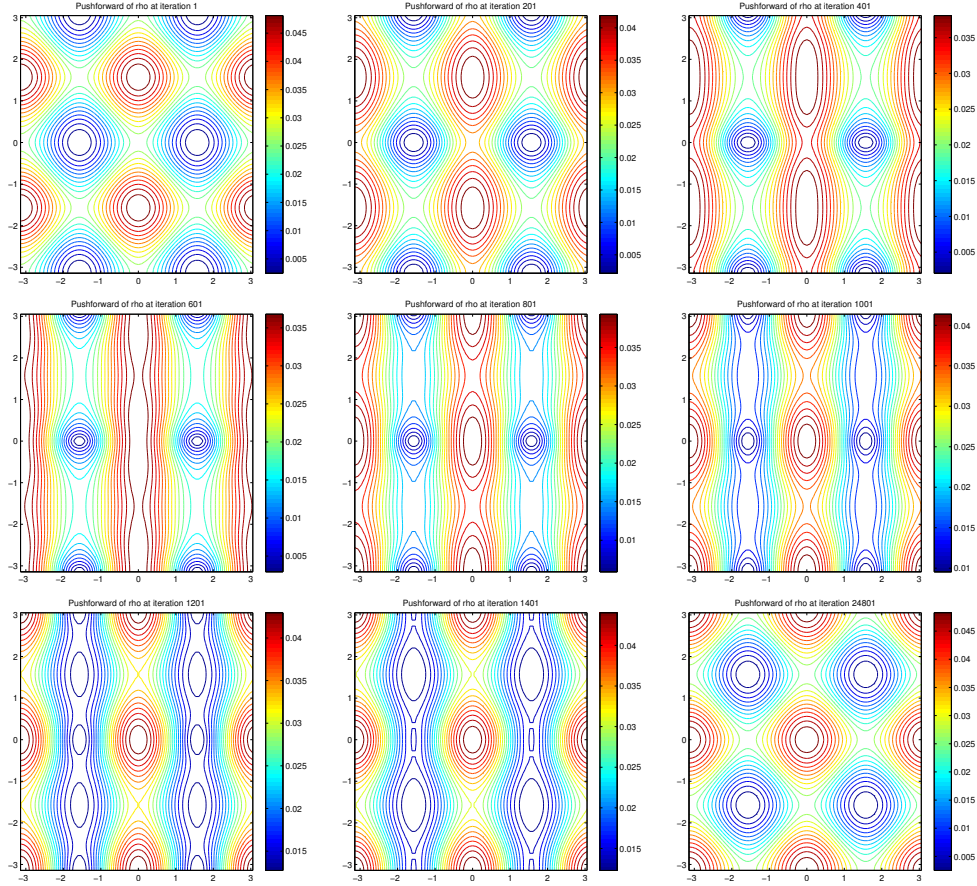


Figure 3.1: Time evolution of the probability density $\rho(x_1, x_2)$, defined in (3.21) and represented in the upper left panel, to the target density μ obtained from ρ by a translation of $\pi/2$ of the x_2 coordinate (lower right right).

- As expected, the starting density ρ is mapped into μ .
- Such map is not a rigid translation. This is in accordance with analogous numerical experiments (see [3] and [18], for instance) in which optimal transport was investigated on periodic domain between probability distributions obtained one from the other by a rigid translation.

The second point may lead to think that the approach described in section (3.2) does not only find a map from ρ to μ but somehow finds the optimal map. To further speculate the optimality of the numerical map, we computed the value of the curl and

the divergence of the displacement field of the map to see whether the first one can be considered negligible compared to the second one.

Because the displacement field is updated at every time in the z space, particular care has to be taken in computing the curl of the displacement field in the original x space. This amounts to evaluate a function on a non structured grid given a uniform sampling of its Fourier modes. The task is achieved using a fast interpolating method developed by Greengard and Lee and described in [17].

The average value of the divergence and the curl of the displacement field of the map are reported in figure (3.2). They show that, even though the L^2 norm of the curl is about two order of magnitude smaller than the L^2 norm of the divergence, the map is not curl-free (see upper right panel). In the next section, following the spirit of what is described in Chapter 2, we will describe the equivalent mesh-free version of the flow algorithm described above. We will then propose a projection method to further improve the optimality of the map.

3.2.1 Mesh-free implementation

In order to reformulate the algorithm described in section (3.2) in a mesh-free framework, let us fix some notation:

- Let $z^k(x)$ be the map at the k -th iteration step. We will also define $x_i^k = z^k(x_i)$ to be the points obtained by applying z^k to the original sample points $\{x_i\}$ drawn from $\rho(x)$. We define $z^0(x) = x$
- Let $\rho^k(x)$ be the density that is obtained by mapping $\rho(x)$ via the map $z^k(x)$ and let

$$LL^k \equiv \frac{1}{N} \sum_{i=1}^N u(x_i^k) + \frac{1}{M} \sum_{j=1}^M u^*(y_j) \quad (3.22)$$

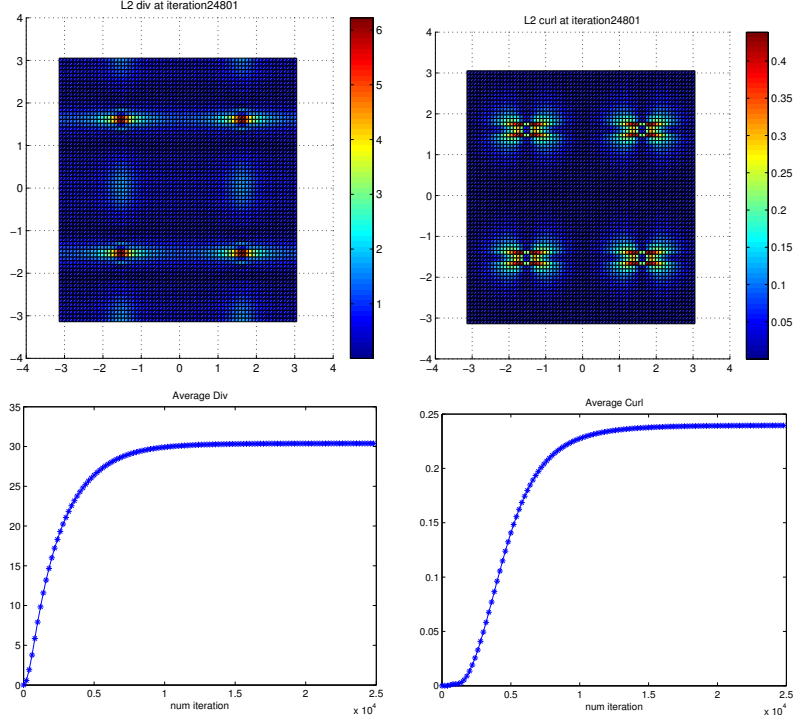


Figure 3.2: Given the displacement field $M(x) = z(x) - x$ of the map, we compute the average divergence, $\int \rho(x) \|\nabla \cdot M(x)\|^2 dx$, and the average curl, $\int \rho(x) \|\nabla \times M(x)\|^2 dx$, as the iteration step advances (lower left and right panels, respectively). The upper panels report the pointwise absolute value of $\nabla \cdot M(x)$ and $\nabla \times M(x)$ on the periodic domain of the steady displacement field.

be the Monte Carlo estimate of the functional (1.20) at the k -th iteration step

$$\tilde{L}^k(u) = \int_{K_1} u(x) \rho^k(x) dx + \int_{K_2} u^*(y) \mu(y) dy. \quad (3.23)$$

In the mesh-free version of the algorithm rather than following the gradient of the variation $\tilde{L}_t$, we restrict the elementary map to a family of maps that are perturbations of the identity map. Given the density $\rho^k(x)$ and a set of N points $\{x_i^k\}$ sampled from it:

1. Compute $\bar{\beta}$ in order to minimize the quantity

$$LL^k(\beta) = \frac{1}{N} \sum_{i=1}^N \left[\frac{(x_i^k)^2}{2} + \beta F^k(x_i^k) \right] + \frac{1}{M} \sum_{j=1}^M \left(\frac{x^2}{2} + \beta F^k(x) \right)^* (y_j) \quad (3.24)$$

where $\phi_\beta(x) = \frac{x^2}{2} + \beta F^k(x)$ is a parametric family of functions over which we are taking the minimization step.

This can be done by gradient descent

$$\bar{\beta} \propto \left. \frac{\partial LL^k(\beta)}{\partial \beta} \right|_{\beta=0} \quad (3.25)$$

or by Newton method as shown in section (3.2.1).

2. Compute

$$x_i^{k+1} = \nabla \phi_{\bar{\beta}}(x_i^k) \quad (3.26)$$

and define $\rho^{k+1}(x)$

$$\rho^k(x) = J^{\nabla \phi_{\bar{\beta}}}(x) \rho^{k+1}(\nabla \phi_{\bar{\beta}}(x)) \quad (3.27)$$

where $J^{\nabla \phi_{\bar{\beta}}}(x)$ is the Jacobian of the map $\nabla \phi_{\bar{\beta}}$

3. Repeat step 1 with LL^k replaced by

$$LL_{\beta}^{k+1} = \frac{1}{N} \sum_{i=1}^N \left[\frac{(x_i^{k+1})^2}{2} + \beta F^{k+1}(x_i^{k+1}) \right] + \frac{1}{M} \sum_{j=1}^M \left(\frac{x^2}{2} + \beta F^{k+1}(x) \right)^* (y_j) \quad (3.28)$$

From the numerical point of view the algorithm converges to a certain density ρ^κ when $\beta \approx 0$ at the κ -th iteration and all the successive iterations (we anticipate that the function F appearing in (3.24) changes at each iteration step in a way we will specify later). Since the derivative of $\tilde{L} \left(\frac{x^2}{2} + \beta F^k(x) \right)$ with respect to β is

$$\left. \frac{\partial \tilde{L} \left(\frac{x^2}{2} + \beta F^k(x) \right)}{\partial \beta} \right|_{\beta=0} = \int F^k(x) \rho^k(x) dx - \int F^k(y) \mu(y) dy. \quad (3.29)$$

then $\beta \approx 0$ corresponds to

$$\int F^\kappa(x) \rho^\kappa(x) dx - \int F^\kappa(y) \mu(y) dy \approx 0 \quad (3.30)$$

that, if we choose F^κ among a big enough family of functions, it is equivalent to affirm that $\rho^\kappa \approx \mu(y)$. We will specify later the shape of the elementary functions F^k keeping in mind that (3.30) should imply somehow that $\rho^\kappa \approx \mu(y)$. Iterating over k the second step of the algorithm we have that

$$\rho(x) = \left[\prod_{i=1}^{\kappa} J^{\nabla \phi^i} \right] \rho^\kappa(\nabla \phi^\kappa \circ \dots \circ \nabla \phi^1)(x) \approx \mu(T(x)), \quad (3.31)$$

where ϕ^i indicates the ϕ in (3.26) at the i -th iteration step. Therefore the map $T(x)$ mapping $\rho(x)$ into $\mu(y)$ is obtained by composing the elementary maps ϕ^i at each time step: $T(x) = \nabla \phi^\kappa \circ \dots \circ \nabla \phi^1$. It is important to notice that, because each elementary map is a one-to-one mass preserving map, the final map $T(x)$ is as well one-to-one and mass preserving.

Computing β

Because of the simplicity of each elementary map, it is convenient to choose β at each time step such to maximize the quadratic approximation to the LL

$$LL \approx LL_0 + G\beta + \frac{1}{2}\beta^T H \beta \quad (3.32)$$

³ where

$$LL_0 = LL|_{\beta=0}, \quad G_i = \left. \frac{\partial LL}{\partial \beta_i} \right|_{\beta=0}, \quad H_{ij} = \left. \frac{\partial^2 LL}{\partial \beta_i \partial \beta_j} \right|_{\beta=0}. \quad (3.33)$$

³In this expression we are considering β as a vector of parameters

Maximizing (3.32) with respect to β yields

$$\beta = -H^{-1}G. \quad (3.34)$$

It may happen that the quantity LL is locally very flat causing the Hessian H to be a singular matrix and resulting in a diverging value of β . In order to avoid this at each iteration step we impose a threshold, called *learning rate* on the modulus of β .

In the following, we will compute the first and second derivative with respect to β of the quantity defined in (3.6). Since computing the derivative of the first sum does not represent a problem, we will directly focus on computing

$$\frac{\partial \left[\left(\frac{x^2}{2} + \beta F \right)^*(y_j) \right]}{\partial \beta} \quad (3.35)$$

where we remind that

$$\left(\frac{x^2}{2} + \beta F \right)^*(y_j) = \max_{x \in K_1} \left[xy_j - \frac{x^2}{2} - \beta F(x) \right]. \quad (3.36)$$

Let

$$\bar{x}_j(\beta) = \arg \max_{x \in K_1} \left[xy_j - \frac{x^2}{2} - \beta F(x) \right] \quad (3.37)$$

then

$$\begin{aligned} \left. \frac{\partial \left(\frac{x^2}{2} + \beta F \right)^*(y_j)}{\partial \beta} \right|_{\beta=0} &= \frac{\partial \bar{x}_j(\beta)}{\partial \beta} \left[y_j - \frac{\bar{x}_j(\beta)^2}{2} - \beta F'(\bar{x}_j(\beta)) \right]_{\beta=0} + \\ &\quad - F(\bar{x}_j(\beta))|_{\beta=0} = -F(\bar{x}_j(0)) \end{aligned} \quad (3.38)$$

where in the second equality of (3.38) we used

$$y_j = \bar{x}_j(\beta) + \beta F'(\bar{x}_j(\beta)) \quad (3.39)$$

assuming that in correspondence of the maximum $\bar{x}_j(\beta)$ the first derivative of $xy_j - \frac{x^2}{2} - \beta F(x)$ is zero. It is important to notice because $|\beta| \ll 1$, the function $xy - 0.5x^2 - \beta F$ is a convex function of x with a unique maximum located at the point y .

Concerning the second derivative we have that

$$\left. \frac{\partial^2 [(u + \beta F)^*(y_j)]}{\partial \beta^2} \right|_{\beta=0} = -F'(\bar{x}_j(0)) \left. \frac{\partial \bar{x}_j(\beta)}{\partial \beta} \right|_{\beta=0}. \quad (3.40)$$

In order to compute $\frac{\partial \bar{x}_j(\beta)}{\partial \beta}$ we differentiate eq. (3.39) on both sides with respect to β in order to obtain

$$\left. \frac{\partial \bar{x}_j(\beta)}{\partial \beta} \right|_{\beta=0} = -F'(\bar{x}_j(0)) \quad (3.41)$$

Using (3.38), (3.40) and (3.41) together with (3.6) we get closed expressions for the quantities in (3.33):

$$\left. \frac{\partial LL^{n+1}(\beta)}{\partial \beta} \right|_{\beta=0} = \frac{1}{N} \sum_{i=1}^N F(x_i) - \frac{1}{M} \sum_{j=1}^M F(\bar{x}_j(0)) \quad (3.42)$$

$$\left. \frac{\partial^2 LL^{n+1}(\beta)}{\partial \beta^2} \right|_{\beta=0} = \frac{1}{M} \sum_{j=1}^M [F'(\bar{x}_j(0))]^2 \quad (3.43)$$

The only information that is missing in order to compute (3.42) and (3.43) is about the specific form of the elementary potential F and the value of $\bar{x}_j(0)$.

Regarding the choice of the elementary potential F we repeat here the same considerations we reported in Chapter 2. The potential F should be easy to compute, have radial symmetry in order for the algorithm to perform well in high dimensional spaces, and should be characterized by a typical length scale α to avoid the problem of over resolving (see section 2.1.1.2).

Concerning the value of $\bar{x}_j(0)$, if we assume that equation (3.39) holds then, by setting $\beta = 0$, we obtain $\bar{x}_j(0) = y_j$. Since the $\arg \max$ in (3.37) is defined over $x \in K_1$ saying that $\bar{x}_j(0) = y_j$ is equivalent to assume that the support of the target density μ is

contained in the support of the starting probability density ρ . This may not always be the case in practical applications. We will proposed a possible solution to this problem in the section (3.2.2)

3.2.2 Preconditioning

As we mentioned at the end of section (3.2.1) having that $K_2 \subseteq K_1$ simplifies the computation of $\bar{x}_j(0)$ since in this case results $\bar{x}_j(0) = y_j$. In order to deal with cases in which $K_2 \not\subseteq K_1$, we notice that we can always pre-treat the original data samples to have that $K_2 \subseteq K_1$ without compromising the computation of the optimal map of the original problem. This can be done by using isotropic stretching and rigid translation maps. The reason why we are authorized to do so, is that this kind of transformations can be composed with optimal maps giving still an optimal map as a result.

In order to show that the composition of a gradient of a convex function with a translation or an isotropic stretching are still gradients of convex functions it is sufficient to realize that, given $g(x) = \nabla_x \phi(x)$ with ϕ convex, and given $s(x) = ax + b$, the map resulting from the composition $c(x) = g \circ s(x)$ can be rewritten as $c(x) = \nabla_x \frac{1}{a} \phi(ax + b)$ where $\frac{1}{a} \phi(ax + b)$ it is still a convex function. Another way to look at it is to notice that the Jacobian matrix of the composition of a gradient of a convex function and a map of the form $y = ax + b$ is given by the product of two symmetric positive definite matrices that commute. Such product therefore is still a symmetric positive definite matrix. Using the n -dimensional formulation of the Stokes theorem it is possible to show that a vector field with symmetric positive definite Jacobian matrix is the gradient of a convex scalar function.

3.2.3 Enforcing the optimality of the map

In section (3.2) we showed that the flow in (3.12) determines a one-to-one mass preserving map mapping ρ to μ . We investigated the optimality of the map via a numerical

experiment performed on a spatial grid and observed that, even though the curl of the displacement field of the map is small when compared to the divergence, the map is not curl-free as required by optimal transport theory.

In this section, we will propose a method to improve the optimality of the map. In order to do so we start by considering the work described in [18]. Given a mass preserving map $w(x)$ mapping ρ into μ , the authors of [18] presented a method to eliminate the divergence-free part of the field $w(x)$ so that the resulting map is the gradient of a convex function still mapping ρ into μ .

The idea laying behind such method is related to a fundamental result in optimal transport theory due to Brenier and Gangbo ([5] and [16] respectively) saying that given any mass preserving map $w(x)$ mapping ρ into μ , $w(x)$ admits the unique decomposition of the form

$$w(x) = \nabla\phi \circ s(x), \quad (3.44)$$

where ϕ is a scalar convex function and s is a mass preserving map, mapping ρ into itself.

Given $w(x)$ we can therefore look for the mass preserving map s^{-1} satisfying $w \circ s^{-1} = \nabla\phi$ in order to obtain the optimal map. Because the map s^{-1} modifies the action of w preserving the mapping from ρ into μ , we say that s^{-1} acts as a *rearrangement* map.

While in [18] the rearrangement map was found using a grid in space by means of a constraint minimization process, in our context the goal is to rearrange the points $\{w(x_i)\}$, with w any mass preserving map, in order to minimize

$$\frac{1}{M} \sum_{i=1}^M |w(x_i) - x_i|^2 \quad (3.45)$$

while keeping the underlying distribution of $\{w(x_i)\}$ fixed to μ .

We will pursue this goal without recurring to a constraint minimization algorithm, but rather “rearranging” the points, originally distributed according to ρ , in a gradual

manner while they are mapped to μ . The idea is to alternate iteration steps in which we move according to the flow described by (3.12) with steps in which we move the sampled points compromising in between the direction of the flow and the direction minimizing the cost of the map. In the following, we will described how to implement this second category of steps that will be called “projection steps”.

Rather than a one parameter family of elementary potential, the elementary potential for the projection step depends on two parameters β_1 and β_2 . In particular, given $\beta = (\beta_1, \beta_2)$, we used the elementary potential

$$\phi_\beta(x) = \frac{x^2}{2} + \beta_1 F_1(|x - x_1|) + \beta_2 F_2(|x - x_2|) \quad (3.46)$$

where x_1 and x_2 are two points chosen at random among the sampled points from ρ and F_1 and F_2 have the form defined in (2.20).

Let

$$LL^k(\beta) = \frac{1}{N} \sum_{i=1}^N \phi_\beta(x_i^k) + \frac{1}{M} \sum_{j=1}^M \phi_\beta^*(y_j) \quad (3.47)$$

and

$$C^k(\beta) = \frac{1}{N} \sum_i^N \left| \nabla \phi_\beta(z_i^k) - x_i \right|^2 \quad (3.48)$$

be the Monte Carlo estimates of the functional $\tilde{L}$ and the L^2 cost respectively. Given the elementary potential in (3.46), a projection step is described by the following algorithm:

1. Compute $\beta_L = \nabla_\beta LL^k(\beta)|_{\beta=0}$ and $\beta_C = \nabla_\beta C^k(\beta)|_{\beta=0}$
2. If $\langle \beta_L, \beta_C \rangle \geq 0$ then define $\bar{\beta} = \beta_L$ and move the sampled points according to $x^{k+1} = x^k + \nabla_x \phi_{\bar{\beta}}$
3. Otherwise compute the $\bar{\beta}$ such that the relation $\langle \beta_L, \bar{\beta} \rangle = \langle \beta_C, \bar{\beta} \rangle$ is satisfied and again move the points according to $x^{k+1} = x^k + \nabla_x \phi_{\bar{\beta}}$

In words, when the angle between the direction β_L , minimizing LL^k , and the direction

β_C , minimizing C^k , is less than $\pi/2$ we move according to β_L . If on the other hand then angle in between β_L and β_C is bigger than $\pi/2$ then we choose the direction along which β_C and β_L have the same projection. Such direction by definition forms an angle smaller than $\pi/2$ with both β_C and β_L that assures we are moving the points to a new configuration that has a lower cost without increasing the value of LL^k .

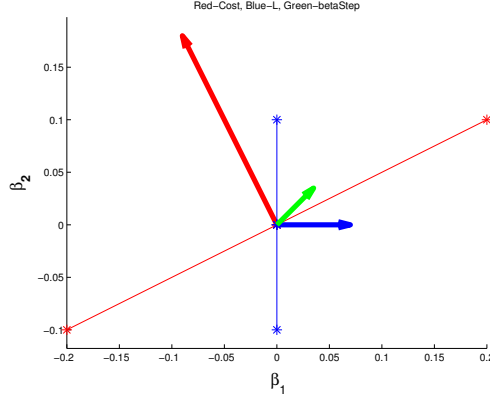


Figure 3.3: Projection step illustration. The red arrows represent β_C while the thinner red line segment delimited by the * symbol, is perpendicular to β_C . Similarly for β_L (blue arrow). The green arrow represents β .

3.3 Numerical results

In this section we will present some numerical results relative to the mesh-free implementation of the method described in this chapter.

The numerical experiments are performed using artificial data set drawn from densities with known analytic form. This does not only allow for an easy construction of examples in which it is possible to compute the optimal map analytically, but offers also the possibility of exploring how the algorithm described in the previous sections can be used to perform density estimation in the spirit of what is described in [28].

Let us suppose to have a data set $\{x_i\}$ sampled from ρ and to choose μ to be a Normal distribution. In this case, we can find a map $T(x)$ from ρ to μ using the pro-

cedure explained in the previous sections, either by using data drawn from the Normal distribution, or by choosing F such that the quantity

$$\frac{\partial}{\partial \beta} \left[\int \left(\frac{x^2}{2} + \beta F(x) \right)^* (y) \frac{e^{-\frac{|y|^2}{2}}}{(2\pi)^{d/2}} dy \right]_{\beta=0} \quad (3.49)$$

can be computed analytically.

Because we know the exact target density, we can actually use the map $T(x)$ to estimate the starting density ρ by the change of variable formula $\rho_{est}(x) = J^T(x)\mu(T(x))$. This is done by evaluating μ and the Jacobian of the map $J^T(x)$ on a regular grid of Lagrangian markers x_g that move passively while computing $T(x)$.

It is important to notice for this purpose, that the Jacobians of the gradients of the two elementary potentials defined in (2.20) and (3.46) can be computed in the closed form given by

$$J^{\nabla \phi} = a^{n-1} [a + br] \quad (3.50)$$

and

$$J^{\nabla \phi} = e^{n-2} [e^2 + ebr_1 + ecr_2 + bc(r_1^2 r_2^2 - \langle r_1, r_2 \rangle)] \quad (3.51)$$

for (2.20) and (3.46) respectively. Given that we defined

$$r = |x - \tilde{x}|, \quad r_1 = |x - \tilde{x}_1| \quad \text{and} \quad r_2 = |x - \tilde{x}_2| \quad (3.52)$$

the remaining quantities appearing in (3.50) and (3.51) are defined as

$$a = 1 + \beta f, \quad b = \beta_1 \frac{1}{r_1} \frac{df_1}{dr}, \quad c = \beta_2 \frac{1}{r_2} \frac{df_2}{dr} \quad \text{and} \quad e = 1 + \beta_1 f_1 + \beta_2 f_2 \quad (3.53)$$

where

$$f(r) = \frac{1}{r} \frac{dF}{dr} \quad (3.54)$$

and n is the dimension of the ambient space where x is defined.

In this section we will use density estimation to monitor the quality of the map. In particular we will report the contour lines of the estimated density ρ_{est} and the exact starting density ρ obtained using the Lagrangian markers x_g (see for instance figure (3.4)).

By repeating the same argument, we can estimate the target density μ by supposing the knowledge of the analytic form of ρ and a data set $\{y_j\}$ sampled from μ . In this case the estimated target density will be given by

$$\mu_{est}(T(x_g)) = \frac{\rho(x_g)}{J^T(x_g)} \quad (3.55)$$

In addition to performing density estimation, we use the value of $T(x)$ and the theoretical value of ρ , both evaluated on the grid points x_g , to compute:

- the Kullback-Leibler divergence in between $\rho^k(x_g) = J^k(x_g)\mu(z^k(x_g))$ (where z^k is the map at the k -th iteration step) and the exact starting densities $\rho(x_g)$.
- the L^2 cost of the map $\int \rho(x)|z^k(x) - x|^2 dx$
- the average error, computed on the sampled points, in between z^k and the exact optimal map $\bar{T}$ given by $\frac{1}{N} \sum_{i=1}^N |z^k(x_i) - \bar{T}(x_i)|^2$
- the absolute error in between the numerical optimal map T and $\bar{T}$ on the grid points x_g .

Non isotropic Gaussian to non isotropic Gaussian

In figures (3.4) and (3.5) are shown results relative to optimal transport of a non isotropic Gaussian distribution ρ into a different non isotropic Gaussian distribution μ . Variants of this numerical experiment are often used in literature (see for instance [13] and [23]) to test the optimality of the map transporting ρ into μ .

In particular it is possible to find the analytic form of the optimal map between two non isotropic Gaussian distributions as described in [10]. Let Σ_1 and Σ_2 be the two

non singular covariant matrices characterizing ρ and μ respectively (we will assume both distributions to have mean zero) the optimal map is then given by

$$\bar{T} = \Sigma_2^{1/2} \Sigma_0^{-1} \Sigma_2^{1/2} \quad (3.56)$$

where

$$\Sigma_0 = \left(\Sigma_2^{1/2} \Sigma_1 \Sigma_2^{1/2} \right)^{1/2} \quad (3.57)$$

and where $N = M^{1/2}$ indicates the unique positive square root matrix of the positive definite matrix M such that $N^t N = M$.

In the upper left and right panels of figure (3.4) are displayed the initial ρ and the target density μ obtained using

$$\Sigma_1 = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \quad \Sigma_2 = \begin{pmatrix} 0.5 & 0.25 \\ 0.25 & 0.25 \end{pmatrix} \quad (3.58)$$

respectively. The fact that the KL divergence is converging to zero (see figure (3.5)) and that the cost of the map z^k is approaching the cost of the the exact map $\bar{T}$ (both evaluated using the grid points x_g and reported in the panel in the middle row of the second column of figure (3.5)) shows that z^k is converging to $\bar{T}$ as the iteration step k advances. This is confirmed by the two lower panels reporting the average error (left) and the pointwise absolute value of the difference between the numerical map and the exact one.

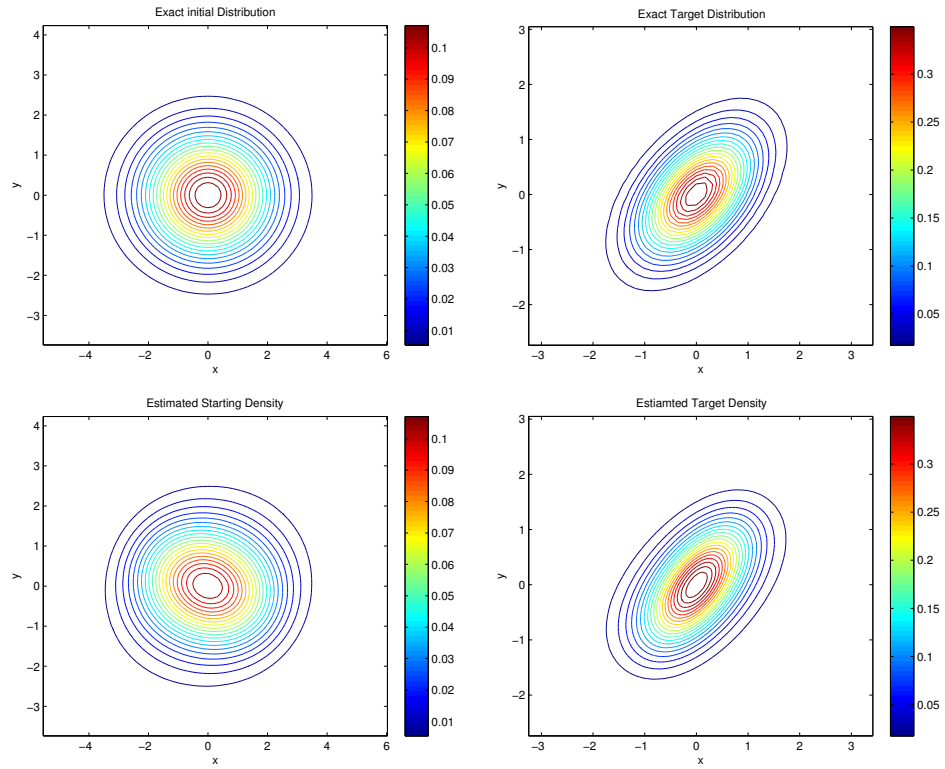


Figure 3.4: Upper panels: Exact starting density ρ (left) and the exact target density μ (right). Lower panels: Estimated starting density ρ_{est} (left) and estimated target density μ_{est} (right). The algorithm is implemented using 1000 points sampled from ρ and μ respectively.

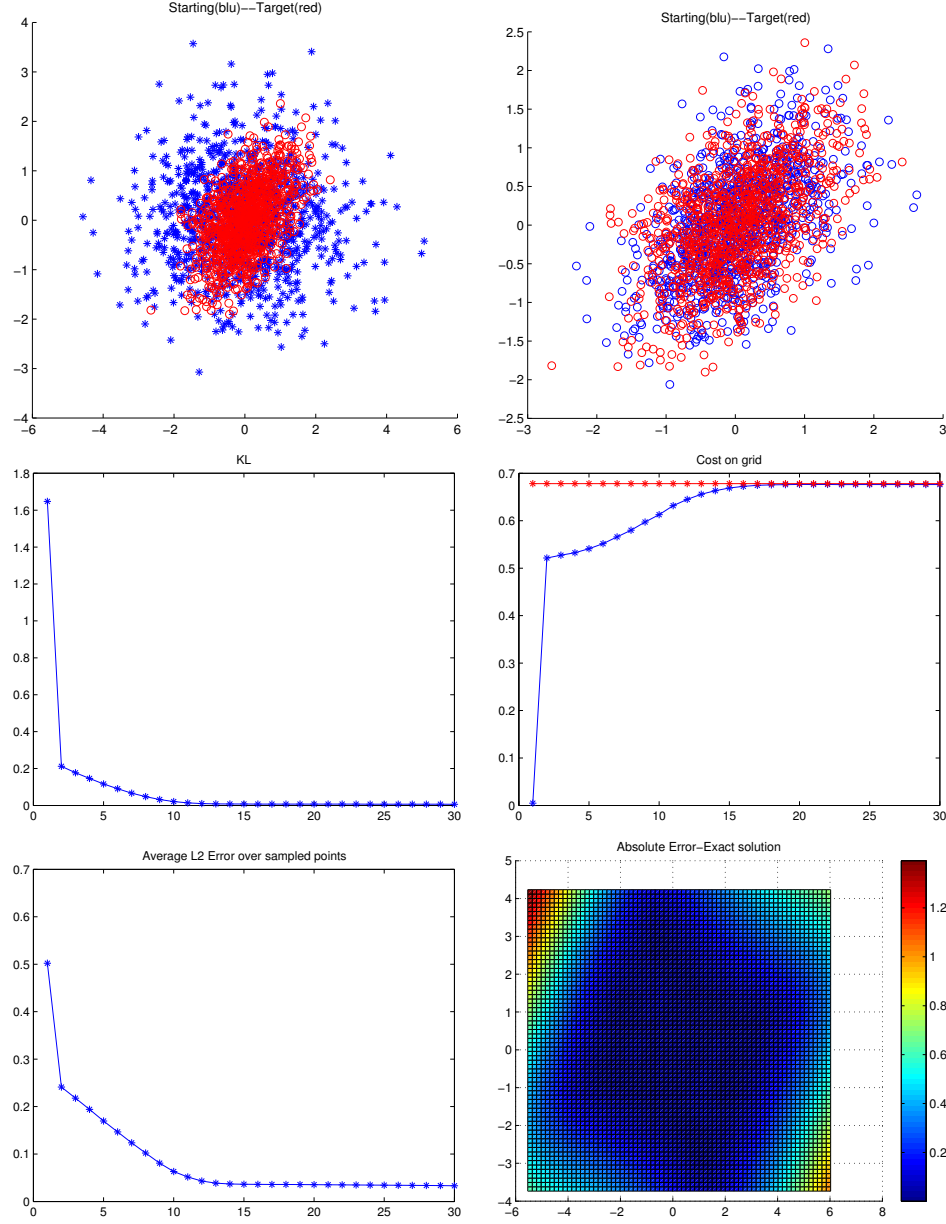


Figure 3.5: First row: On the left panel are displayed the sample points independently drawn from ρ (blue) and μ (red), on the right are reported the points mapped with the numerical solution obtained from the algorithm. Second row: Leibler-Kullback divergence in between $\rho^k(x)$ and $\rho(x)$ on the left; on the right panel are reported the cost of the map z^k at iteration k (in blue) and the cost of the optimal map $\bar{T}$ (in red). Third row: Average error (left) and absolute value of the difference in between z^k and $\bar{T}$ (right).

Mass splitting in 2 dimension

In this subsection we give an example of mass splitting. The map is computed in between a normal distribution and the target distribution given by

$$\mu(x_1, x_2) = \frac{9x_1x_2}{2\pi} e^{-\frac{x_1^6+x_2^6}{2}}. \quad (3.59)$$

The optimal map is given by

$$\bar{T} = \begin{pmatrix} (1/3)x^{1/3} \\ (1/3)y^{1/3} \end{pmatrix}. \quad (3.60)$$

The computation of the map is more intensive than usual due to a divergence in the gradient of the map along the Cartesian axis. This is generally true when the support of the initial density ρ is connected while that one of μ is not. In this case the mass, initially supported in the domain of ρ , needs to be splitted in the different parts of the domain of μ .

In figure (3.8) we report the results relative to the same map described above computed without using the projection step described in section (3.2.3). In particular we compare the values of the cost and the KL obtained using the projection algorithm (figure (3.7)) with the values obtained without implementing the projection algorithm. Even though the cost of the map is lower when we implement the projection step, the map obtained without using projections step is still a good approximation of the optimal map. This is quite surprising if we consider that the flow described in (3.12) doesn't have memory of the initial density of the sampled points $\{x_i\}$ and therefore in principle should not lead to a map given by the gradient of a potential u minimizing (3.8).

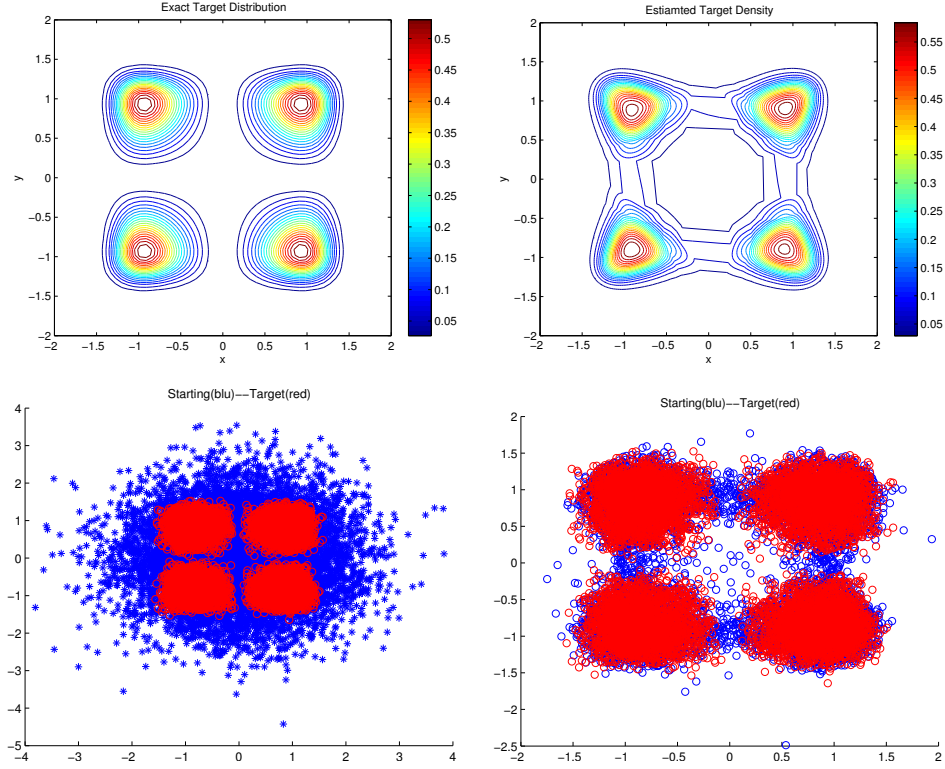


Figure 3.6: Results relative to optimal transport from a normal distribution to the distribution defined in (3.59). The upper right panel represents the estimated target distribution. The lower panels represents the data sampled from the starting and the target density respectively (on the left), and the data mapped with the numerical approximation of the optimal map (on the right). The algorithm is implemented using 10^4 points sampled from ρ and μ respectively.

Compact support

One of the main advantages in computing the optimal map using only sampled points from the starting and the target distributions is the fact that we can avoid to deal with boundary conditions when the distributions have compact support. This is not the case when dealing with PDE methods in which the Monge-Ampere equation needs to be equipped with a set of non trivial boundary conditions (see for instance [13]). In our case, instead, the elementary maps are well defined also outside the support of the starting and the target distributions. The fact that those distributions have compact

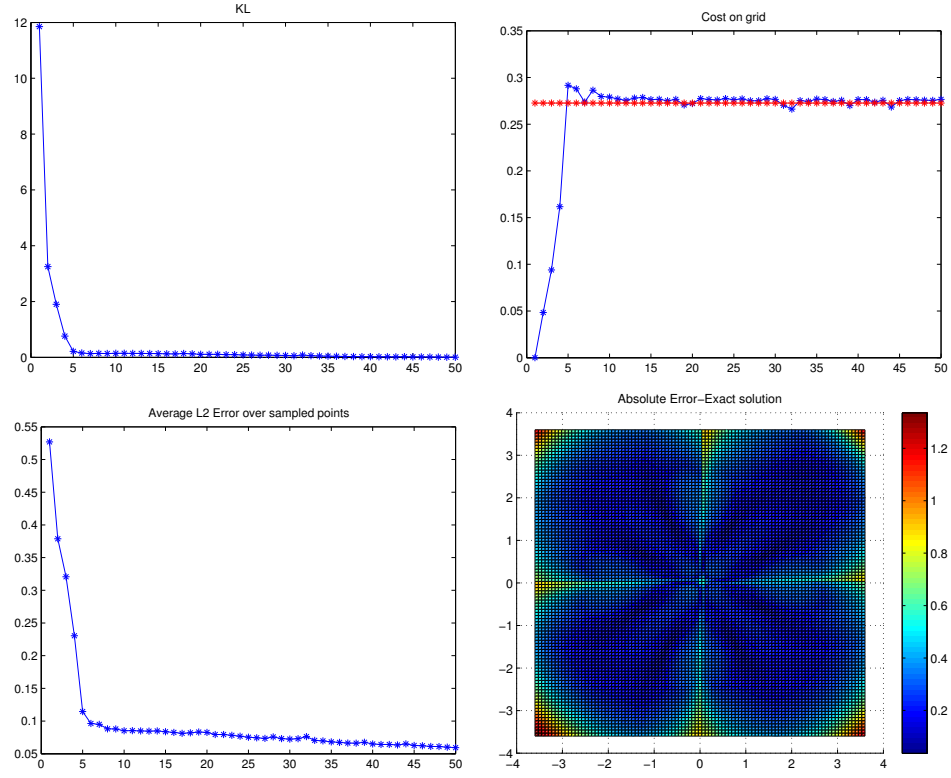


Figure 3.7: Same quantities described in figure (3.5) relative to the map between two non isotropic Gaussian distributions.

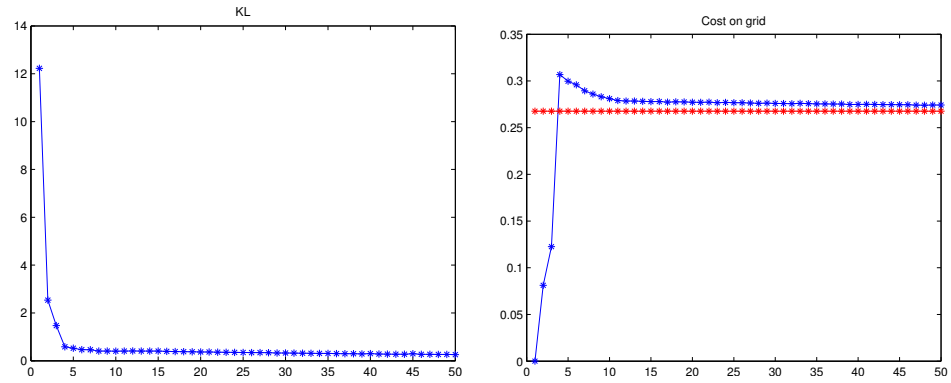


Figure 3.8: To be compared with the upper row panels of figure (3.7). In this case we compute the map without alternating the iteration step described in section (3.2) with the projection step described in (3.2.3).

support is hidden in the Monte-Carlo estimates of the functional (3.6) we are minimizing at each iteration step.

In figure (3.9) we report the result relative to the map from a uniform distribution supported on a square with side length $\ell = 3$ and centered at the origin, to the uniform square distribution of side $\ell = 2$ still centered at the origin.

The points sampled from the starting density (points in blue in the upper left panel of figure (3.9)) are mapped into points whose distribution overlap quite accurately with the support of the target distribution spanned by the red points (upper right panel). This shows that the boundary of the starting distribution is accurately mapped into the boundary of the target distribution.

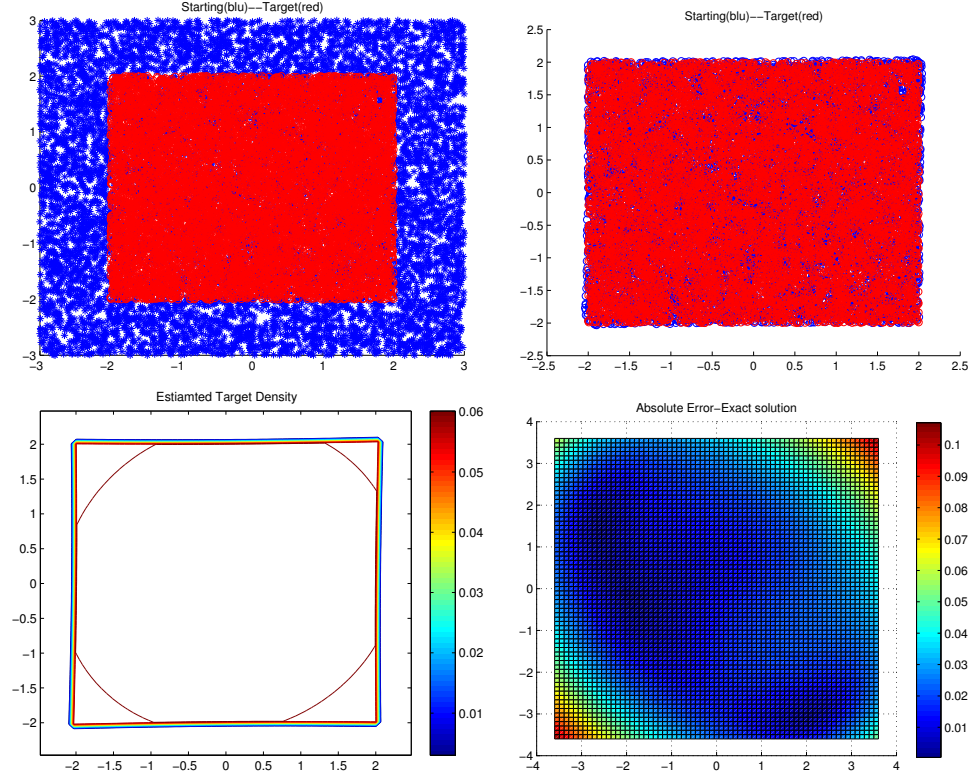


Figure 3.9: Compact support on uniform square. The algorithm is implemented using 10^4 points sampled from ρ and μ respectively.

Higher dimensional tests

An example of optimal transport in dimension higher than three is given in figures (3.10) and (3.11). In this context we are mapping a non isotropic Gaussian with covariance matrix Σ given by a 7×7 diagonal matrix with entries different from zero equal to $\Sigma_{ii} = 1, 1.5, 2, 2.5, \dots, 4$ to a Normal distribution. Figure (3.10) shows the projection along three different planes of the estimated starting density ρ_{est} (panels on the left column). The projections of ρ_{est} should be compared with the values of the exact starting density ρ projected on the same planes (panels on the right column). Each plane is specified by the two coordinates labeling the axis of each panel in figure (3.10) and by setting all the entries relative to the other coordinates to zero. In the first row, for instance, are represented the contour lines of the projection of ρ_{est} (left panel) and ρ (right panel) on the (x_4, x_5) plane.

Given the fact that $\Sigma_{11} = 1$ and $\Sigma_{77} = 4$, we have that the biggest deformation that needs to be performed in order to map ρ into μ occurs along the planes parallel to the (x_1, x_7) plane. By looking at the projection of ρ_{est} on this plane, we see that the projection is not as accurate in reproducing ρ , when compared to the projections along the (x_4, x_5) and (x_2, x_6) planes.

In figure (3.12) we report the value of the execution time per iteration. Such value is obtained using 10^4 points sampled from ρ and μ respectively and a number of dimensions ranging from 2 to 10. The time is computed running the algorithm for a fixed amount of iteration (in this case 10^4) and then taking the average over all their execution times.

Since the computational cost of the algorithm is essentially that one needed to compute β (see equation 3.25), we expect that the time per iteration needed scales linearly as the dimension of the space increases as confirmed by the figure (3.12).

However, this does not mean that the execution time for the algorithm to converge to an acceptable solution scales linearly with the dimension. In general the number of iterations needed for the algorithm to converge to the optimal map depends from

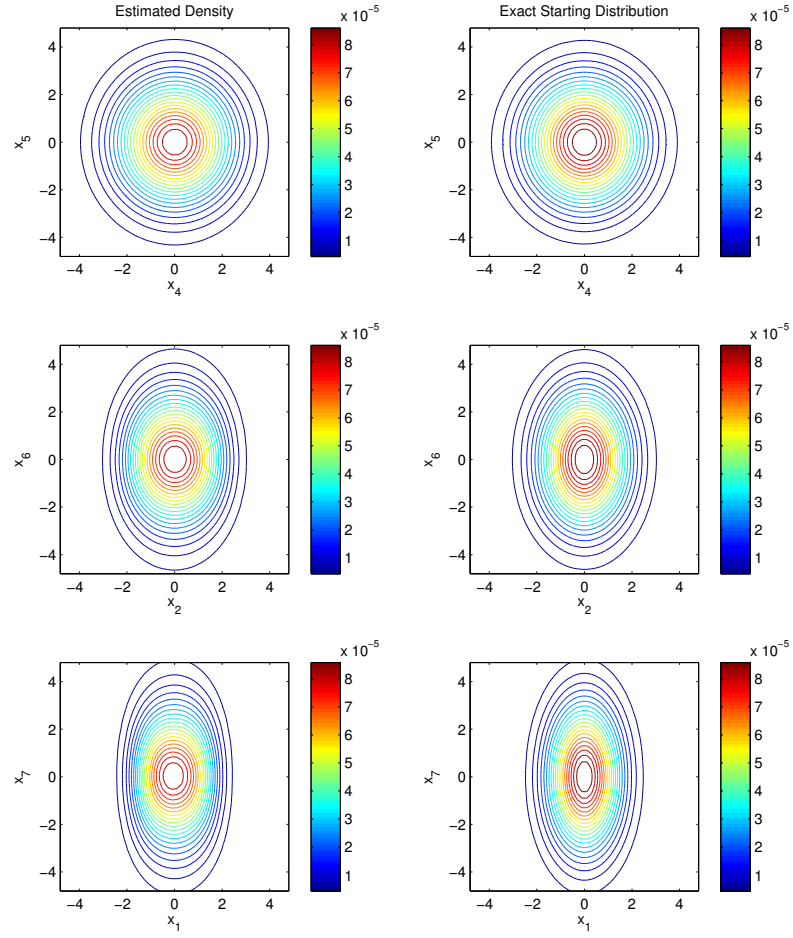


Figure 3.10: The algorithm is implemented using 10^4 points sampled from ρ and μ respectively.

the number of the dimensions and on the specific form of the starting and the target density. The linear dependence of the computational cost per iteration from the number of dimension is a necessary condition for the algorithm to be a good candidate when used in spaces with dimension higher than three.

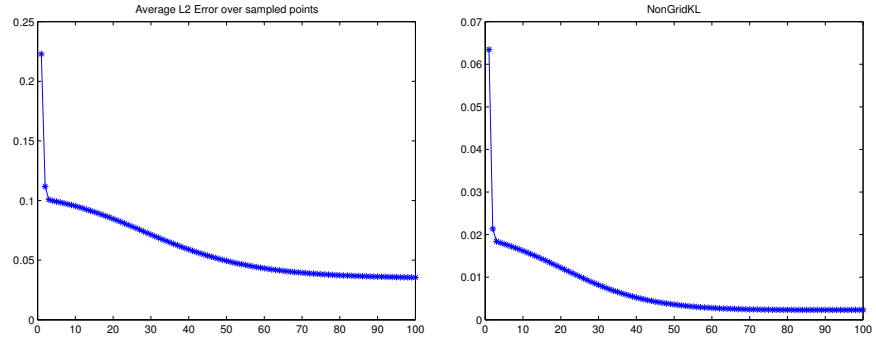


Figure 3.11: Convergence of Kullback-Leibler divergence and of the average error both computed on sampled points.

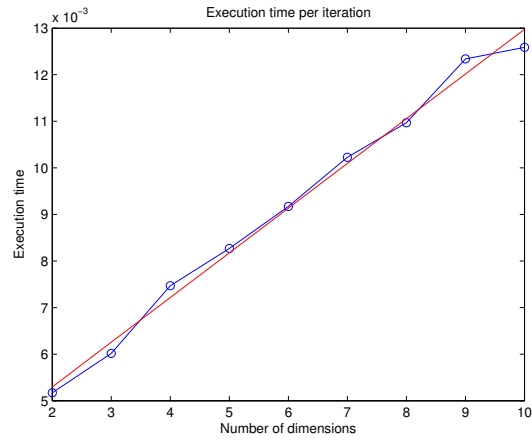


Figure 3.12: Average time per iteration, computed over 10^4 iteration when using 10^4 points.

Chapter 4

An event driven approach to binary integer linear programming

In this chapter we present an algorithm for the solution of binary integer linear programming (BILP) problems. These kind of problems originate from ordinary linear programming (LP) problems when we enforce on the entries of the minimizer x of the objective function the additional constraint $x_i \in \{0, 1\}$.

Even though BILP and optimal transport share common features from the mathematical point of view (both can be seen as particular instances of linear programming theory), this chapter is treated separately from the rest of the thesis.

The algorithm presented here is based on the branch and bound strategy that is commonly used to handle the binary integer constraint. However, the main difference with the existing approaches is that, in our case, we avoid solving a relaxed linear programming problem at each node of the binary search tree.

The algorithm presented here is instead based on an event driven kind of dynamic used to optimize the Lagrangian through which we reformulate the BILP problem. Nu-

merical tests are performed to compare our approach to the Matlab solver `bintprog`.

The chapter is organized as follows. We will first reformulate a LP problem using a Lagrangian formalism, where the Lagrange multipliers are the variables associated with the dual problem, and then propose an evolutionary game theoretical algorithm for optimizing the Lagrangian.

Finally, we will briefly review the branch and bound technique for BILP problems and illustrate how to use it in combination with the Lagrangian approach previously introduced in the context of pure linear programming.

4.1 Linear programming, inequality constraints

We start by considering the inequality-constrained, linear programming problem

$$\min_x c'x, \quad x \geq 0, \quad Ax \geq b \quad (4.1)$$

with dual

$$\max_y y'b, \quad y \geq 0, \quad y'A \leq c. \quad (4.2)$$

The two problems are linked by the Lagrangian

$$L = c'x + y'b - y'Ax, \quad (4.3)$$

in terms of which they adopt the form of a two-player, zero-sum game:

$$\min_{x \geq 0} \max_{y \geq 0} L \quad (4.4)$$

for the primal, and

$$\max_{y \geq 0} \min_{x \geq 0} L \quad (4.5)$$

for the dual. The minimax theorem states that, if at least one of the primal and dual

problems is feasible, we have zero duality gap:

$$\min_{x \geq 0} \max_{y \geq 0} L = \max_{y \geq 0} \min_{x \geq 0} L. \quad (4.6)$$

4.2 The algorithm

Starting with an initial guess $x \geq 0, y \geq 0$, the idea is to simultaneously move x so as to reduce the value of L and y so as to increase it. Thus, we might propose the system of ODEs

$$\dot{x} = (-\nabla_x L)^{x+} \quad (4.7)$$

$$\dot{y} = (\nabla_y L)^{y+}, \quad (4.8)$$

where we have introduced the notation

$$F^{x+} = \begin{cases} F & \text{for } x > 0 \\ F^+ = \max(F, 0) & \text{for } x = 0, \end{cases} \quad (4.9)$$

and similarly for F^{y+} . We do not need to define these for $x, y < 0$, since negative values of x and y will never occur while implementing the algorithm.

Yet such ODEs would not converge, but rather oscillate around the optimal (x^*, y^*) . To see this, consider the simplest example

$$\begin{aligned} \min_x x, & \quad x \geq 1 \\ \max_y y, & \quad 0 \leq y \leq 1, \end{aligned}$$

with

$$L = x + y - xy.$$

The system (4.7,4.8) yields

$$\begin{aligned}\frac{d(x-1)}{dt} &= (y-1)^{x+} \\ \frac{d(y-1)}{dt} &= (1-x)^{y+},\end{aligned}$$

resulting in circles around the optimal $x^* = 1, y^* = 1$ for initial values within the unit circle $(x-1)^2 + (y-1)^2 \leq 1$.

From a game-theoretical perspective, one could argue that this procedure failed because the two players did not take into consideration, while improving their strategies x and y , the fact that the strategy of the other player was also being improved. Thus the chosen $\dot{x}$ disregards the ‘ $\max_y$ ’ in (4.4), as the chosen $\dot{y}$ does with the ‘ $\min_x$ ’ in (4.5). An easy remedy to this is to replace (4.7, 4.8) with the forward looking

$$\dot{x} = \left(-\nabla_x L|_{(y+\alpha\dot{y})^+} \right)^{x+} \quad (4.10)$$

$$\dot{y} = \left(\nabla_y L|_{(x+\alpha\dot{x})^+} \right)^{y+}, \quad (4.11)$$

where $\alpha > 0$ is the look-ahead time.

It turns out that this is all that is required. Applying it to the simple example above, one obtains, when far enough from $x = 0, y = 0$ that the various ‘+’ superscripts can be dropped,

$$\begin{aligned}\frac{d(x-1)}{dt} &= (y-1) + \alpha \frac{d(y-1)}{dt} \\ \frac{d(y-1)}{dt} &= -(x-1) - \alpha \frac{d(x-1)}{dt}.\end{aligned}$$

In terms of $\xi = x - 1, \eta = y - 1$, we have

$$\begin{pmatrix} 1 & -\alpha \\ \alpha & 1 \end{pmatrix} \begin{pmatrix} \dot{\xi} \\ \dot{\eta} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} \xi \\ \eta \end{pmatrix},$$

which multiplied by

$$\begin{pmatrix} 1 & \alpha \\ -\alpha & 1 \end{pmatrix},$$

yields

$$\begin{pmatrix} 1 + \alpha^2 & 0 \\ 0 & 1 + \alpha^2 \end{pmatrix} \begin{pmatrix} \dot{\xi} \\ \dot{\eta} \end{pmatrix} = \begin{pmatrix} -\alpha & 1 \\ -1 & -\alpha \end{pmatrix} \begin{pmatrix} \xi \\ \eta \end{pmatrix}.$$

It follows that

$$\frac{d}{dt} (1 + \alpha^2) (\xi^2 + \eta^2) = -2\alpha (\xi^2 + \eta^2),$$

and the solution converges to $(\xi, \eta) = (0, 0)$ for any $\alpha > 0$, at a rate that is fastest when $\alpha = 1$.

4.3 Algorithm refinement and proof of convergence

For the specific Lagrangian (4.3) corresponding to the linear programming problem (4.1), the ODEs in (4.10, 4.11) defining the algorithm become

$$\dot{x} = (A'(y + \alpha \dot{y})^+ - c)^{x+} \quad (4.12)$$

$$\dot{y} = (-A(x + \alpha \dot{x})^+ + b)^{y+}. \quad (4.13)$$

It is convenient, both for the proof of convergence and for the practicality of implementation, to modify this algorithm, making it more explicit. We shall use, instead of (4.12, 4.13), the simpler version

$$\dot{x} = [(A'y - c) - \alpha_x A'(Ax - b)]^{x+} \quad (4.14)$$

$$\dot{y} = [-(Ax - b) - \alpha_y A(A'y - c)]^{y+}, \quad (4.15)$$

where α_x is a diagonal matrix with

$$\alpha_x(i, i) = \begin{cases} \alpha & \text{for } x_i > 0 \\ 0 & \text{otherwise,} \end{cases}$$

and similarly for α_y .

This modification amounts to replacing the $\dot{x}$ and $\dot{y}$ on the right-hand side of (4.12, 4.13) by their estimates with $\alpha = 0$, and eliminating the inner $+$ operators on the right-hand side. The elimination of the inner $+$ operators is done exclusively to simplify the proof. From a conceptual perspective, it makes more sense to leave the $+$, since each player can safely anticipate that the others variables will never become negative.

Due to the positivity constraints left on the right-hand side of equations (4.14, 4.15), the time evolution can be divided into time intervals during which fixed subsets of the x and y are nonzero. For each of these intervals, we need only consider the non-zero x 's and y 's. Let us denote by $\tilde{x}$ and $\tilde{y}$ these smaller-dimensional vectors with strictly positive components, and similarly by $\tilde{c}$ and $\tilde{b}$ the corresponding smaller-dimensional projection of the constant vectors c and b . Along the same lines, we will denote by $\tilde{A}$ the matrix A with only the corresponding rows and columns retained. Then the system becomes

$$\dot{\tilde{x}} = (\tilde{A}'\tilde{y} - \tilde{c}) - \alpha\tilde{A}'(\tilde{A}\tilde{x} - \tilde{b}) \tag{4.16}$$

$$\dot{\tilde{y}} = -(\tilde{A}\tilde{x} - \tilde{b}) - \alpha\tilde{A}(\tilde{A}'\tilde{y} - \tilde{c}), \tag{4.17}$$

Introducing the vectors

$$\xi = \tilde{A}\tilde{x} - \tilde{b}$$

and

$$\eta = \tilde{A}'\tilde{y} - \tilde{c},$$

we have

$$\begin{pmatrix} \dot{\xi} \\ \dot{\eta} \end{pmatrix} = \begin{pmatrix} -\alpha \tilde{A} \tilde{A}' & \tilde{A} \\ -\tilde{A}' & -\alpha \tilde{A}' \tilde{A} \end{pmatrix} \begin{pmatrix} \xi \\ \eta \end{pmatrix}. \quad (4.18)$$

It follows that

$$\frac{d}{dt} (\|\xi\|^2 + \|\eta\|^2) = -2\alpha (\|\tilde{A}' \xi\|^2 + \|\tilde{A} \eta\|^2) \leq 0. \quad (4.19)$$

Even though the definitions of ξ and η change over time, since they depend on $\tilde{x}$, $\tilde{y}$, $\tilde{A}$, $\tilde{b}$ and $\tilde{c}$, the condition above stating that the sum of their squared norms never increases, is global. To see this, consider first a scenario in which a currently positive component x_i of x reaches $x_i = 0$, and hence the corresponding column of $\tilde{A}$ is eliminated. This does not change the value of ξ , since in its definition that column multiplies x_i , which reaches zero at the time of transition. By contrast, η is changed instantaneously at the transition: the component corresponding to the x_i variable is suddenly eliminated. Because eliminating a component can only decrease the norm of η , the monotonicity of the time evolution of $(\|\xi\|^2 + \|\eta\|^2)$ still holds.

Consider now the reciprocal scenario in which an x_i currently zero develops a positive derivative, and hence $\tilde{A}$ gains a new column. Again, since x_i is zero at the transition time, this does not change the value of ξ . Yet this time the dimensionality of η grows by one, adding a new component with value $A^{i'} y - c_i$. The value of the new component, though, is zero at transition since it corresponds to the time derivative of x_i , which is switching from negative to positive. It was to simplify this element of the proof that α was replaced by α_x in (4.14): we need α_x to be zero when $x = 0$, so that the norm of η does not jump to a bigger value.

Mutatis mutandis, the same argument applies when it is a component y_j of y that transitions in or out of $y_j = 0$.

We conclude from the above that the value of $(\|\xi\|^2 + \|\eta\|^2)$ will continue to decrease until, at some point, $\tilde{A}'(\tilde{A}\tilde{x} - \tilde{b}) = \tilde{A}(\tilde{A}'\tilde{y} - \tilde{c}) = 0$. At this point, $\dot{\xi} = \dot{\eta} = 0$, so the values of $\dot{x}$ and $\dot{y}$ do not depend on time, and one of the following holds:

1. $\tilde{A}\tilde{x} - \tilde{b} = \tilde{A}'\tilde{y} - \tilde{c} = 0$. This represents convergence to an optimal solution to the problem. Notice that $Ax \geq b$ and $A'y \leq c$, since otherwise components of x and y currently set at zero would have positive time derivatives.
2. At least one component of $\tilde{A}\tilde{x} - \tilde{b}$ is positive. Then the corresponding component of y has a negative -and constant- time derivative, so in a finite time it will reach a zero value and be excluded from the set.
3. Similarly for at least one component of $\tilde{A}'\tilde{y} - \tilde{c}$ being negative, and the corresponding component of x decreasing to zero.
4. None of the above, and at least one component of $\tilde{A}\tilde{x} - \tilde{b}$ is negative, so the corresponding component of y will grow linearly -and unboundedly- with time. In this case, the primal problem is unfeasible.
5. None of the above, implying that at least one component of $\tilde{A}'\tilde{y} - \tilde{c}$ is positive. In this case, the corresponding component of x will grow linearly with time, the dual is unfeasible and the primal objective function is unbounded.

The justification of the statements for each of the cases above goes as follows:

In the case numbered 1, we have found a feasible solution x to the primal and a feasible solution y to the dual problem. Moreover, all y 's corresponding to strict inequalities in the primal solution problem are zero. Hence the two solutions are optimal, and $c'x = b'y$. In fact, it follows directly from the identities $\tilde{A}\tilde{x} - \tilde{b} = \tilde{A}'\tilde{y} - \tilde{c} = 0$ that $b'y = y'Ax = c'x$.

Cases 2 and 3 are self-explanatory.

To justify the statement of unfeasibility in case 4, we need the following solvability condition due to Farkas (see appendix):

A necessary and sufficient condition for the system of inequalities $Ax \geq b$ to have a solution $x \geq 0$ is that, for all dual vectors $z \geq 0$, $A'z \leq 0 \rightarrow b'z \leq 0$.

Hence, to prove unfeasibility of the primal problem, it is enough to display a $z \geq 0$

such that $A'z \leq 0$ and $b'z > 0$. A z with this property is given by

$$z = (b - Ax)^+.$$

Since this is the rate at which y is changing in case 4 without affecting the condition $\tilde{A}(\tilde{A}'\tilde{y} - \tilde{c}) = 0$, it follows that $\tilde{A}\tilde{A}'\tilde{z} = 0$, so $\tilde{A}'\tilde{z} = 0$, and therefore $A'z = 0$. On the other hand, $0 < \|z\|^2 = z'(b - Ax)^+ = z'(b - Ax) = z'b$, proving the statement.

Mutatis mutandis, the argument above shows that, in case 5, the dual problem is unfeasible, so, if the primal is feasible, its objective function is necessarily unbounded. Since by assumption (the “none of the above” opening clause) $Ax - b \geq 0$, the primal is indeed feasible, and the conclusion follows.

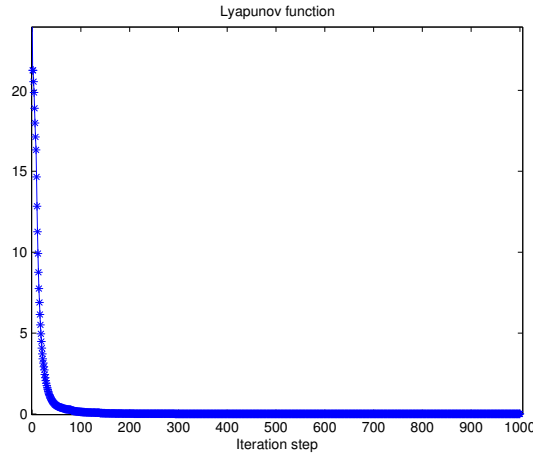


Figure 4.1: Convergence of the quantity $\|\xi\|^2 + \|\eta\|^2$ as indicated in equation (4.19).

4.4 Extension: equality constraints

The extension of the procedure above to the more general linear programming problem with inequality and equality constraints

$$\min_x c'x, \quad x \geq 0, \quad A_{in}x \geq b_{in}, \quad A_{eq}x = b_{eq} \quad (4.20)$$

is completely straightforward: we only need to relax the condition that the y be positive for the equality constraints. Then the system (4.14,4.15) becomes

$$\dot{x} = [(A'y - c) - \alpha_x A'(Ax - b)]^{x+} \quad (4.21)$$

$$\dot{y}_{in} = [-(A_{in}x - b_{in}) - \alpha_y A_{in}(A'y - c)]^{y+} \quad (4.22)$$

$$\dot{y}_{eq} = -(A_{eq}x - b_{eq}) - \alpha A_{eq}(A'y - c), \quad (4.23)$$

$$(4.24)$$

where the vector y represents the concatenation of y_{in} and y'_{eq} . A similar extension applies when not all components of x are required to be positive. Mutatis mutandis, the proof of convergence remains the same. In the unfeasible case, y_{eq} can blow either in the positive or the negative direction.

4.5 Binary integer linear programming, an event driven approach

The idea developed so far can be adapted to solve binary integer linear programming (BILP) problems:

$$\min_x c'x, \quad x \in \{0, 1\}, \quad Ax \geq b \quad (4.25)$$

where now we consider the additional constraints $x \in \{0, 1\}$.

Depending on the structure of A , b and c there are specific algorithms that can be used to solve problems like (4.25). An example is the Hungarian method used when the problem (4.25) reduces to pure assignment problem.

In the general case, where no particular assumptions on A , b and c are made, a common used approach is the branch and bound family of algorithms.

In this context, the BILP problem is initially solved without imposing the binary integer constraint. Such constraint is then enforced in a progressive way on each variable

x_i proceeding according to a binary tree. A given node of the tree represents the enforce-

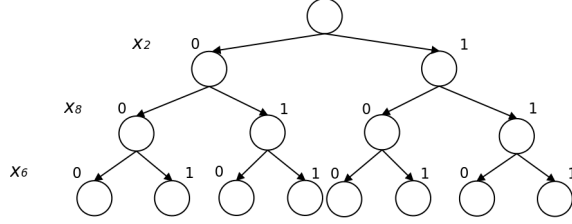


Figure 4.2: Binary tree

ment of the binary constraint on a subset of the entries of x . Two (children) nodes are added to a given node if the value of the objective function of the LP relaxation problem at that node is lower than the best integer solution (if available) found so far. In figure (4.2) we illustrate a binary search tree where, starting from the root, the entries 2, 6 and 8 of the variable x are progressively enforced to be either zero or one. For instance, the fifth node from the left, on the last row of the tree, corresponds to a scenario in which all the entries of x are not constrained except the entries x_2 , x_6 and x_8 that are constrained to be equal to 1, 0 and 0 respectively.

In principle, there are 2^n nodes in the tree but some branches of the tree are eliminated from the search, either because of the unfeasibility of the solution of the relaxation problem, or because the optimal value of the relaxation problem is greater than the value of the best integer solution found so far.

In general, the number of nodes does not reduce in a relevant way and, because an LP problem needs to be solved at each node of the tree (using either simplex or interior point method), the bound and branch strategy may be too slow.

In this section, we propose a new branch and bound kind of algorithm where we avoid to solve a LP relaxation problem at each node of the tree.

The starting point is still the Lagrangian $L = c'x + y'(b - Ax)$ relative to the optimization problem

$$\min_{x \in \{0,1\}} \max_{y \geq 0} L. \quad (4.26)$$

In a similar way to what was done in the previous sections, the idea is to find the solution of the min-max problem evolving x and y so to minimize and maximize L respectively. The main difference with the pure linear programming problem, though, is that in (4.26) the values to which x is constrained form a non-convex set. In this case, it is possible to show that the minimax theorem doesn't hold and that in principle we can have $\nabla_x L = \nabla_y L = 0$ even with a feasible binary integer solution that does not minimize $c'x$. The way we will handle the non-convexity of the set of values for the x variable, is through the bound and branch strategy as will be explained now.

We start by considering how the dynamics in (4.14) and (4.15)

$$\begin{aligned}\dot{x} &= [A'(y + \alpha_x \dot{y}) - c]^{x+} \\ \dot{y} &= [-A(x + \alpha_y \dot{x}) + b]^{y+}\end{aligned}$$

should be modified in order to enforce the binary integer constraint on x . The idea is to define the evolution of x via a jump process in which the entries x_i can only change from 0 to 1 and viceversa without assuming intermediate, non integer, values. In this case, considering the quantity $\dot{x}_i$ does not have any more sense. The derivative of x would be always zero except when x_i jumps. At the time in which x_i jumps, in theory, we would have $\dot{x}_i$ equal to infinity.

It appears more natural in this context to consider $\dot{x} = 0$ and move the x variable according to a step function defined by the sign of $-\nabla_x L = A'y - c$. This amount to set $\dot{x} = 0$ in the equation (4.15) defining $\dot{y}$ and define directly the value of each entry of x using a step function as defined below.

For the sake of simplicity, we will consider the time discrete case of (4.14) and (4.15) obtained using a forward Euler scheme:

$$x(t+h) = x(t) + h [A'(y(t) + \alpha_x \dot{y}(t)) - c]^{x+} \quad (4.27)$$

$$y(t+h) = y(t) + h [-A(x(t) + \alpha_y \dot{x}(t)) + b]^{y+} \quad (4.28)$$

where $h > 0$ is the time step.

Given the function

$$\theta(u, v) = \begin{cases} 1 & \text{for } u > 0 \\ 0 & \text{for } u < 0 \\ v & \text{for } u = 0 \end{cases} \quad (4.29)$$

we modify (4.27) and (4.28) as follows

$$x(t+h) = \theta(A'y(t+h) - c, x(t)) \quad (4.30)$$

$$y(t+h) = y(t) + h(-Ax(t) + b)^{x^+}. \quad (4.31)$$

where in (4.30) we don't need to consider anymore the look-ahead time α , since we can directly use the value of $y(t+h)$ from (4.31).

Starting with the initial condition $x_i(t=0) \in \{0, 1\}$, the variable x_i is forced to assume only binary integer values, switching in between 0 and 1 according to the sign of $-\nabla_x L = A'y - c$.

If at time step t , we have that $(A'y(t) - c)_i(t) > 0$ then x_i will either switch from 0 to 1 or maintain its value at 1 depending on whether at the previous time step x_i was equal to 0 or 1 respectively. Similarly in the case $(A'y(t) - c)_i(t) < 0$, x_i will either switch from 1 to 0 or keep its value at 0 based on its value at the previous time step.

The only situation in which a decision has to be taken is when $(A'y(t) - c)_i$ switches sign at the same time in which x_i changes its value. This is the scenario in which, if x_i was not constrained to be 0 or 1, x_i would assume an intermediate value in between 0 and 1.

To handle this situation, in which it is not clear a priori whether we should set x_i equal to 0 or to 1, the algorithm *branches* creating two nodes in the binary search tree, simulating the two scenarios. At this point x_i is not anymore a *free variable* in the sense that it is not anymore free to oscillate in between the values of 0 or 1, it rather becomes

a *fixed variable* whose value depends on the node of the binary tree we are exploring.

Because we are always exploring the two possible values of each fix variable and we are evolving the free variables according to $-\nabla_x L$ and $\nabla_y L$ respectively, we are sure that the algorithm converges to the saddle point of L defined by eq. (4.26).

Finally a consideration has to be made in order to handle BILP problems that do not have a feasible solution. Un feasible solutions are detected as in the usual linear programming case, by establishing an upper value for the Lagrangian, or on the Lagrange multipliers y , above which the problem is considered unfeasible. In particular, if no feasible solution exists, one of the entries of the vector $b - Ax$ has to be positive, resulting in a diverging value of the Lagrangian due to the max over all the positive values of y :

$$\min_{x \in \{0,1\}} \max_{y \geq 0} L.$$

4.5.1 Event driven dynamics and implementation

The dynamics described in (4.30) and (4.31) is an example of *event driven* dynamics: the time evolution is divided into temporal windows in which x is constant in time while y evolves with constant velocity according to (4.31).

Given the value of $x(t)$ and $y(t)$ at a given time t , each temporal window is defined by the smallest of the following three times:

- $T_1 = \min_i \{t_i^{up}\}$ where t_i^{up} is the time required for the quantity $(A'y(t) - c)_i \leq 0$ to become strictly positive
- $T_2 = \min_i \{t_i^{down}\}$ where t_i^{down} is the time required for the quantity $(A'y(t) - c)_i \geq 0$ to become strictly negative
- $T_3 = \min_i \{t_i^0\}$ where t_i^0 is the time required for a variable $y_i \neq 0$ with $\dot{y}_i > 0$ to reach 0

This means that y is evolved according to (4.31) with constant x until one of the

following three events happens:

1. One of the entries, x_i , of x wants to switch from 0 to 1 because the variable $(A'y - c)_i$, initially non positive, becomes strictly positive
2. One of the entries, x_i , of x wants to switch from 1 to 0 because the variable $(A'y - c)_i$, initially non negative, becomes strictly negative
3. One of the entries, y_i , of y , initially different from zero, reaches zero.

It is important to notice that the T_i 's can be computed analytically. When we consider x constant in time, y is a linear function of time implying that

$$t_i^0 = -\frac{y(t)_i}{(b - Ax(t))_i}. \quad (4.32)$$

Because y is linear in time, $A'y(t) - c$ is also a linear function of time therefore leading to

$$t_i^{up,down} = -\frac{(A'y(t) - c)_i}{(A'\dot{y}(t))_i}. \quad (4.33)$$

This particular structure allows a fast integration of the dynamics in (4.30) and (4.31). In the time window in which x is constant the right hand side of (4.13) is constant and equation (4.31), choosing the time step h of the length of the temporal window, gives therefore an exact integration of (4.13).

In the panel (1), we give the pseudocode of the event driven algorithm described above. In particular, the pseudocode is relative to the “Depth First Search” strategy (see below) that is possible to adopt to explore the binary tree.

The variable “leaf=true” at line 3 indicates that the algorithm reached a leaf of the binary search tree. A leaf of the tree is reached either when all the variables $\{x_i\}$ are constrained or when the value of the objective function, in our case the Lagrangian appearing in (4.26), is bigger than the value corresponding to the best feasible solution so far. The steady state of the dynamics in (4.30) is reached when none of three events

described above is going to happen. This condition is verified by the **if** statement at line 8 and corresponds to being in a local minimum of the objective function with respect to the variables that are free at that node of the search tree. In order to look for the global minimum it is necessary to explore different branches of the search tree by analyzing the list of unexplored nodes (line 32).

The nodes to be explored first are specific to the way the branch and bound algorithm is implemented. The two main ways of exploring the search tree are called Depth First Search (DFS) and Best Node Search (BNS) respectively. In the DFS case we explore the tree as far as possible along each branch before backtracking in order to look for other branches to explore. In the BNS kind of search, instead, we explore proceeding along the branches characterized by the node corresponding to the lowest value of the objective function.

The **if** statements at lines 17 and 23 check whether it is necessary or not to add new nodes to the search tree.

4.5.2 Numerical results

In this section, we compare the performances of the event driven algorithm with the ordinary branch and bound algorithm using the simplex method to solve the relaxation problem at each node of the binary tree. More specifically, we coded our algorithm in Matlab and compared the performances against the Matlab built-in function called **bintprog**. The execution time were computed using the **tic-toc** Matlab built-in functions. In both approaches the binary tree is built using a “best node” strategy.

In figures (4.3) we report results relative to 10^4 simulations obtained using random numbers from a uniform distribution on $[0, 1]$ for the entries of the 15×17 (15 variables and 17 inequalities) matrix A and the entries of the vector c . The upper panel represents the convergence of the average execution time while the lower panel those one of the average number of nodes over the problems solved. In order to take into account the

possible different compilation time of the function we were comparing, we dropped the first test measurement.

As we can see, even though in average the event driven algorithm needs to explore roughly three times the number of nodes `bintprog` needs to explore, the execution time averaged over the 10^4 sample problems is smaller.

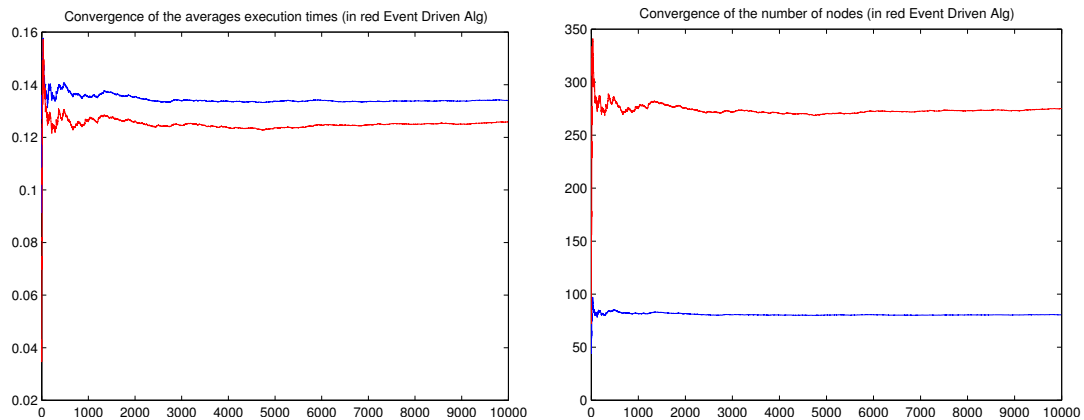


Figure 4.3: Data relative to the solution of 10^4 feasible BILP problems with random data. Each problem is characterized by 17 variables and 15 inequalities. On the x axis is represented the number of problems solved. The corresponding value on the y axis represents the average of the execution time (left panel) or the number of nodes (right panel) computed over the corresponding number of problems solved.

```

Data:  $x_0, y_0, A, b, c$ 
 $x = x_0; y = y_0; \text{optimal} = \text{false};$ 
while ( $\text{done} == \text{false}$ ) do
    leaf = false;
    while ( $\text{leaf} == \text{false}$ ) do
        Use  $x, y$  to compute;
         $[T_1, i_1] = \min_i \{t_i^{up}\} \quad [T_2, i_2] = \min_i \{t_i^{down}\} \quad [T_3, i_3] = \min_i \{t_i^0\};$ 
         $dt = \min\{T_1, T_2, T_3\};$ 
        if ( $\text{isempty}(dt)$ ) then /* steady state */
            leaf = true;
            If  $x$  is feasible compare it with best feasible solution so far and in case
            update;
        else
             $y = y + dt(b - Ax)^{x+};$ 
            Update value of the objective function;
            if ( $dt = T_1$ ) then /*  $x(i_1)$  goes from 0 to 1 */
                 $x(i_1) = 1;$ 
                Update the value of  $D_{i_1} = (A'y - c)_{i_1};$ 
                if ( $D_{i_1} < 0$ ) then
                    | branch: create a new node by fixing  $x(i_1)$  equal to 1 or 0;
                end
            else /*  $x(i_2)$  goes from 1 to 0 */
                 $x(i_2) = 0;$ 
                Update the value of  $D_{i_2} = (A'y - c)_{i_2};$ 
                if ( $D_{i_2} > 0$ ) then
                    | branch: create a new node by fixing  $x(i_2)$  equal to 1 or 0;
                end
            end
        end
        if (all the variables  $x_i$  are constrained or value of the obj. function is bigger than the best feasible value so far or max number of nodes has been reached) then
            | leaf = true
        end
    end
    Explore a new node in the list of the nodes to explore by getting  $x$  and  $y$  relative to the new node;
    if (All nodes in the node list have been explored) then
        | done = true
    end
end

```

Algorithm 1: Event driven pseudocode when using “Depth First Search” algorithm.

Chapter 5

Conclusion

In this thesis we analyzed, from the numerical point of view, two general optimization problems: optimal transport and binary integer linear programming. Even though the two problems can be viewed as particular instances of linear programming theory, we treated them separately, proposing algorithms based on different principles that, to the best of our knowledge, are new in this area of research.

Regarding optimal transport, what characterized our approach was that, even though we focused on finding numerical solutions to the continuous aspect of this problem, we used discrete data sets drawn from the starting ρ and the target μ probability density respectively. The use of these samples allowed the algorithms to be mesh-free, an essential feature to handle cases in which ρ and μ are defined in a space of dimension bigger than three.

In particular, in Chapters (2) and (3), we described two algorithms, based on two different strategies, for finding the optimal map.

In Chapter (2), the map was found by a linear combination of gradient of convex, elementary potentials, by minimizing the Kullback-Leibler divergence. In particular, in order to make sure that the linear combination still represented the gradient of a convex function, we adopted a rejection strategy over the coefficients of the linear combination.

Under the hypothesis that local elementary maps can always move the sampled points from ρ in order to adapt them according to μ , this assures by proposition (1.1), that the map we found is optimal.

In Chapter (3), by using the dual formulation of the Kantorovich problem, we defined a variational problem that allowed to map the starting density ρ into μ by means of the gradient flow of a given functional. In this case, the map from ρ to μ was not any more obtained as a linear combination of elementary maps, but rather as composition of them. If on the one hand this allowed for a more general and less computational expensive algorithm than that one proposed in Chapter (2), on the other hand, the map obtained by using the gradient flow is not in general optimal.

These algorithms represented the two philosophies we had to face numerous times while studying optimal transport. On the one hand, we can look for the optimal map by “carefully” adding gradients of convex potentials providing we are choosing them among a rich enough family of functions. This approach ensures the obtained map to be optimal, but is computationally demanding since a strategy to keep the convexity of the linear combination is needed. On the other hand we can find the map from ρ to μ by “forgetting the past” and following the gradient flow of a functional that is defined at each iteration step based on the position of the mapped points at that step. This approach is computationally easier to implement, but has the drawback that the overall map is built up by composing elementary maps. Because the composition of the gradients of convex functions in general does not result in a gradient of a convex function, the use of the projection strategy explained at the end of Chapter (3) was needed.

What was quite surprising was that the approach described in Chapter (3), when used without the projection step, still provided a good approximation of the optimal map. A good topic for a future research project would be to further analyze this result. The aim would be to avoid the use of any projection strategy at all.

Motivated by a collaboration with the Department of Education of New York City,

in Chapter (4) we developed an algorithm for binary integer linear programming based on the branch and bound strategy. The main difference from conventional algorithms for solving this problem is that, in our case, we avoided solving a relaxed linear programming problem at each node of the binary search tree as used in the branch and bound strategy. This was possible by using a Lagrangian reformulation of the binary integer linear programming problem together with an efficient event-driven dynamics to optimize the Lagrangian.

Simple tests were performed to assert the efficiency of our approach. They provided results that were competitive with the Matlab solver `bintprog`.

A long-term research project in this context would be to achieve a better understanding of the jump dynamic used to optimize the Lagrangian. The specific aim would be to reduce the number of nodes that need to be explored. In addition it appears straightforward to generalize the event-driven dynamics in order to handle more general combinatorial problems than binary integer linear programming.

Appendix A

Linear Programming

In this section we will review the fundamentals of the theory of duality in linear programming. We will start showing how the theorem of the alternative for linear systems is generalized when inequality constraints are added. This will be useful to prove the duality principle. We will proof only the main theorems referring to the literature when a proof is not provided.

The material presented here is a summary of what contained in chapter 12 and 13 of [21] and chapter 8 of [26]. Every vector is intended to be a column vector, while row vectors are marked with an apex. Apices are also used to indicate the transpose of a matrix.

A.1 Theorem of the alternative

Let $A \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$ and $b \in \mathbb{R}^m$. The solvability of the linear system $Ax = b$ is characterized by the following theorem

Theorem A.1. (*Fredholm*). *Either the linear system $Ax = b$ has a solution or exists v such that $v'A = 0$ and $v'b \neq 0$.*

In words either b is in the column space of A or otherwise b has a nonzero component

in the null space of A' .

The theorem (A.1) can be generalized in the case we are requiring the solution of $Ax = b$ to be (component-wise) non negative

Theorem A.2. (*Farkas*). *Either the problem*

$$\begin{cases} Ax = b \\ x \geq 0 \end{cases} \quad (\text{A.1})$$

has a solution or there exists a v such that $v'A \geq 0$ and $v'b < 0$.

The details of the proof can be found in [21], here we will just give the main idea of why the theorem holds.

Let K be the set of vectors obtained by a linear combination, with non negative coefficients, of the columns of A . This set can be represented as an open-ended cone (with its vertex in the origin) and it can be proved that K is convex and closed. These two condition on the set K , are sufficient for the following theorem to hold:

Theorem A.3. (*Hyperplane separation*). *Let S be a closed, convex set and y a point not in S . Then there is a closed half-space that contains S but not y .*

Because of theorem (A.3) either b is in K , namely system (A.1) has a solution, or otherwise it is possible to find an hyperplane that goes through the origin and that has the vector b on one side and the set K on the other. Let v be the vector perpendicular to such plane. Then the angle in between v and any vector in K will be less or equal to $\pi/2$, namely $v'A \geq 0$ while the angle in between v and b is going to be bigger than $\pi/2$ resulting in $v'b < 0$.

Theorem (A.2) can be easily generalized to the following case

Theorem A.4. *Either the problem*

$$\begin{cases} Ax \leq b \\ x \geq 0 \end{cases} \quad (\text{A.2})$$

has a solution or there exists a vector $v \geq 0$ such that $v'A \geq 0$ and $v'b < 0$.

Proof: It is sufficient to notice that the problem (A.4) can be recasted in the form of (A.2) by using a *slack variable* z . By defining

$$z = b - Ax, \quad \tilde{x} = \begin{bmatrix} z \\ x \end{bmatrix} \quad \text{and} \quad \tilde{A} = [\ I \ A \] \quad (\text{A.3})$$

we can rewrite (A.1) as

$$\begin{cases} \tilde{A}\tilde{x} = b \\ \tilde{x} \geq 0 \end{cases} \quad (\text{A.4})$$

The equations in (A.4) have the same form of those one in (A.1) so we can use the theorem (A.2) to assert that either (A.4) has a solution or there exists v s.t. $v'\tilde{A} \geq 0$ and $v'b < 0$. This is equivalent to the statement of the theorem. $\square$

A.2 Duality

In this section we will use theorem (A.4) to prove the principle of duality and then the minimax theorem. The principle of duality concerns with the two following optimization problems:

$$\begin{cases} s = \inf_x c'x \\ Ax \geq b \\ x \geq 0 \end{cases} \quad \text{and} \quad \begin{cases} S = \sup_y y'b \\ y'A \leq c \\ y \geq 0 \end{cases} \quad (\text{A.5})$$

where $A \in \mathbb{R}^{m \times n}$, x and $c \in \mathbb{R}^n$ and $y, b \in \mathbb{R}^m$. We will call the vector x feasible if the constraints $Ax \geq b$ and $x \geq 0$ are satisfied. Similarly y is defined to be feasible if $y'A \leq c$ and $y \geq 0$ are satisfied.

Theorem A.5. (*Duality*). *Suppose there exist feasible vectors x and y . Then S and s are finite and $S = s$.*

Proof: Since x must be a vector with non negative components then by multiplying both sides the first inequality constraint for the sup problem we get

$$c'x \geq y'Ax \geq y'b \quad (\text{A.6})$$

or equivalently $s \geq S$. To prove the equality in between s and S it is sufficient to show the existence of a feasible y for the sup problem in (A.5) such that

$$y'b \geq s. \quad (\text{A.7})$$

In order to consider a feasible y for the sup problem that at the same time satisfies eq. (A.7) we will combine the inequalities $A'y \leq c$ (equivalent to $y'A \leq c$) and $y \geq 0$ with (A.7) as follow

$$\begin{bmatrix} A' \\ -b \end{bmatrix} y \leq \begin{bmatrix} c \\ -s \end{bmatrix}, \quad y \geq 0 \quad (\text{A.8})$$

Let's show the existence of such y by contradiction. Let's suppose that there is a feasible y for the sup problem that does not satisfies (A.7). Then such y does not satisfies (A.8) and according to theorem (A.4) there exist a vector v and a scalar a such that

$$[v' \ a] \geq 0, \quad [v' \ a] \begin{bmatrix} A' \\ -b \end{bmatrix} \geq 0, \quad [v' \ a] \begin{bmatrix} c \\ -s \end{bmatrix} < 0. \quad (\text{A.9})$$

Clearly if $a = 0$ the system (A.9) reduces to

$$v \geq 0, \quad v'A' \geq 0, \quad v'b < 0 \quad (\text{A.10})$$

meaning that y (again in virtue of theorem (A.4)) is not feasible contrary to our assumption. This means that a has to be positive rather than non negative. Without loss of generality let $a = 1$. With this choice eq. (A.9) leads to the contradiction

$$v \geq 0, \quad v'A \geq b, \quad v'b < s. \quad (\text{A.11})$$

saying that exists a feasible vector v for the infimum problem satisfying $v'b < \inf_x c'x$. This contradiction is consequence of the fact that we supposed the existence of a feasible y not satisfying (A.7). Summarizing, we can find a feasible y satisfying (A.7) and $S \geq s$. This fact, together with (A.6), proves that $S = s$. $\square$

Remark 3. The minimization and maximization problem in (A.6) are often called *primal* and *dual* problem respectively.

Remark 4. Clearly s is finite since the entries of b , A and c are finite. The inequality (A.7) is indeed an equality since otherwise would contradict the fact that $S = s$. This says that the supremum in (A.6) is achieved, namely that the supremum is indeed a maximum. Analogously one can show that the infimum is indeed a minimum by turning the supremum into an infimum (this can be realized replacing A , c and b with $-A$, $-c$ and $-b$).

Remark 5. By using theorem (A.2) rather than its generalization (A.4) we can, repeating the same proof for the theorem (A.5), show that given the primal and dual linear

programming problems

$$\begin{cases} s = \min_x c'x \\ Ax = b \\ x \geq 0 \end{cases} \quad \text{and} \quad \begin{cases} S = \max_y y'b \\ c \geq y'A \end{cases} \quad (\text{A.12})$$

we have that $S = s$

We close this section with a set of relation that must be satisfied in correspondence of the optimal solution and that goes under the name of *complementary slackness conditions*.

Theorem A.6. (*Equilibrium theorem*). *The feasible vectors x and y are optimal if and only if the following relations are satisfied at the same time*

- If $(Ax)_i > b_i$ then $y_i = 0$
- If $(y'A)_j < c_j$ then $x_j = 0$

The proof of the theorem relies on the fact that at the optimal solution the primal and the dual problem give the same result. In particular starting from the true relation

$$yb \leq y(Ax) = (yA)x \leq cx \quad (\text{A.13})$$

we should investigate under which conditions we have that $yb = cx$. This equality is true if and only if the solution is optimal. There is only one way in which $yb = y(Ax)$ holds: every time we have a feasible solution satisfying strictly the inequality constraint $b_i < (Ax)_i$ the factor y_i multiplying the i -th component must be zero. At the same time the only way of having $(yA)x = cx$ is that whenever we have $(yA)_j < c_j$ then the j -th component of x must be zero.

A.3 A particular case of LP: discrete optimal transport

In the following we will describe the discrete version of the Monge-Kantorovich problem in its primal and dual form. Let $\rho \in \mathbb{R}^m$, $\mu \in \mathbb{R}^n$ and $c \in \mathbb{R}^{m \times n}$ be characterized by nonnegative entries and by $\sum_i \rho_i = \sum_j \mu_j$.

The discrete version of the Monge Kantorovich problem is

$$\left\{ \begin{array}{l} \min_{\pi} \sum_{i,j} c_{ij} \pi_{ij} \\ \sum_j \pi_{ij} = \rho_i \\ \sum_i \pi_{ij} = \mu_j \\ \pi_{ij} \geq 0 \end{array} \right. \quad (\text{A.14})$$

where ρ and μ represents respectively the starting and the target mass densities and c is the cost matrix.

The problem in (A.14) is equivalent to the primal linear programming problem in (A.12) once we make a correspondence in between the following quantities:

•

$$B = \begin{pmatrix} I & 0 & \cdots & 0 \\ 0 & I & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & I \end{pmatrix}, \quad C = \begin{pmatrix} e^1 & e^1 & \cdots & e^1 \\ e^2 & e^2 & \cdots & e^2 \\ \vdots & \vdots & \ddots & \vdots \\ e^m & e^m & \cdots & e^m \end{pmatrix}, \quad A = \begin{bmatrix} B \\ C \end{bmatrix} \quad (\text{A.15})$$

where B is a $(n \times nm)$ matrix with $I = (1, \dots, 1)$ and C is a $(m \times nm)$ matrix with $e_i^k = \delta_{i,k}$

•

$$\begin{aligned} x &= (\pi_{11}, \pi_{12}, \dots, \pi_{1m}, \pi_{21}, \dots, \pi_{nm})' \\ c &= (c_{11}, c_{12}, \dots, c_{1m}, c_{21}, \dots, c_{nm}) \\ b &= (\rho_1, \dots, \rho_m, \mu_1, \dots, \mu_m) \end{aligned} \quad (\text{A.16})$$

By defining

$$y = (u_1, \dots, u_n, v_1, \dots, v_m) \quad (\text{A.17})$$

and using (A.15) and (A.15), we can formulate the dual of the problem (A.14) as

$$\begin{cases} \max_{u,v} \sum_i u_i \rho_i + \sum_j v_j \mu_j \\ u_i + v_j \leq c_{ij} \text{ for } i = 1, \dots, n; j = 1, \dots, m. \end{cases} \quad (\text{A.18})$$

Bibliography

- [1] JP Agnelli, M Cadeiras, EG Tabak, CV Turner, and E Vanden-Eijnden. Clustering and classification through normalizing flows in feature space. *Multiscale Modeling & Simulation*, 8(5):1784–1802, 2010.
- [2] Sigurd Angenent, Steven Haker, and Allen Tannenbaum. Minimizing flows for the Monge-Kantorovich Problem. *SIAM journal on mathematical analysis*, 35(1):61–97, 2003.
- [3] Jean-David Benamou and Yann Brenier. A computational fluid mechanics solution to the Monge-Kantorovich mass transfer problem. *Numerische Mathematik*, 84(3):375–393, 2000.
- [4] Jean-Daniel Boissonnat, Olivier Devillers, and Samuel Hornus. Incremental construction of the Delaunay triangulation and the delaunay graph in medium dimension. In *Proceedings of the 25th annual symposium on Computational geometry*, pages 208–216. ACM, 2009.
- [5] Yann Brenier. Polar factorization and monotone rearrangement of vector-valued functions. *Communications on pure and applied mathematics*, 44(4):375–417, 1991.
- [6] Yann Brenier. Optimal transport, convection, magnetic relaxation and generalized Boussinesq equations. *Journal of nonlinear science*, 19(5):547–570, 2009.
- [7] Luis A Caffarelli. The Monge-Ampère equation and optimal transportation, an

- elementary review. In *Optimal transportation and applications*, pages 1–10. Springer, 2003.
- [8] Guillaume Carlier. Duality and existence for a class of mass transportation problems and economic applications. In *Advances in mathematical economics*, pages 1–21. Springer, 2003.
- [9] Rick Chartrand, K Vixie, B Wohlberg, and Erik Bollt. A gradient descent solution to the Monge-Kantorovich problem. *Applied Mathematical Sciences*, 3(22):1071–1080, 2009.
- [10] DC Dowson and BV Landau. The Frechet distance between multivariate normal distributions. *Journal of Multivariate Analysis*, 12(3):450–455, 1982.
- [11] Lawrence C Evans. Partial differential equations and Monge-Kantorovich mass transfer. *Current developments in mathematics*, pages 65–126, 1997.
- [12] Uriel Frisch, Sabino Matarrese, Roya Mohayaee, and Andrei Sobolevski. A reconstruction of the initial conditions of the Universe by optimal mass transportation. *Nature*, 417(6886):260–262, 2002.
- [13] Brittany D Froese. A numerical method for the elliptic Monge-Ampère equation with transport boundary conditions. *SIAM Journal on Scientific Computing*, 34(3):1432–1459, 2012.
- [14] Brittany D Froese and Adam M Oberman. Convergent finite difference solvers for viscosity solutions of the elliptic Monge-Ampère equation in dimensions two and higher. *SIAM Journal on Numerical Analysis*, 49(4):1692–1714, 2011.
- [15] Brittany D Froese and Adam M Oberman. Fast finite difference solvers for singular solutions of the elliptic Monge-Ampère equation. *Journal of Computational Physics*, 230(3):818–834, 2011.

- [16] Wilfrid Gangbo. An elementary proof of the polar factorization of vector-valued functions. *Archive for rational mechanics and analysis*, 128(4):381–399, 1994.
- [17] Leslie Greengard and June-Yub Lee. Accelerating the nonuniform fast Fourier transform. *SIAM review*, 46(3):443–454, 2004.
- [18] Steven Haker, Lei Zhu, Allen Tannenbaum, and Sigurd Angenent. Optimal mass transport for registration and warping. *International Journal of Computer Vision*, 60(3):225–240, 2004.
- [19] Angelo Iollo and Damiano Lombardi. A lagrangian scheme for the solution of the optimal mass transfer problem. *Journal of Computational Physics*, 230(9):3430–3442, 2011.
- [20] Peter Laurence, Ricardo J Pignol, and Esteban G Tabak. Constrained density estimation. *Proceedings of the 2011 Wolfgang Pauli Institute conference on energy and commodity trading*, Springer Verlag.
- [21] Peter D Lax. *Linear algebra and its applications*. Wiley, 2013.
- [22] Robert J McCann. A convexity principle for interacting gases. *advances in mathematics*, 128(1):153–179, 1997.
- [23] Robert J McCann and Adam M Oberman. Exact semi-geostrophic flows in an elliptical ocean basin. *Nonlinearity*, 17(5):1891, 2004.
- [24] R Tyrell Rockafellar. *Convex analysis*, volume 28. Princeton university press, 1997.
- [25] Torsten Rohlfing and Calvin R Maurer Jr. Nonrigid image registration in shared-memory multiprocessor environments with application to brains, breasts, and bees. *Information Technology in Biomedicine, IEEE Transactions on*, 7(1):16–25, 2003.
- [26] Gilbert Strang. *Linear Algebra and Its Applications*. Thomson Brooks/Cole, 2006.

- [27] Mohamed M Sulman, JF Williams, and Robert D Russell. An efficient approach for the numerical solution of the Monge-Ampère equation. *Applied Numerical Mathematics*, 61(3):298–307, 2011.
- [28] Esteban G Tabak and Cristina V Turner. A family of nonparametric density estimation algorithms. *Communications on Pure and Applied Mathematics*, 66(2):145–164, 2013.
- [29] Esteban G Tabak and Eric Vanden-Eijnden. Density estimation by dual ascent of the log-likelihood. *Communications in Mathematical Sciences*, 8(1):217–233, 2010.
- [30] Cédric Villani. *Topics in optimal transportation*, volume 58 of *Graduate Studies in Mathematics*. American Mathematical Soc., 2003.
- [31] William R Wade. *An introduction to analysis*. Prentice Hall, 1995.